Institute for Machine Tools and Industrial Management (*iwb*)
TUM School of Engineering and Design
Technical University of Munich

An Autonomous Agent for Parameter Optimization and Simulation-Based Verification in ROS2

Ein autonomer Agent zur Parameteroptimierung und simulationsbasierten Verifikation in ROS2

Scientific work for obtaining the academic degree

Master of Science (M.Sc.)

at the TUM School of Engineering and Design of the Technical University of Munich

Examined by	Prof. Dr. Rüdiger Daub Chair for Production Technology and Energy Storage Systems
Submitted by	Reuben George Steinheilstraße 16 80333 München
Submitted on	January 1, 2024 in Garching

Institute for Machine Tools and Industrial Management (*iwb*)
TUM School of Engineering and Design
Technical University of Munich

Task Description

English Title of the Master's Thesis:

An Autonomous Agent for Parameter Optimization and Simulation-Based Verification in ROS2

German Title of the Master's Thesis:

Ein autonomer Agent zur Parameteroptimierung und simulationsbasierten Verifikation in ROS2

Written by:	B. Sc. Reuben George
Matriculation-No.:	03728047
Degree Program:	M.Sc. Robotics, Cognition and Intelligence
TUM School:	TUM School of Computation, Information and Technology
Advised by:	M. Sc. Sebastian Kurscheid
Time period:	01.12.2025 to 31.05.2026

Initial Situation
Industrial production is becoming more variant-rich, but when robots are introduced to new product variants, process parameters such as acceleration, velocity, gripping force etc. still need to be selected and tuned manually. This is a problem facing the industry, as qualified personnel for robot programming and process tuning is limited. These parameters are not independent: for example, increasing acceleration can improve takt time but may destabilize the grip. Because of these trade-offs, the final decision is often made by trial-and-error or expert knowledge during deployment. This makes the process of validation slow and not scalable.

Modular skill frameworks support task decomposition, skill composition, and execution pipelines, and the skills are often designed as executable modules. While these modules may offer tunable parameters, they often lack clearly defined operating limits, safety constraints, and standardized feedback needed for automated validation and optimization.

Objective
The master thesis's objective is to develop and evaluate an agent, built on ROS 2, MoveIt2 and Isaac Sim, that enables industrial robot skills to be automatically adapted through optimized parameterization. This involves (1) establishing a clear taxonomy that separates task- and process-related parameters from tunable dynamic ones, (2) designing a monitoring layer that extracts meaningful KPIs from simulated executions, and (3) implementing a closed-loop mechanism that adjusts selected dynamic parameters to achieve safe, stable, and efficient task behavior.

Approach and Working Methodology
The following approach is planned to achieve the desired objectives:

- Literature research and presentation of the state of the art in science and technology
- Requirements and criteria definition
- Agent concept development
- Set up the simulation environment and implement the optimizing agent
- Validation of requirements
- Critical evaluation and detailed documentation of the results

Agreement
With the supervision of B. Sc. Reuben George by M. Sc. Sebastian Kurscheid, the intellectual property of the *iwb* is incorporated into this work. Publication of the work or disclosure to third parties requires the permission of the chair holder. I agree to the archiving of the thesis in the *iwb*'s library, which is only accessible to *iwb* employees, and in the *iwb*'s digital thesis database as a PDF document.

Garching, 01.05.2026

Prof. Dr. Rüdiger Daub B. Sc. Reuben George

Abstract

Industrial robot applications must increasingly handle variant-rich production tasks, yet the selection and tuning of process parameters such as acceleration, velocity, and gripping force still often rely on manual trial-and-error and expert knowledge. This limits scalability, especially because parameter choices are interdependent and can involve trade-offs between cycle time, stability, and safety. This thesis develops and evaluates an autonomous agent for parameter optimization and simulation-based verification in ROS 2. Building on MoveIt2 and Isaac Sim, the proposed approach aims to automatically adapt industrial robot skills through optimized parameterization. For this purpose, the thesis distinguishes task- and process-related parameters from tunable dynamic parameters, introduces a monitoring layer for extracting meaningful key performance indicators from simulated executions, and implements a closed-loop optimization mechanism. The resulting agent is intended to support safe, stable, and efficient robot skill execution while reducing the manual effort required for validation and process tuning.

Kurzfassung

Industrieroboter müssen zunehmend vielfältige Produktionsaufgaben bewältigen, doch die Auswahl und Feinabstimmung von Prozessparametern wie Beschleunigung, Geschwindigkeit und Greifkraft beruhen nach wie vor oft auf manuellem Ausprobieren und Expertenwissen. Dies schränkt die Skalierbarkeit ein, insbesondere da die Parameterauswahl von Wechselwirkungen geprägt ist und Kompromisse zwischen Zykluszeit, Stabilität und Sicherheit erfordern kann. In dieser Arbeit wird ein autonomer Agent zur Parameteroptimierung und simulationsbasierten Verifizierung in ROS 2 entwickelt und evaluiert. Aufbauend auf MoveIt2 und Isaac Sim zielt der vorgeschlagene Ansatz darauf ab, die Fähigkeiten von Industrierobotern durch optimierte Parametrisierung automatisch anzupassen. Zu diesem Zweck unterscheidet die Arbeit zwischen aufgaben- und prozessbezogenen Parametern einerseits und einstellbaren dynamischen Parametern andererseits, führt eine Überwachungsebene zur Extraktion aussagekräftiger Leistungskennzahlen aus simulierten Ausführungen ein und implementiert einen Optimierungsmechanismus mit geschlossenem Regelkreis. Der daraus resultierende Agent soll die sichere, stabile und effiziente Ausführung von Roboterfähigkeiten unterstützen und gleichzeitig den manuellen Aufwand für die Validierung und Prozessoptimierung reduzieren.

Contents

List of Abbreviations **xi**

1 Introduction **1**

2 Basics **3**
2.1 Sampling methods . . . 3
2.1.1 Monte Carlo sampling . . . 3
2.1.2 Latin hypercube sampling . . . 4
2.1.3 Quasi-Monte Carlo sampling . . . 4
2.2 The Bayesian optimization problem . . . 5
2.2.1 Gaussian process . . . 7
2.2.2 Acquisition function . . . 8
2.2.3 Constrained bayesian optimization . . . 9
2.3 k-NN and distance measurement . . . 11
2.4 Coulomb friction model . . . 12

3 State of the art **15**
3.1 Taxonomy of skill parameters . . . 15
3.1.1 Interface-role taxonomy: input vs. output parameters . . . 16
3.1.2 Temporal taxonomy: configuration parameters vs. runtime state variables . . . 16
3.1.3 Responsibility and visibility taxonomy: primary, secondary, internal . . . 16
3.1.4 Origin and acquisition taxonomy: manual parameters vs. derived parameters 17
3.1.5 Automation-oriented taxonomy: inference source, completion strategy, verification . . . 18
3.1.6 Policy-structure taxonomy: intrinsic vs. extrinsic parameters . . . 18
3.2 Monitoring . . . 19
3.2.1 Monitoring execution semantics in MoveIt2 . . . 19
3.2.2 Monitoring physical interaction outcomes in Isaac Sim . . . 20
3.2.3 Practical integration and complexity management . . . 21
3.3 Optimization . . . 21
3.3.1 Constrained / safe Bayesian optimization . . . 21
3.3.2 Contextual Bayesian optimization in robotics . . . 23
3.3.3 Evolutionary and population-based optimization . . . 24
3.3.4 Case-based reasoning . . . 25

4 Design **29**
4.1 Design requirements . . . 29
4.1.1 Functionality . . . 29
4.1.2 Verification . . . 30
4.1.3 Safety and feasibility . . . 30
4.1.4 Optimization . . . 31

4.1.5 Adaptivity and generalization . . . 31
4.1.6 Requirements traceability . . . 32
4.2 Parameter taxonomy . . . 33
4.2.1 Context and decision variables . . . 33
4.2.2 Context Variables . . . 33
4.2.3 Decision variables . . . 34
4.2.4 Evaluation metrics . . . 34
4.3 Problem formulation . . . 35
4.3.1 Nominal constrained problem . . . 35
4.3.2 Constraint handling under surrogate uncertainty . . . 36
4.4 Monitoring loop . . . 37
4.5 Optimization architecture . . . 41
4.5.1 Offline dataset . . . 41
4.5.2 Online decision cycle . . . 42

5 Implementation **47**
5.1 System architecture . . . 47
5.1.1 Skill trial runner . . . 49
5.1.2 Hardware choice . . . 49
5.2 The pick-and-place skill . . . 51
5.3 Skill trial runner . . . 54
5.3.1 Execution life cycle: `run_cycle()` . . . 54
5.3.2 Data contract: `SkillTrialResult` . . . 56
5.3.3 Integration with ROS2 . . . 56
5.4 Monitoring: data nodes . . . 57
5.4.1 Energy monitor . . . 58
5.4.2 Slip-detection node . . . 59
5.4.3 Summary . . . 60
5.5 Offline context sweep . . . 60
5.5.1 Sobol sampling . . . 61
5.5.2 Context sweep execution . . . 61
5.5.3 Recording the data schema in CSV . . . 62
5.6 Feasibility gates . . . 63
5.6.1 Physics and input gate . . . 63
5.6.2 Context feasibility gate . . . 64
5.7 Warm start . . . 64
5.7.1 k-Nearest-neighbor lookup . . . 64
5.7.2 Warm-start computation . . . 65
5.8 Constrained Bayesian optimization loop . . . 65
5.8.1 Ax/BoTorch integration . . . 66
5.8.2 Seed phase . . . 67
5.8.3 Failure penalty scheme . . . 68
5.8.4 Early stopping . . . 68
5.8.5 Policy extraction . . . 69
5.9 Case-based reasoning: retain and retrieve . . . 70
5.9.1 Case-based storage . . . 70
5.9.2 Lookup logic . . . 70
5.9.3 Duplicate detection . . . 71
5.9.4 Closing the four-R cycle . . . 71

6 Evaluation **75**

6.1 Evaluation overview . . . 75
6.2 Input rejection . . . 76
6.3 Static-hold rejection . . . 77
6.4 Infeasibility transparency due to cycle time constraint . . . 77
6.5 Constrained optimization on admissible contexts . . . 78

7 Conclusion and future outlook **79**
7.1 Requirements fulfillment . . . 79
7.2 Assessment of the agent . . . 80
7.3 Future outlook . . . 81

Bibliography **83**

AI-Based Tools **89**

Glossary **91**

A Appendix **93**
A.1 Online Optimization Trials . . . 93
A.1.1 Test case 1 . . . 93
A.1.2 Test case 2 . . . 94
A.1.3 Test case 7 . . . 95
A.1.4 Test case 9 . . . 96
A.1.5 Test case 10 . . . 97

List of Abbreviations

BO	Bayesian Optimization
cBO	Constrained Bayesian Optimization
CBR	Case Based Reasoning
EI	Expected Improvement
GP	Gaussian Process
HMLV	High-Mix Low-Volume
IMU	Inertial Measurement Unit
KM	Knowledge model
KNN	K-Nearest Neighbors
KPI	Key Performance Indicator
KPIs	Key Performance Indicators
LHS	Latin Hypercube Sampling
LineBO	Line Bayesian Optimization
MCS	Monte Carlo Sampling
MTC	MoveIt Task Constructor
NSGA-II	Non-dominated Sorting Genetic Algorithm II
QMCS	Quasi-Monte Carlo Sampling
ROS	Robot Operating System
ROS2	Robot Operating System 2
SafeOpt	Safe Bayesian Optimization
TCP	Tool Center Point

Chapter 1 Introduction

High-Mix Low-Volume (HMLV) manufacturing has changed the requirements for modern automation systems. Conventional production lines with fixed procedures and stationary industrial robots often lack the flexibility needed to meet these new demands (Simon Bøgh et al., 2012). A possible solution is to develop more adaptable robots that can respond to new scenarios and perform a wider range of tasks. This is particularly important in small- and medium-volume production, where new and unique parts often need to be transported between workstations and machines. As a result, robots must be able to operate reliably despite uncertainty in environments.

Skill-based robotics has emerged as a promising approach for reducing the mismatch between the flexibility required in modern production and the engineering effort traditionally associated with industrial robot deployment (Heuss et al., 2022). In HMLV manufacturing, robot applications must be adapted frequently to new products, variants, and task conditions. Conventional robot programming, however, often requires expert knowledge and remains time-consuming, which limits the scalability of automation in such environments. Skill-based approaches address this issue by encapsulating robot capabilities into reusable functional units. In this way, skill-based programming supports a transition from product-specific robot programs toward configurable robot capabilities.

Such structures are important as they help make robot behavior more explicit and connected to a task description, combining world-state reasoning and automated planning. Different product variants can be handled by the same skill structure, provided that the correct task and process parameters are available. Kurscheid and Daub (2024) connect this idea directly to HMLV manufacturing by arguing that robot skills can be adapted to new variants or products through the right parameterization.

However, the existence of parameterized skills does not automatically solve the problem of selecting suitable parameter values. A skill may expose parameters such as target poses, approach directions, gripping force, acceleration, velocity, contact force, tolerances, or controller gains. Some of these parameters define the task itself, whereas others determine how the task is executed dynamically. In practice, these parameters are often coupled and therefore cannot be chosen independently. For example, increasing acceleration may reduce execution time, but it also increases the risk of the object slipping. As a result, skill parameterization is not merely a configuration step, but a trade-off between performance, safety, stability, and efficiency.

This limitation has already been identified in previous work. Albore et al. (2023) note that the implementation of skills can still require laborious parameterization and manual tuning, while A. Björkelund et al. (2011) emphasize that the reuse of parameters, controls, and process tuning becomes increasingly important as robot setup and programming become more challenging. Mayr et al. (2023) also points toward this issue through learning-based use cases in which skill parameters that cannot be fully specified beforehand are automatically identified and optimized through learning procedures. These observations indicate that parameterization alone is insufficient when successful execution depends on interacting process variables and task-specific constraints.

Therefore, this thesis recognizes this challenge by not only defining the parameter taxonomy needed for correct skill execution but also optimizing them. As seen in Figure 1.1, the thesis builds on

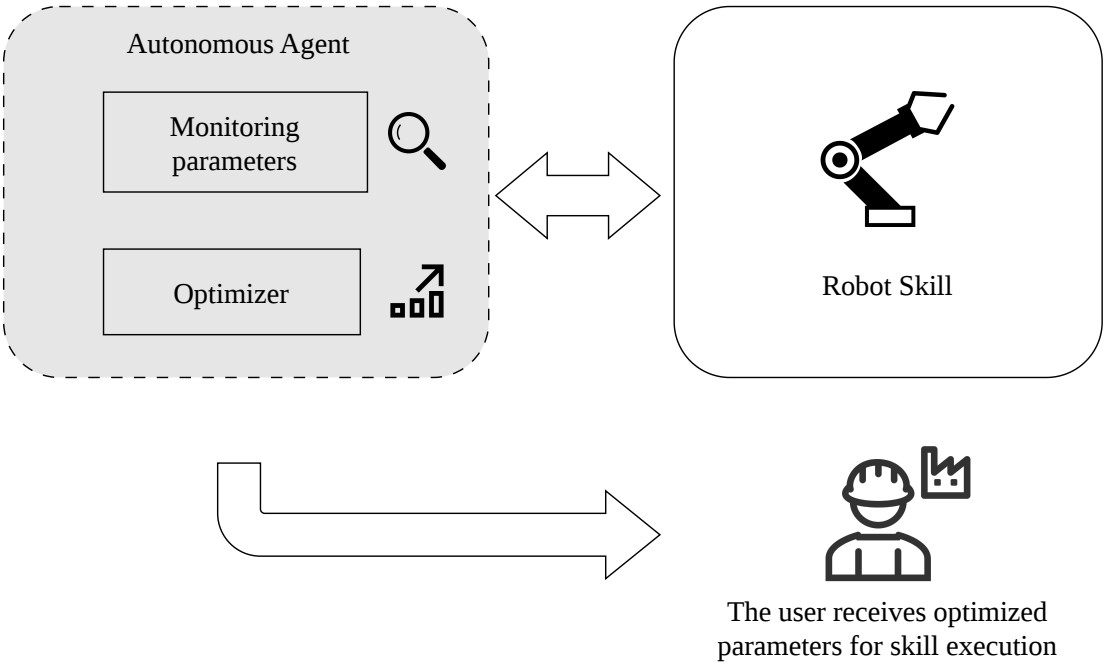


Figure 1.1: Core contribution of the thesis.

skill-based robotics, by developing a Robot Operating System (ROS)-2 agent that classifies skill parameters, monitors relevant Key Performance Indicators (KPIs) in simulation, and closes the loop by optimizing selected parameters. The goal is to support safe, stable, and efficient skill execution while reducing the manual effort required for deploying robot skills in HMLV manufacturing environments.

Chapter 2 Basics

2.1 Sampling methods

Sampling methods are used to represent uncertainty in a model by selecting a finite set of input combinations. In uncertainty analysis, parameters are usually described by probability distributions. A sampling method then generates combinations of these uncertain inputs, runs the deterministic model on those combinations, and uses the resulting outputs to estimate quantities such as means, variances, percentiles, or failure probabilities (Kucherenko et al., 2015). The resulting model can be further used for the optimization loop as seen in Figure 2.1. The central difficulty is that each model evaluation can be computationally expensive; therefore, the efficiency of the sampling method directly affects the practicality and reliability of the analysis (Hou et al., 2019).

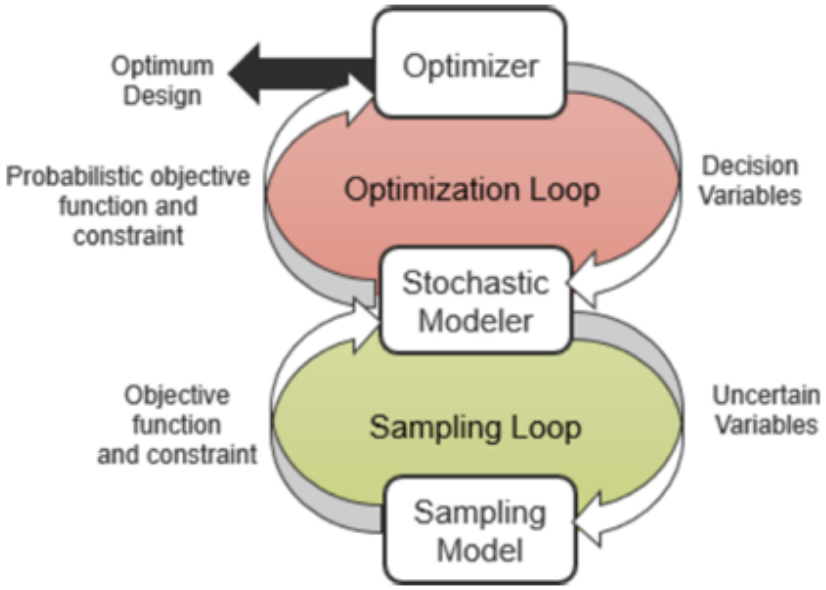


Figure 2.1: The role of a sampling method in an optimization framework. (Dige & Diwekar, 2018).

2.1.1 Monte Carlo sampling

The simplest and most widely used approach is Monte Carlo Sampling (MCS). In this method, input values are generated using pseudo-random numbers, usually from a uniform distribution, and then transformed into the required probability distributions. The model output is approximated by averaging the results over all sampled points. Monte Carlo sampling is attractive because it is simple, robust, and broadly applicable. Its convergence rate is proportional to $N^{-1/2}$, where N is the

number of samples. A notable feature of this rate is that it does not directly depend on the number of input dimensions. However, the convergence is slow, and pseudo-random points can cluster in some regions while leaving other regions undersampled. As a result, a large number of samples may be required before the sampled input space becomes sufficiently representative (Kucherenko et al., 2015).

The quantity of interest is typically an expectation, which on the unit hypercube can be written as

$$Q = \mathbb{E}[f(\boldsymbol{x})] = \int_{[0,1]^d} f(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}, \tag{2.1}$$

and MCS approximates it by the sample mean

$$\hat{Q}_N = \frac{1}{N}\sum_{i=1}^{N} f(\boldsymbol{x}_i). \tag{2.2}$$

The root-mean-square error of this estimator scales as

$$\varepsilon_{\mathrm{MC}} = \frac{\sigma_f}{\sqrt{N}}, \qquad \sigma_f^2 = \mathrm{Var}[f(\boldsymbol{x})], \tag{2.3}$$

which makes the $N^{-1/2}$ convergence and its independence from the dimension d explicit.

2.1.2 Latin hypercube sampling

Latin Hypercube Sampling (LHS) improves upon crude MCS by enforcing stratification in each input dimension (Dige & Diwekar, 2018). For a sample size N, the range of each input variable is divided into N equally probable intervals. One value is sampled from each interval, and the sampled values from different variables are randomly paired to form multidimensional sample points. This ensures that every marginal distribution is well covered, which often reduces variance compared with ordinary Monte Carlo sampling. LHS is therefore useful when good one-dimensional coverage of each uncertain parameter is desired. In fact, Kucherenko et al. (2015) report that LHS can be more efficient than both Monte Carlo and Quasi-Monte Carlo Sampling (QMCS) sampling, although this advantage holds only for a restricted class of functions and at small sample sizes. However, standard LHS does not guarantee good uniformity in the full multidimensional space. Even though each variable is individually well stratified, the combined points may still show gaps or clustering when viewed in two or more dimensions. Optimized LHS variants attempt to improve this space-filling behavior, but they can become computationally expensive and are less convenient when samples need to be added sequentially (Dige & Diwekar, 2018).

2.1.3 Quasi-Monte Carlo sampling

QMCS replaces pseudo-random numbers with deterministic low-discrepancy sequences. These sequences are designed to fill the unit hypercube more uniformly than random points. Common QMCS sequences include Sobol, Halton, Hammersley, Niederreiter–Xing, and lattice sequences (Blatman et al., 2007). Instead of relying on randomness, QMCS methods aim to reduce the discrepancy of the sample set. Discrepancy is a measure of how far the empirical distribution of sample points deviates from an ideal uniform distribution. A lower-discrepancy design generally

gives better space-filling behavior and can hence reduce error (Kucherenko et al., 2015). For low-discrepancy sequences such as Sobol sequences, the star discrepancy satisfies

$$D_N^* = O\left(\frac{(\log N)^d}{N}\right), \tag{2.4}$$

where D_N^* quantifies the deviation of the sample set from a perfectly uniform distribution and N is the number of samples. This bound makes explicit how space-filling quality depends jointly on the sample size and the dimension d.

Figure 2.2 shows why, among sampling methods, Sobol sequences are especially popular: they are constructed to give good uniformity in multidimensional spaces, particularly for sample sizes based on powers of two. They are digital low-discrepancy sequences and are often preferred in engineering and financial applications because they provide more even coverage of the input domain (Hou et al., 2019). This is why sampling based on Sobol sequences can be more effective than Monte Carlo sampling: Monte Carlo points are random and may cluster, whereas points generated from Sobol sequences are deliberately arranged to fill the space more evenly. The theoretical error of QMCS methods is often written in terms of a low-discrepancy bound, commonly of the form $O\left(N^{-1}(\log N)^d\right)$, compared with the $O\left(N^{-1/2}\right)$ rate of Monte Carlo (Kucherenko et al., 2015). In practice, this means that sampling based on Sobol sequences can achieve the same accuracy with fewer model evaluations. Hou et al. (2019) confirm this for a range of engineering applications, finding that QMCS generally outperforms standard Monte Carlo, with the size of the advantage depending on the smoothness of the model response and the number of dominant parameters.

Curse of dimensionality

The curse of dimensionality refers to the rapid growth of the input space as the number of uncertain variables increases, making it increasingly difficult to achieve representative coverage with a finite number of samples. For QMCS methods, this challenge appears in the theoretical error bound through the term $O\left(N^{-1}(\log N)^d\right)$, where the logarithmic factor grows with the dimension d. However, Sloan and Woźniakowski (1998) show that the practical performance of QMCS depends less on the nominal dimension and more on the effective dimension of the problem. Consequently, Sobol sequences and other QMCS methods can still provide substantial advantages over MCS in high-dimensional problems when only a limited number of variables or low-order interactions dominate the model response. In contrast, when many variables contribute equally, or the response function is highly non-smooth, the benefits of QMCS are reduced (Hou et al., 2019).

2.2 The Bayesian optimization problem

Bayesian Optimization (BO) is designed for optimization problems in which the objective function is expensive to evaluate, has no simple analytic form, and is observed only through a limited number of experiments or simulations (Frazier, 2018). In its basic form, the goal is to find an input configuration that maximizes an unknown objective function over a feasible search space:

$$x^\star \in \arg\max_{x \in \mathcal{X}} f(x) \tag{2.5}$$

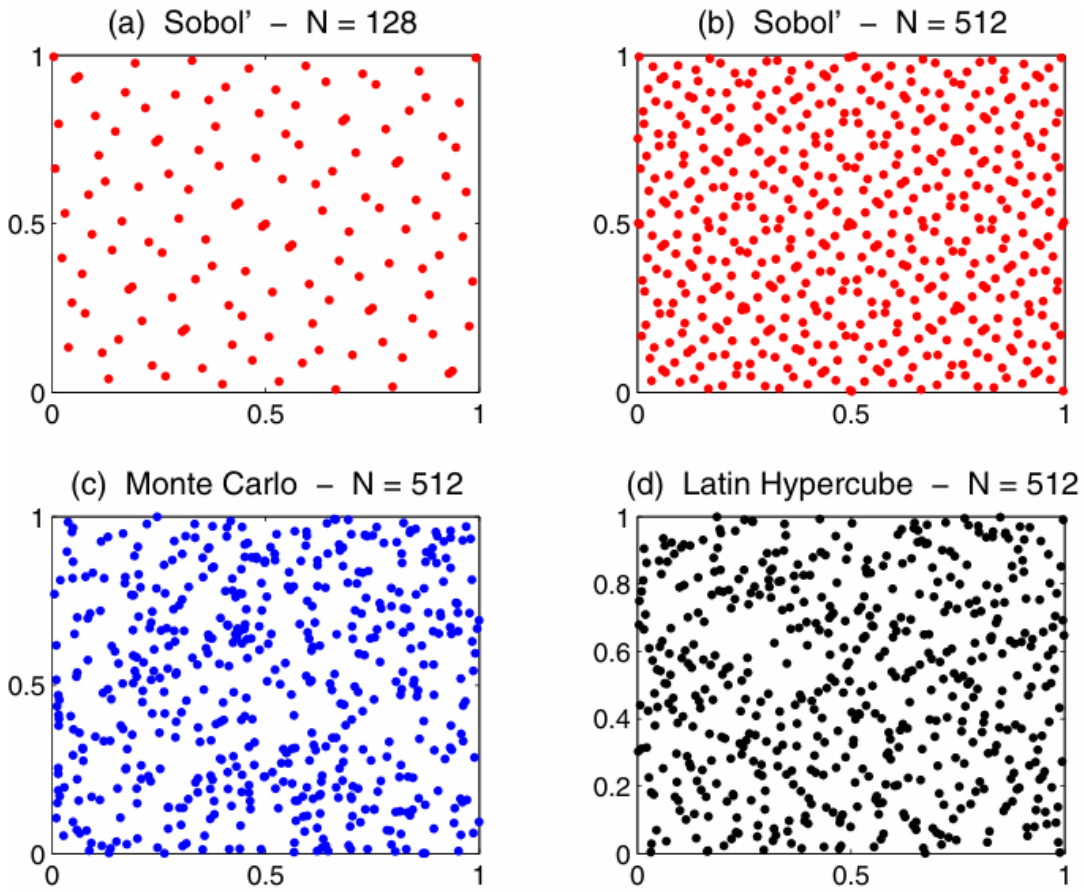


Figure 2.2: Space-filling properties of a two-dimensional Sobol sequence compared with MCS and LHS (Blatman et al., 2007).

Here, x denotes a candidate design, configuration, or decision variable, $\mathcal{X}$ is the search space, and $f(x)$ is the true but unknown objective value. The defining difficulty is that f is treated as a black-box function: it can be queried at selected points, but its internal structure is not available in a form that would allow direct optimization (Frazier, 2018). In many settings, each query may correspond to training a model, running a simulation, carrying out an experiment, or evaluating a costly engineering design. BO is therefore especially useful when the evaluation budget is small relative to the size or complexity of the search space (Shahriari et al., 2016).

In the noisy setting, the optimizer does not observe $f(x)$ directly. Instead, querying the objective at a point x_n returns a noisy observation y_n, for example

$$y_n = f(x_n) + \varepsilon_n, \qquad \mathbb{E}[\varepsilon_n] = 0 \tag{2.6}$$

or, under the common Gaussian-noise assumption,

$$\varepsilon_n \sim \mathcal{N}(0, \sigma_\varepsilon^2) \tag{2.7}$$

This formulation captures the idea of an expensive, noisy oracle: the function can be sampled, but each sample is costly and imperfect. After n evaluations, the available information is summarized by the dataset

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n \tag{2.8}$$

2.2.1 Gaussian process

The surrogate model in BO is commonly chosen to be a Gaussian Process (GP), because it provides a probabilistic model over unknown functions rather than only a point prediction (Garnett, 2023). Following the standard treatment of Rasmussen and Williams (2005), a GP is defined by a mean function and a covariance function, which in the BO setting allows the unknown objective function to be represented as

$$f \sim \mathcal{GP}(m(\cdot), k(\cdot,\cdot)) \tag{2.9}$$

where $m(\cdot)$ denotes the prior mean function and $k(\cdot,\cdot)$ denotes the covariance function, or kernel. The kernel encodes assumptions about similarity between inputs: points that are close in the search space, or similar according to the chosen kernel, are expected to have related function values. This makes the kernel a central modeling choice, because it determines the smoothness and correlation structure assumed for the unknown objective.

After observing a dataset $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, the prior distribution is updated to a posterior distribution over functions. In the common noisy-observation setting, the observed output is modelled as

$$y_i = f(x_i) + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma_\varepsilon^2) \tag{2.10}$$

The noise term accounts for stochastic evaluations, numerical variability, or measurement error. In matrix form, the Gaussian likelihood leads to posterior predictions whose covariance depends on the kernel matrix plus a noise term:

$$K_y = K + \sigma_\varepsilon^2 I \tag{2.11}$$

For a new candidate point x, the GP posterior provides both a predictive mean and a predictive variance:

$$\mu_n(x) = m(x) + k(x,X)K_y^{-1}\big(y - m(X)\big) \tag{2.12}$$

$$\sigma_n^2(x) = k(x,x) - k(x,X)K_y^{-1}k(X,x) \tag{2.13}$$

where $X = \{x_i\}_{i=1}^n$, $\mathbf{y} = (y_1, \ldots, y_n)^\top$, and K_y includes the observation-noise term. The predictive mean $\mu_n(x)$ represents the model's current estimate of the objective at x, while the predictive variance $\sigma_n^2(x)$ quantifies uncertainty. This combination is what makes the GP particularly suitable for BO: the model can identify regions that appear promising while also indicating where additional evaluations may be informative.

2.2.2 Acquisition function

The acquisition function is the decision rule that uses the Gaussian-process posterior to select the next evaluation point. Rather than optimizing the expensive objective function directly, BO optimizes a cheaper acquisition function that depends on the current posterior mean and uncertainty. At iteration n, the next point is chosen as

$$x_{n+1} \in \arg\max_{x \in \mathcal{X}} \alpha_n(x) \tag{2.14}$$

where $\alpha_n(x)$ denotes the acquisition function. Its purpose is to balance exploitation and exploration: exploitation favours points with high predicted objective values, while exploration favours points with high uncertainty (Greenhill et al., 2020). In this way, the acquisition function converts the probabilistic surrogate model into an actionable sampling decision (Shahriari et al., 2016).

A standard acquisition function is Expected Improvement (EI). In a maximization problem with noise-free observations, EI measures the expected amount by which evaluating a candidate point improves over the current best observed value $f_n^\star$:

$$\mathrm{EI}_n(x) = \mathbb{E}_n\left[\max(0, f(x) - f_n^\star)\right] \tag{2.15}$$

This criterion is attractive because it naturally combines the posterior mean and variance: a point can have high EI either because its predicted value is high, or because uncertainty leaves open the possibility of a large improvement (Frazier, 2018). The combination of a GP surrogate with the EI criterion has been shown to perform well across a range of expensive optimization tasks, provided the surrogate model and its hyperparameters are specified with care (Snoek et al., 2012). However, in the noisy setting, the current best value is not known exactly. The observations y_i are noisy estimates of the latent objective values $f(x_i)$, so simply comparing against the best observed y_i can lead to misleading or overly exploitative decisions.

This motivates noisy variants of EI. Letham et al. (2019) formulate noisy EI by integrating over the posterior distribution of the unknown true objective values at the previously evaluated points. In simplified form, the acquisition can be understood as

$$\mathbb{E}_{f_1 \mid \mathcal{D}_n}\left[\mathrm{EI}_n(x \mid f_1)\right] \tag{2.16}$$

where $f_1 = (f(x_1), \ldots, f(x_n))$ denotes the latent, noise-free objective values at the observed locations. Instead of treating the current best value as fixed, noisy EI averages the improvement calculation over plausible latent objective values under the posterior. This makes the acquisition function better aligned with stochastic experiments, where uncertainty about previous evaluations should influence the next sampling decision.

Letham et al. (2019) further extend this idea to noisy constraints and batch optimization, using quasi-Monte Carlo approximation to make the required posterior integration computationally practical. For the present work, the key point is that acquisition functions should be chosen consistently with the observation process. If the objective evaluations are noisy, then the acquisition function should account not only for uncertainty at new candidate points, but also for uncertainty in the values already observed. This is why noisy EI provides a natural bridge between the basic BO loop and later extensions involving constrained optimization.

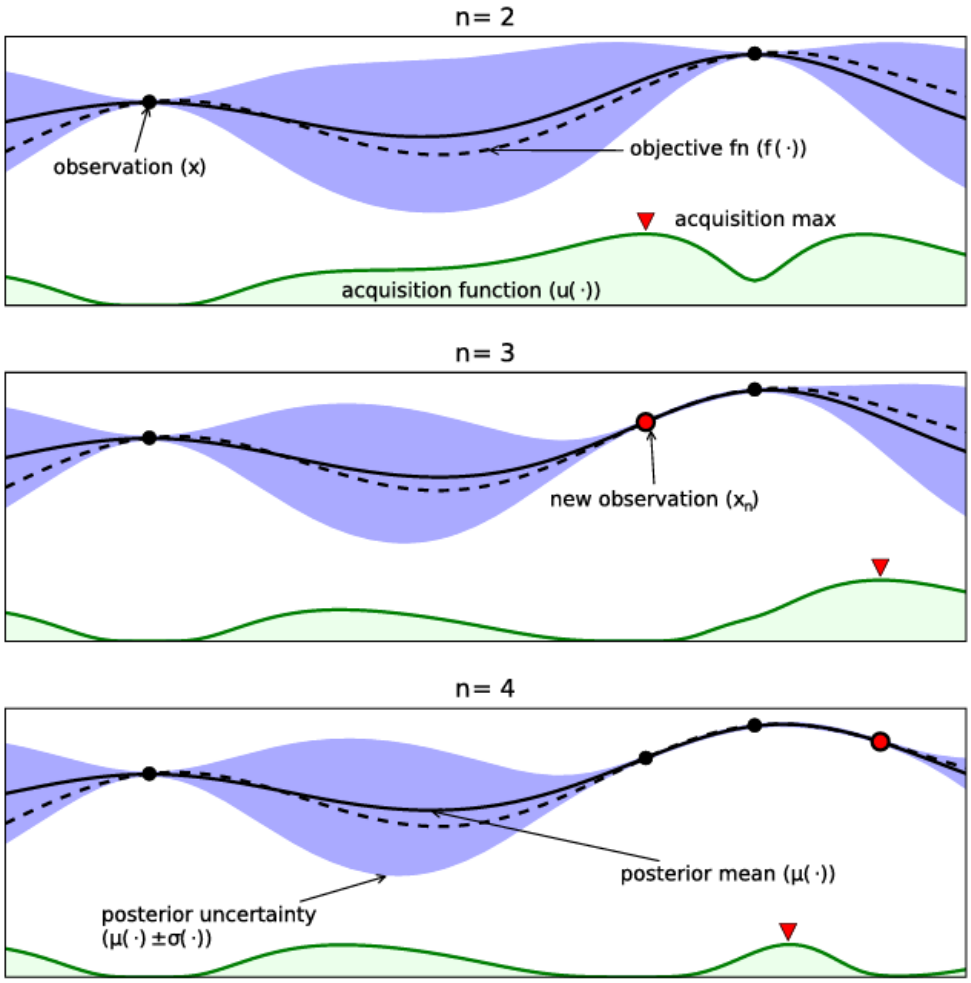


Figure 2.3: The BO loop over three iterations (n = 2, 3, 4). (Shahriari et al., 2016).

Figure 2.3 illustrates the BO loop over three iterations ($n = 2, 3, 4$). Each upper plot shows the true objective $f(\cdot)$ (solid), the GP posterior mean $\mu(\cdot)$ (dashed), and the posterior uncertainty $\mu(\cdot) \pm \sigma(\cdot)$ (shaded); the uncertainty vanishes at observed points and grows in between. Each lower plot shows the acquisition function $u(\cdot)$, whose maximizer (red triangle) determines the next evaluation. The new observation x_n (red) collapses the local uncertainty before the surrogate and acquisition are recomputed (Shahriari et al., 2016).

2.2.3 Constrained bayesian optimization

The formulation in the previous subsections assumes that every point in the search space $\mathcal{X}$ is a valid candidate. In many engineering problems this assumption does not hold: a configuration may be physically realizable yet violate requirements on safety, quality, or resource usage, and whether it does so is often not known until the configuration has been evaluated. Such problems are naturally expressed as constrained optimization, in which the objective is optimized only over the subset of the search space that satisfies a set of constraints (Greenhill et al., 2020):

$$x^\star \in \arg\max_{x \in \mathcal{X}} f(x) \quad \text{subject to} \quad c_j(x) \leq 0, \quad j = 1, \ldots, J \tag{2.17}$$

Here each $c_j(x)$ is a constraint function, and the feasible region is the set of points for which all constraints are satisfied. A key difficulty in the black-box setting is that the constraint functions, like the objective, are typically unknown in closed form and must themselves be learned from observations. A common approach is therefore to place an independent GP prior on each constraint function in addition to the objective, yielding for every c_j a posterior mean $\mu_{c_j}(x)$ and variance $\sigma^2_{c_j}(x)$ (Gardner et al., 2014).

Given these posteriors, the central quantity is the probability that a candidate is feasible. Assuming the constraints are conditionally independent given x, the probability of feasibility is the product of the individual feasibility probabilities:

$$\mathrm{PF}(x) = \prod_{j=1}^{J} \mathbb{P}\big(c_j(x) \le 0\big) = \prod_{j=1}^{J} \Phi\left(\frac{-\mu_{c_j}(x)}{\sigma_{c_j}(x)}\right) \tag{2.18}$$

where $\Phi(\cdot)$ is the standard normal cumulative distribution function. Gardner et al. (2014) use this quantity to extend EI to constrained problems by weighting the unconstrained acquisition with the probability of feasibility. Letting $f_c^\star$ denote the best feasible observation, the constrained EI of a candidate is

$$\mathrm{EIC}_n(x) = \mathrm{EI}_n(x \mid f_c^\star)\ \mathrm{PF}(x) \tag{2.19}$$

so that a point is only attractive if it both promises improvement over the best feasible value and is likely to be feasible. Infeasible regions are suppressed because their feasibility probability tends to zero, while the objective continues to drive the search inside the feasible region.

This formulation assumes that the best feasible value $f_c^\star$ can be identified, which requires knowing with certainty which observations are feasible. Gelbart et al. (2014) address the more general case in which the constraints are unknown a priori and may be observed independently of the objective; when no observation can be confirmed feasible, they fall back to selecting the candidate that maximizes the probability of feasibility. As with the objective, noise in the constraint observations makes feasibility itself uncertain, so that $f_c^\star$ is no longer well defined. Letham et al. (2019) resolve this by treating the constraints in the same Bayesian manner as noisy objectives. The constrained acquisition is integrated over the posterior of the latent objective and constraint values at the observed points rather than relying on a single plug-in estimate of the best feasible value. This yields a constrained, noise-aware acquisition function that is consistent with the noisy-observation model introduced earlier in this section and that extends naturally to the batch setting.

2.3 k-NN and distance measurement

The k-nearest neighbors (k-NN) algorithm is one of the oldest and most intuitive methods in machine learning. It was first formalized by Fix and Hodges (1951) and given its theoretical foundation by Cover and Hart (1967), and it rests on a single assumption: objects that lie close to one another in some feature space tend to be similar, and therefore tend to share the same label or value. Syriopoulos et al. (2025) describe it as a simple yet powerful non-parametric classifier that is robust to noisy data and easy to implement.

Two properties define the method. It is non-parametric, meaning it makes no assumption about the form of the underlying data distribution and does not fit a fixed set of parameters to summarize the data. It is also instance-based, or "lazy", meaning it builds no explicit model during a training phase. Instead, it simply stores the training examples and defers all computation to the moment a prediction is requested. Training is therefore trivial; the cost is paid at prediction time.

The procedure is straightforward. Given a query point $\mathbf{x}_q$ and a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, the algorithm computes the distance between the query and every training point, ranks the points by that distance, and keeps the k closest, which together form the neighborhood $N_k(\mathbf{x}_q)$. For a classification task, the predicted label is the majority class among those neighbours,

$$\hat{y}(\mathbf{x}_q) = \arg\max_{c} \sum_{\mathbf{x}_i \in N_k(\mathbf{x}_q)} \mathbb{1}(y_i = c), \tag{2.20}$$

where $\mathbb{1}(\cdot)$ is the indicator function, equal to 1 when its argument is true and 0 otherwise. For a regression task, the prediction is usually the mean of the neighbors' values,

$$\hat{y}(\mathbf{x}_q) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x}_q)} y_i. \tag{2.21}$$

Because every prediction is built by comparing the query against the stored examples, the operation that most determines the outcome is the measurement of distance.

Distance measurement

The informal idea of "nearness" must be made precise through a distance function. The most widely used family is the Minkowski distance, defined for two points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ as

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}. \tag{2.22}$$

The single parameter p recovers several familiar metrics as special cases. For $p = 1$ it gives the Manhattan (city-block) distance, the sum of absolute differences along each dimension. For $p = 2$ it gives the Euclidean distance, the ordinary straight-line distance. As $p \to \infty$ it tends to the Chebyshev distance, $\max_i |x_i - y_i|$, which is simply the largest difference along any one dimension. One expression, therefore, unifies the most common choices under a single parameter (Rodrigues, 2018).

Abu Alfeilat et al. (2019) review a large number of measures across several real-world datasets, and find that classifier performance depends significantly on the distance used, with sizeable gaps between the best- and worst-performing measures. Rodrigues (2018) reached a similar conclusion in a survey spanning many datasets: no single metric is universally best, but some perform well more often than others. The practical lesson is that the distance function deserves to be treated as a genuine design decision, comparable in importance to the value of k.

A closely related concern is feature scaling. Because most distance functions sum contributions across dimensions, a feature expressed on a large numerical range will dominate one expressed on a small range, regardless of its true relevance. It is therefore standard practice to normalize or standardize the features before applying k-NN, so that each dimension contributes comparably to the computed distance.

Choosing k and weighting neighbors

The parameter k controls a bias–variance trade-off. A small k makes each prediction sensitive to individual points, and therefore to noise, producing high variance. A large k averages over a wider region, which smooths the prediction but risks blurring genuine boundaries, producing high bias. In practice, k is chosen empirically, for example by cross-validation, rather than fixed in advance.

A common refinement is to weight the neighbors by proximity, so that closer points have greater influence than more distant ones. A typical scheme assigns each neighbor a weight inversely proportional to its squared distance,

$$w_i = \frac{1}{d(\mathbf{x}_q, \mathbf{x}_i)^2}, \tag{2.23}$$

and then forms a weighted vote (for classification) or a weighted average (for regression). Weighting also mitigates a well-known failure mode: Sun and Chen (2021) note that on imbalanced data, the plain majority-vote rule tends to favor the more frequent class, and that distance weighting and related modifications are among the standard remedies.

2.4 Coulomb friction model

When a parallel-jaw gripper holds an object, nothing physically attaches the object to the fingers. The only thing keeping it from sliding out is friction at the contact patches, and that friction is generated by the gripper squeezing the object. The question this subsection answers is a simple one, how hard must the gripper squeeze so that the object does not slip? The classical answer comes from the Coulomb model of friction.

Coulomb's law states that the maximum friction force available at a contact is proportional to the normal force pressing the surfaces together, $F_t \leq \mu F_n$, where F_n is the clamping force, F_t the friction force, and μ the friction coefficient of the finger–object material pair. The coefficient μ sets the "exchange rate" between how hard the gripper presses and how much holding force it gets back. A parallel-jaw gripper applies this at two contacts, one per finger; more generally, for a gripper with n contacting fingers each pressing with force F_{grip}, the total available friction is

$$F_{friction} = n\mu F_{grip}.$$

The other half of the problem is the force the task actually demands. A robot does not hold objects still — it accelerates them through pick-and-place motions, and during those motions the object experiences inertial forces on top of its own weight. Martucci et al. (2020) make this point directly: the forces on a grasped object grow with its acceleration, and high accelerations can drive the object to slip. Nakazawa et al. (2001) studying how humans modulate grip force under fluctuating load, model exactly this situation (Figure 2.4) and show that the required clamping force scales with the object's acceleration:

$$F_G = \frac{m_o (g + a)}{2 \mu_s},$$

where m_o is the object mass, g gravitational acceleration, a the object's acceleration, and μ_s the static friction coefficient (Nakazawa et al., 2001). The intuition is that when the gripper accelerates the object, the contacts must support both the weight and the inertial term, so the worst-case demand combines them additively as $g + a$.

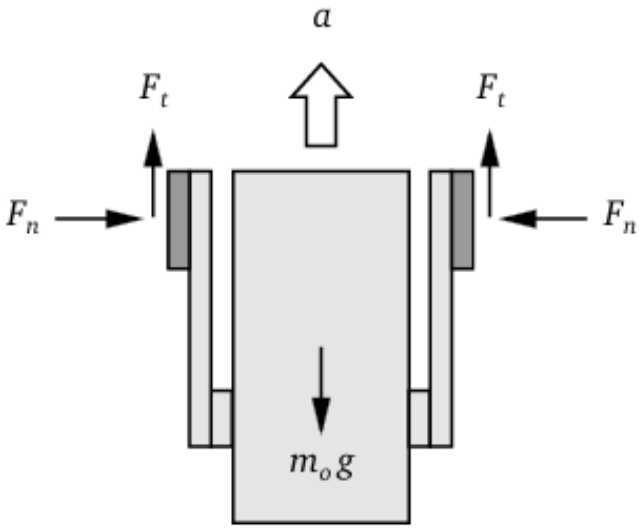


Figure 2.4: Model for grasping an object considering the influence of acceleration (Nakazawa et al. 2001).

Reading this as the demand side, the required holding force can be written as

$$F_{required} = S\, m (g + a),$$

which isolates the physical load $m(g + a)$ from a dimensionless safety factor S. Real grasps are never run at the exact slip threshold since contact geometry is imperfect, μ is only known approximately, and a grasp sitting right at the edge is one disturbance away from failing. The factor $S > 1$ over-provisions the holding force to keep a margin against slip, mirroring the safety margin that Nakazawa et al. (2001) observe in human grasping.

This can be reduced to a single checkable condition: a grasp does not slip as long as the friction the gripper can generate is at least the friction the task demands,

$$\underbrace{n \mu F_{grip}}_{\text{available friction}} \geq \underbrace{S\, m (g + a)}_{\text{required holding force}} .$$

Everything on the left is fixed by the gripper (n, μ) or commanded (F_{grip}); everything on the right is set by the task (m, a, S). If the available friction is greater, the grasp is safe.

Chapter 3 State of the art

This chapter reviews the state of the art in three principal areas. First, Section 3.1 surveys robot skill parameter taxonomies, with a focus on how the literature classifies the parameters required to define and reuse skills. Second, Section 3.2 discusses monitoring methods for the thesis's software stack (ROS 2, MoveIt 2, and Isaac Sim) to obtain robust, episode-level KPIs for closed-loop evaluation. Finally, Section 3.3 reviews optimization algorithms that fit the thesis requirements, spanning safe/constrained Bayesian optimization and population-based multi-objective methods.

3.1 Taxonomy of skill parameters

Robot skills are a widely adopted abstraction for making industrial robot programming more modular and reusable (Andersen et al., 2014; Pedersen et al., 2016; Simon Bøgh et al., 2012; Steinmetz & Weitschat, 2016). In skill-based approaches, a recurring bottleneck is *parameterisation*: supplying concrete parameter values so that a reusable skill fits a specific process.

The reviewed literature does not converge on a single, universal parameter taxonomy. Across the surveyed approaches, however, a small set of recurring and largely orthogonal classification axes can be identified. Each answers a different underlying question about a parameter:

- Who is expected to specify it (operator, expert, or system): captured by the primary, secondary, internal visibility split (Steinmetz & Weitschat, 2016), by the distinction between manually defined and system-derived parameters (Pantano et al., 2022), and by user gap-filling for missing inputs (Kurscheid & Daub, 2024).
- When it is provided (design time vs. run time): inputs configured before execution versus outputs produced during it (Andersen et al., 2014; Profanter et al., 2019), and configuration parameters versus the evolving runtime world state (Pedersen et al., 2016).
- How interpretable it is (semantic, task-level choices vs. low-level controller tuning): the interpretable, non-interpretable contrast (Pantano et al., 2022) and the intrinsic policy structure versus extrinsic execution tuning (Ahmad et al., 2024; Ahmad et al., 2023).
- How it connects to skill composition (interface role, pre-/post-conditions, and data flow): the input/output parameter sets of a skill interface (Andersen et al., 2014; Profanter et al., 2019) and the conditioning of execution on the modeled world state (Pedersen et al., 2016).

The following subsections expand on these taxonomies.

3.1.1 Interface-role taxonomy: input vs. output parameters

A practical and widely used way to classify parameters is by their role in a skill's interface: some parameters are provided as inputs to configure the skill before or at the start of execution, while others are produced as outputs during execution to expose results. In OPC UA skill modelling, Profanter et al. (2019) claim that this separation is made explicit by defining optional input-parameter sets for configuration and optional output-parameter sets for representing the execution outcome. These parameters are typically grouped into a parameter set within the OPC UA address space and written by a client before execution is triggered.

A similar taxonomy is introduced by Andersen et al. (2014), where primitive skills are characterized by sets of input and output parameters: inputs specify the intended behavior of an action, whereas outputs capture run-time information that can be shared with downstream actions. Formally, Andersen et al. (2014) define a Primitive as the pair of input/output parameter sets, a Skill as a directed graph over actions and connections augmented with such parameter sets, and a Parameter as a type-name-value triplet.

$$Primitive ::= \langle par_{\text{input}},\ par_{\text{output}} \rangle \tag{1}$$

$$Skill ::= \langle A,\ C,\ par_{\text{input}},\ par_{\text{output}} \rangle \tag{2}$$

$$Parameter ::= \langle type,\ name,\ value \rangle \tag{3}$$

where A is the set of action nodes and C is the set of connections (including data-flow and conditional flow edges).

3.1.2 Temporal taxonomy: configuration parameters vs. runtime state variables

Several approaches distinguish between skill configuration and the evolving runtime world state. In the manufacturing-oriented skill concept of Pedersen et al. (2016), tasks can be executed with predefined parameters, or planned based on the current world model and desired goal settings of state variables. This gives a conceptual boundary:

- **Skill parameters** configure a specific skill invocation (e.g., target settings, tolerances, tool settings).
- **State variables** represent the current modeled world state and change during runtime (e.g., updated through sensing/estimation and through skill effects).

This separation supports automation, as many methods infer configuration parameters from engineering data, whereas state variables are updated online during execution.

3.1.3 Responsibility and visibility taxonomy: primary, secondary, internal

Steinmetz and Weitschat (2016) classify parameters by visibility and who is responsible for providing them. They distinguish three parameter sets:

Primary parameter set: Mandatory parameters that must be specified for execution.

Secondary parameter set: "Safe" parameters with defaults; advanced users may tune them, and they may be derived from other parameters.

Internal parameters: Fixed by the skill developer and hidden from end users.

This taxonomy is tightly connected to teach-in: if a required parameter is missing, execution can be paused until the user provides it, then resumed (Steinmetz & Weitschat, 2016). It also supports parameter reduction via internal knowledge: for example, multiple screwing-related parameters can be reduced to a single higher-level input (such as screw type) if the skill contains lookup knowledge to derive the remaining values (Steinmetz & Weitschat, 2016). Figure 3.1 illustrates how parameters become entangled within a skill implementation. For example, the user may only need to specify the desired screw pose (a primary parameter), while additional execution settings such as robot positioning or guidance modes can be treated as secondary parameters. The remaining checks, state transitions, and controller variables are internal parameters hidden from the user.

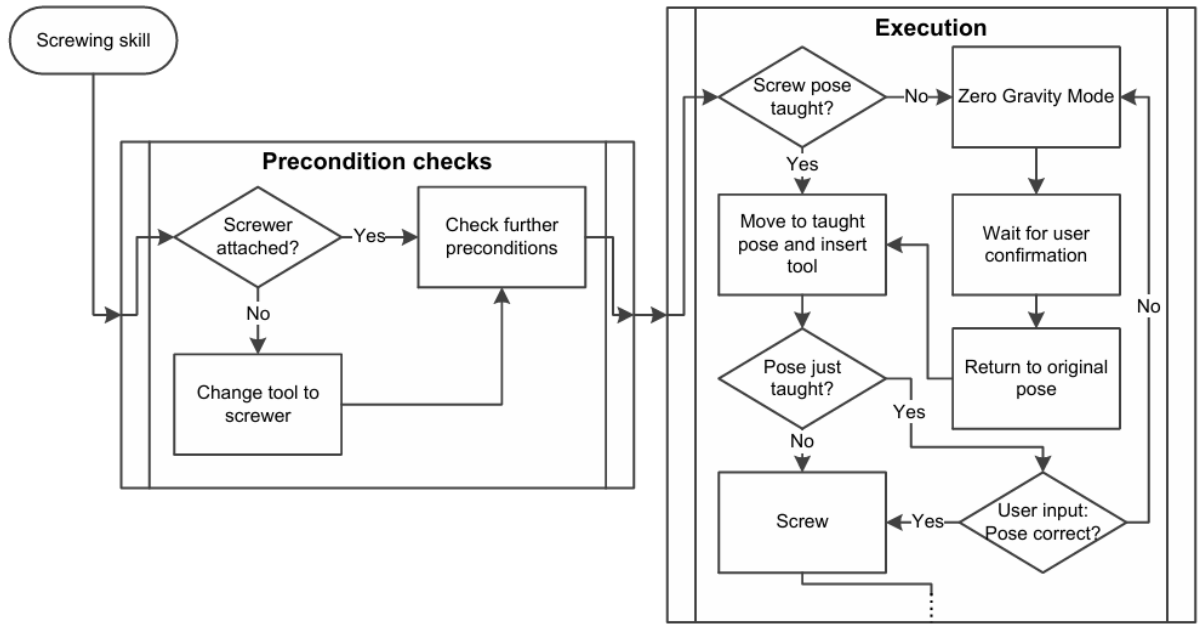


Figure 3.1: The parametrization of the screw pose is entangled in the skill (Steinmetz & Weitschat, 2016).

3.1.4 Origin and acquisition taxonomy: manual parameters vs. derived parameters

Another axis classifies parameters by how they are obtained. In the capability-based survey of Pantano et al. (2022), a parameter is described as manually defined and interpretable, whereas a derived parameter is extracted from sensor/system data and may not be interpretable for the user. This is useful for deciding which parameters can be exposed to end users, which should be inferred automatically, and which should remain internal to the skill implementation.

3.1.5 Automation-oriented taxonomy: inference source, completion strategy, verification

Automated robot skill parameterization based on task specifications makes the question "how do we obtain the parameter?" central (Kurscheid & Daub, 2024). The approach emphasizes that parameters may (i) already be available from engineering/planning systems (e.g., MES/PLM data, CAD-derived information), (ii) be computed via reasoning within a knowledge model, or (iii) require simulation-based computation. When information is incomplete, the system can identify information gaps and request minimal missing inputs from a user (Kurscheid & Daub, 2024). Figure 3.2 shows the entire workflow of this parametrization.

In addition, verification introduces another taxonomy dimension: some parameter sets can be validated automatically (e.g., via simulation checks), while others may require manual confirmation (e.g., constraints that are difficult to verify automatically) (Kurscheid & Daub, 2024).

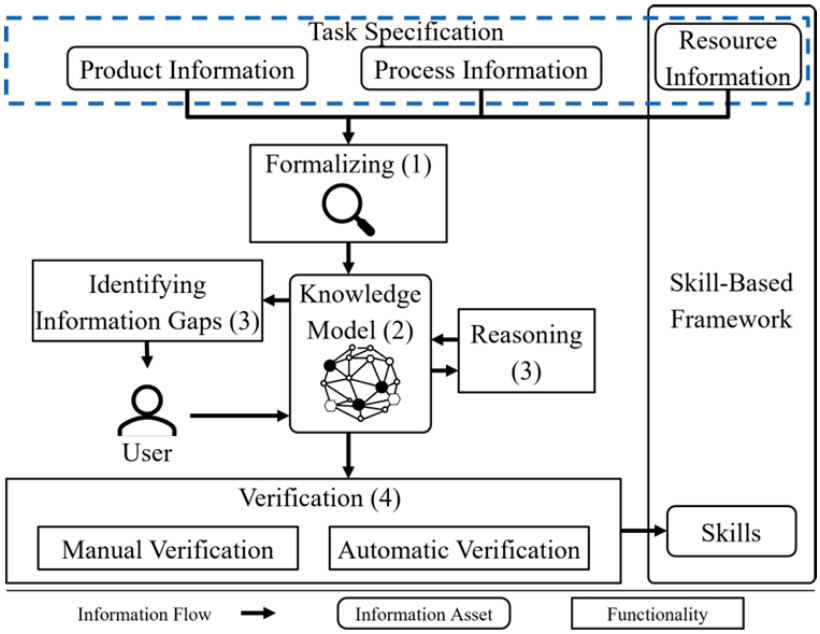


Figure 3.2: Overview of automated robot-skill parameterization. (Kurscheid & Daub, 2024).

3.1.6 Policy-structure taxonomy: intrinsic vs. extrinsic parameters

For skill representations combining behavior trees and motion generators, the policy parameters θ are subdivided into intrinsic and extrinsic parameters (Ahmad et al., 2023):

$$\theta = (\theta_i, \theta_e), \tag{3.1}$$

where intrinsic parameters θ_i describe the policy structure (e.g., BT structure, number of control nodes, motion generator type) and are treated as situation-invariant (Ahmad et al., 2023). Extrinsic parameters θ_e, by contrast, are situation-dependent execution parameters such as applied force, offsets, and end-effector velocity (Ahmad et al., 2024). These extrinsic parameters may be specified manually, inferred through reasoning, or learned via reinforcement learning, providing the flexibility to adapt the representation across task variations (Ahmad et al., 2023) and failures (Ahmad et al., 2024).

3.2 Monitoring

This section describes how the thesis's software stack (ROS 2, MoveIt 2, and Isaac Sim) can be instrumented to produce reliable, episode-level KPIs for closed-loop skill evaluation. We separate monitoring of planning and execution semantics on the MoveIt/MoveIt Task Constructor (MTC) side (cf. Section 3.2.1) from physics grounded interaction outcomes in Isaac Sim (cf. Section 3.2.2), and then outline a lightweight integration strategy (cf. Section 3.2.3). The outcome is a compact episode summary suitable both for optimization and for debugging failure modes.

Monitoring for closed-loop skill evaluation in ROS2, MoveIt2, and Isaac Sim

Industrial manipulation skills such as pick-and-place are commonly executed through a layered software stack, in which high-level task logic triggers motion planning and trajectory execution, and a simulator or real robot produces physical interaction outcomes. Because pick-and-place operations account for a large share of industrial robotic applications, their evaluation is most meaningful at the level of overall system performance rather than at the level of any single component, which in turn motivates reproducible, scenario-based benchmarking with well-defined performance indicators (Mnyusiwalla et al., 2020). In the context of automated closed-loop evaluation, this structure naturally leads to a black-box episode function in which a parameter vector $\boldsymbol{\theta}$ is applied, one trial is executed, and an episode summary of key performance indicators (KPIs) is returned.

A practical monitoring design must therefore answer two questions: (i) where in the stack a KPI can be measured reliably, and (ii) how to produce an integrated episode summary that is simple enough to be robust, yet informative enough to support optimization and debugging.

A central distinction is between execution and physical interaction outcomes. ROS2 execution semantics include whether planning succeeded, whether the controller reported successful execution, and whether timing constraints were respected. Physical interaction outcomes include contact forces, collisions, and whether an object remained stably grasped during transport. A third, complementary family of KPIs captures actuation effort and motion aggressiveness, namely the energy expended and the peak acceleration encountered over an episode, both of which can be derived from proprioceptive signals (joint states and inertial measurements). This split is especially relevant for workflows that combine MoveIt2 with Isaac Sim: MoveIt provides task- and motion-level reasoning, while Isaac Sim provides a PhysX-based world model capable of reporting contacts, forces, and joint-level dynamics. The relevance of monitoring the object–end-effector interaction beyond mere task completion is well established, since the manner in which an object is grasped and held directly affects whether it survives the subsequent transport motion (Mnyusiwalla et al., 2020).

3.2.1 Monitoring execution semantics in MoveIt2

MoveIt 2 provides explicit mechanisms for managing and monitoring trajectory execution at the motion-execution layer. In MoveIt2-based systems, execution is mediated by an execution component, often referred to as the trajectory execution manager. It dispatches planned trajectories to controllers and can incorporate online validity checks during motion. This reflects a notion of execution correctness that is evaluated during execution rather than solely at planning time (Stoop et al., 2024).

For pick-and-place specifically, MTC provides a task-level structure that decomposes manipulation into multiple interdependent sub-tasks. MTC explicitly solves subproblems in modular "black-box"

stages and passes solution hypotheses and world-state information between stages via a common interface based on the planning scene, making failures attributable to specific stages rather than collapsing them into a single binary outcome (Görner et al., 2019). This stage-wise decomposition is advantageous for monitoring because it yields interpretable failure modes, which improve debugging and support closed-loop parameter evaluation.

Additionally, this style of reporting is consistent with emerging whole-stack evaluation practices in the MoveIt ecosystem: the MoveIt Benchmark Suite explicitly treats MTC tasks as a benchmark target and reports statistics measured per stage, illustrating that per-stage monitoring outputs are a recognized and reusable evaluation pattern for integrated pick-and-place pipelines (Görner et al., 2019).

3.2.2 Monitoring physical interaction outcomes in Isaac Sim

While MoveIt provides motion-level status, many safety- and quality-relevant KPIs in manipulation depend on physical interaction: collisions, contact forces, and stable grasp behavior. Recent Omniverse-based simulation frameworks explicitly support physics-grounded contact sensing. In Isaac Lab, the contact sensor reports net contact forces on assigned rigid bodies and can be configured to focus on selected contact interactions, making it suitable for measuring quantities such as peak grasp contact force and for detecting undesired contact relationships (e.g., robot–table or object–environment contacts) during transport and placement (Mittal et al., 2025).

Moreover, TacEx integrates tactile simulation components into Isaac Sim and emphasizes that PhysX is the simulator's underlying physics engine, reinforcing Isaac Sim's role as an authoritative source for physics-based contact events and interaction signals (Nguyen et al., 2024). Together, these works motivate computing episode-level interaction KPIs from Isaac Sim, including maximum contact force during grasp closure or incidental environment interaction, as well as collision flags defined over forbidden contact pairs.

Beyond raw contact signals, grasp stability can be assessed by combining contact persistence with object motion constraints. A simple and robust criterion is to mark a grasp as stable if the object remains in contact with the gripper over a specified time window and if its pose relative to the gripper frame remains bounded while the robot executes the transport and place motions. Such definitions align with physics-based grasp evaluation research that frames grasp success in terms of whether an object can be lifted securely or is dropped during the grasping process (Kim et al., 2022). In this way, Isaac Sim becomes the authoritative source for interaction KPIs, while MoveIt remains authoritative for planning and execution semantics.

A further class of physics-grounded KPIs derives not from contact sensing but from the robot's own joint-level dynamics, which Isaac Sim exposes through joint states and inertial readings. The first such KPI is the peak acceleration encountered during a trial. The dynamic demand placed on a grasp during transport is governed by end-effector acceleration: aggressive accelerations increase the inertial load the grasp must resist and can cause slips or spillage. Ichnowski et al. (2022) make this dependency explicit by constraining end-effector accelerations within a time-optimal transport planner and, for validation, recording the end-effector acceleration norm from an inertial measurement unit (IMU) mounted at the wrist to confirm that a prescribed acceleration limit is respected. The peak acceleration observed over an episode is therefore a physically meaningful KPI: it bounds grasp retention and, through a Coulomb friction argument, determines the minimum grip force required to hold the object securely. See Section 2.4.

The second joint-level KPI is the mechanical energy expended over an episode. Because joint-level torque or motor power is the practical observable on most platforms. It is often more readily

obtained than a dedicated, calibrated energy measurement. An episode energy KPI can be formed by integrating the instantaneous mechanical joint power over the trial (Liu et al., 2018). The equation is presented in Section 4.9. This collapses an entire trajectory into a single scalar suitable as an optimization objective. Computing E from joint states rather than from an idealized analytical model keeps the metric grounded in the executed motion and in the simulator's dynamics.

3.2.3 Practical integration and complexity management

A key implementation concern is avoiding overcomplication while still producing defensible measurements. Full-stack tracing approaches exist for ROS 2 and can provide detailed timing and latency breakdowns with low overhead, such as Ros2-tracing (Bédard et al., 2022), which instruments core ROS 2 packages and supports end-to-end message latency analysis. However, for simulation-based parameter optimization focused on manipulation KPIs, such low-level tracing is often unnecessary.

A lighter-weight alternative is to measure episode timing at the episode level rather than through fine-grained instrumentation. Cycle time is a standardized robot performance criterion (International Organization for Standardization, 1998), and because it directly governs workcell throughput, the number of operations completed per unit time, it is a natural deployment-oriented timing objective (Bottin et al., 2022). In a simulation-based evaluation, recording the realized duration of an episode, rather than the nominal trajectory duration reported by the planner on the ROS/simulation clock, it keeps the KPI aligned with the overheads a deployed system actually incurs, planning, communication, and actuation, while avoiding any dependence on detailed internal tracing.

In summary, the literature and tooling around ROS 2, MoveIt2, and Isaac Sim support a clean split for monitoring: MoveIt-side monitoring provides success semantics and timing around planning and execution, while Isaac-side monitoring provides physically grounded metrics, namely contact forces, collisions, and grasp stability, together with joint-level effort and motion KPIs such as expended energy and peak acceleration. Augmented by the wall-clock cycle time recorded at the Skill Trial Runner, these signals merge into a compact episode summary, turning each simulation episode into a single objective-function evaluation for black-box optimization of skill parameters $\boldsymbol{\theta}$ in pick-and-place tasks.

3.3 Optimization

This section reviews optimization algorithms that match the problem setting: expensive black-box episode evaluations whose outcomes include both performance KPIs and explicit safety/feasibility constraints. We first discuss constrained and safe Bayesian optimization methods that model feasibility and enable sample-efficient exploration under constraints (cf. Section 3.3.1), including Safe Bayesian Optimization (SafeOpt)-style approaches, success-as-constraint formulations, and scalable variants such as Line Bayesian Optimization (LineBO). We then cover evolutionary and population-based methods that naturally handle multi-objective trade-offs and return Pareto-optimal parameter sets (cf. Section 3.3.3), with Non-dominated Sorting Genetic Algorithm II (NSGA-II) as a representative baseline.

3.3.1 Constrained / safe Bayesian optimization

SafeOpt-MC

Constrained Bayesian Optimization (cBO) extends Bayesian optimization and requires the tuning parameters to satisfy explicit safety requirements. This is particularly relevant in robotics, where each evaluation corresponds to a finite-time trial and may lead to undesirable outcomes (e.g., collision, loss of contact, excessive forces). Bayesian optimization is attractive because it enables data-efficient optimization with few evaluations. However, safety cannot generally be treated as a by-product of "good performance": defining safety solely through a minimum performance requirement is restrictive because safety and performance can conflict (Berkenkamp et al., 2023). To address this, Berkenkamp et al. (2023) propose an extension of SafeOpt that supports multiple, arbitrary safety constraints decoupled from the performance objective, while retaining BO's sample efficiency and offering safety guarantees with high probability. Figure 3.3 shows at iteration n, the algorithm selects safe and informative parameters a_n; performance $f(a_n)$ and safety constraints $g_i(a_n)$ are evaluated, and the belief over the functions is updated. The process is repeated until the safely reachable optimum is found.

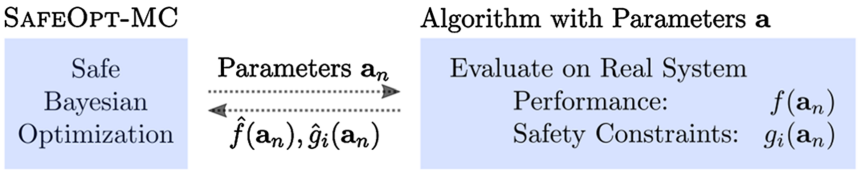


Figure 3.3: Overview of SafeOpt-MC (Berkenkamp et al., 2023).

At a high level, the resulting optimization problem can be written as

$$\max_{\theta \in \mathcal{A}} f(\theta) \quad \text{s.t.} \quad g_i(\theta) \geq 0,\ i = 1, \ldots, q \tag{3.2}$$

where $f(\theta)$ is a scalar performance measure and $g_i(\theta)$ are (unknown) safety constraints over a parameter domain $\mathcal{A}$ (Berkenkamp et al., 2023). A key idea in cBO is to start from an initial set of parameters that is known to be safe and then iteratively balance (i) improving performance within the currently known safe region and (ii) cautiously expanding this safe region based on model uncertainty and smoothness assumptions (Berkenkamp et al., 2023).

This framing aligns well with simulation-based robot skill tuning: constraints such as collision avoidance or force limits are first-class requirements, and the optimizer should avoid repeatedly sampling unsafe regions even when the objective landscape is noisy.

Success as a constraint

Another perspective on cBO is presented by Dries et al. (2017), who emphasize that many manipulation tasks naturally involve discrete success signals (success/failure) alongside continuous performance metrics. Collapsing these into a single scalar objective by assigning large penalties to failures is undesirable because it discards useful structure; instead, success should be treated explicitly as a constraint (Dries et al., 2017). This leads to a constrained formulation of the form

$$\min_{\xi \in \mathcal{P}} J(\xi) \quad \text{s.t.} \quad S(\xi) = 1 \tag{3.3}$$

where $J(\xi)$ is a real-valued cost (e.g., tracking or force error) and $S(\xi) \in \{0, 1\}$ indicates whether the trial was successful (Dries et al., 2017)(Dries et al., 2017). Methodologically, each rollout yields tuples (ξ_l, J_l, S_l), and two probabilistic models are learned: a surrogate model for the cost surface and a probabilistic model for the success region. The next parameters are then selected

by an acquisition function that trades off exploitation (improving J where success is likely) and exploration (reducing uncertainty near the boundary between success and failure) (Dries et al., 2017).

LineBO

A complementary and particularly relevant contribution for scalability is provided by Kirschner et al. (2019). They explicitly note that Bayesian optimization is hard to scale because the acquisition step requires solving a non-convex optimization problem in the same search space (Kirschner et al., 2019). To address this, they propose LineBO, which restricts the problem to a sequence of iteratively chosen one-dimensional sub-problems that can be solved efficiently (Kirschner et al., 2019). The key decomposition idea is to replace one difficult d-dimensional acquisition optimization with repeated one-dimensional BO subproblems along carefully chosen affine subspaces that pass through the current best solution. This strategy is attractive in practice because 1D BO is computationally cheap and can be monitored via slice plots, while a global GP model can still be used to share information between the sub-problems to increase data-efficiency (Kirschner et al., 2019). The constraint function can be vector-valued and have multiple constraints, which matches robotics settings where feasibility depends on several conditions (Kirschner et al., 2019).

Finally, Kirschner et al. (2019) connect their decomposition strategy directly to safe BO by combining LineBO with SafeOpt, yielding a safe Bayesian optimization method with theoretical guarantees that remains applicable in higher-dimensional search spaces (Kirschner et al., 2019). The key practical motivation is that standard SafeOpt typically operates on a discretized candidate set, which becomes difficult to handle as dimensionality increases (often already beyond $d > 3$), whereas applying SafeOpt along a one-dimensional subspace is straightforward to discretize and optimize (Kirschner et al., 2019). For simulation-based robot skill tuning, this offers a pragmatic route to safe exploration when the number of tuned parameters grows, without having to solve the full acquisition problem in the original d-dimensional space.

3.3.2 Contextual Bayesian optimization in robotics

Contextual BO extends standard BO to objective functions that additionally depend on external, non-controllable variables termed *contexts*. Krause and Ong (2011) introduced the paradigm as contextual Gaussian process bandit optimization: a single GP is placed over the joint context–action space, and at each round the learner observes a context, selects an action, and receives a noisy payoff, with similarity in *both* context and action governing the correlation of outcomes. This is the formulation adopted in this work, in which the task context is given and conditioned upon rather than searched (see Section 4.2.1).

Within robotics, the paradigm was first applied to contextual policy search, where a low-dimensional context parameterises a movement primitive. Metzen et al. (2015) model the expected return of a parameterised skill as a GP over the joint context–parameter space and report that BO reaches competitive policies in markedly fewer trials than local search on low-dimensional locomotion and reaching tasks. This is the sample-efficiency argument that motivates BO under expensive real-robot evaluations. A closely related line generalises robot skills across contexts using model-based contextual policy search (Kupcsik et al., 2017), sharing the contextual-generalisation objective while relying on relative-entropy policy updates rather than acquisition-driven search.

Because online robot trials can evaluate parameters that damage the platform, several works couple the contextual formulation with explicit safety or feasibility constraints. Berkenkamp et al. (2021)

extend safe BO to multiple constraints decoupled from the objective and demonstrate automatic, constraint-respecting controller tuning on a quadrotor, with the operating condition treated as a fixed context. The same combination has been carried into industrial deployment: De Blasi et al. (2024) embed safe contextual BO in an edge controller for a self-learning steel-straightening machine, using a data-discard strategy and local GP approximations to sustain continual, in-situ optimisation under drifting process conditions.

Across these works the recurring structure is a contextual GP surrogate over a joint context–parameter space, queried online at a fixed observed context and frequently augmented with safety or feasibility constraints. What remains comparatively unaddressed is the HMLV pick-and-place regime targeted here: minimising energy subject to *hard*, operator-specified constraints on cycle time, peak acceleration, and grip force, while warm-starting the per-context search from an offline, context-indexed memory rather than exploring each context from scratch. The present work occupies that gap by combining the contextual formulation of Krause and Ong (2011) with a constrained single-objective online loop and a memory-based warm-start.

3.3.3 Evolutionary and population-based optimization

Evolutionary and population-based optimization methods are potentially a good fit for simulation-driven tuning of robot skill parameters, where each candidate parameter vector θ must be evaluated by executing a full episode and extracting metrics from a monitoring layer (e.g., cycle time, peak contact force, collision flags, grasp stability). This setting is typically a black box and non-smooth. Also, unlike gradient-based methods, population-based optimizers operate directly on episodic measurements and therefore remain applicable when derivatives are unavailable or uninformative.

A further motivation is that robot skill tuning is often inherently multi-criteria. Faster trajectories can increase acceleration and contact forces but raise the risk of slip or collision, while more conservative parameters reduce risk but increase cycle time. This motivates Pareto-based formulations in which the optimization procedure returns a set of trade-off solutions rather than a single optimum. In addition, population-based methods can preserve diversity (or even explicitly seek it), which is useful when downstream decision layers require multiple alternatives (e.g., “fast” vs. “safe” parameterizations) for different contexts.

NSGA-II for pareto-based multi-objective optimization

A widely used baseline for multi-objective evolutionary optimization is NSGA-II, proposed by Deb et al. (2002). NSGA-II is attractive in robotics because it supports black-box evaluation, scales to multiple objectives, and provides a standard mechanism to approximate a Pareto front while maintaining diversity among solutions. Importantly, NSGA-II also explicitly discusses constrained multi-objective optimization, noting that constrained formulations are central to practical problem solving (Deb et al., 2002). This aligns with monitoring-driven skill tuning, where feasibility conditions such as collision-free execution or bounded contact forces should be treated as first-class requirements.

Operationally, NSGA-II ranks individuals by non-domination level and resolves ties using a crowding-distance measure to preserve spread along the Pareto front. This is captured by the *crowded-comparison operator* (often written as “$\prec_n$”) (Deb et al., 2002):

$$i \prec_n j \iff \big(r_i < r_j\big) \vee \big(r_i = r_j \wedge d_i > d_j\big), \tag{3.4}$$

where r_i denotes the non-domination rank (lower is better) and d_i denotes the crowding distance (larger is preferred, indicating a less crowded region in objective space). In robotic applications, this operator provides a concrete selection principle: candidate solutions are prioritized by Pareto dominance, while diversity is preserved to avoid collapsing the population onto a narrow region of the trade-off curve (Deb et al., 2002).

A typical instantiation in robotic skill or controller tuning is to define an objective vector from monitoring-derived KPIs, for example

$$\mathbf{f}(\theta) = \big[f_1(\theta), f_2(\theta), f_3(\theta)\big] = \big[\text{execution_time}(\theta), \text{max_interaction_force}(\theta), \text{max_joint_acc}(\theta)\big] \tag{3.5}$$

where the components represent common performance and safety-related criteria observed during episodic executions. Feasibility conditions such as collision-free execution, bounded forces, or task completion can be incorporated as constraints, either via constrained-dominance rules or via feasibility-first selection strategies (Deb et al., 2002). The result is a Pareto set of candidate parameterizations representing qualitatively different operating points (e.g., faster but more aggressive behaviors versus slower but more conservative behaviors). This is particularly valuable when downstream decision-making requires flexibility: rather than committing to fixed weights a priori, one can select an operating point from the Pareto set based on externally specified preferences or context.

The main limitation of NSGA-II in simulation is evaluation cost: multi-objective evolutionary algorithms often require many episodes to approximate a stable Pareto front, which can become computationally demanding when rollouts are expensive or when extensive randomization is required. Consequently, NSGA-II is frequently used as a strong and interpretable baseline for moderate-dimensional parameter tuning, while motivating more sample-efficient approaches when rollout budgets are tight.

A central limitation of NSGA-II for simulation-based robotics is its sampling cost. The algorithm queries the objective on a population of size N at initialization and on N further offspring in each generation, so that over G generations the number of objective queries grows on the order of

$$N_{\text{eval}} \approx N\,(G+1) \tag{3.6}$$

(Doerr et al., 2025). This scaling becomes prohibitive when a single query corresponds to an expensive rollout, such as a full physics-simulation episode, a regime typical of robotic parameter tuning (Tesch et al., 2013). NSGA-II therefore serves as a strong and interpretable baseline for moderate-dimensional problems, whereas costly evaluations motivate more sample-efficient methods.

3.3.4 Case-based reasoning

Case Based Reasoning (CBR) is an experience-driven problem-solving paradigm that resolves a new problem by recalling and adapting the solutions of similar problems encountered in the past (Aamodt & Plaza, 1994). Each stored *case* pairs a problem description with its associated solution and, optionally, the outcome that solution produced. The paradigm rests on the assumption that

similar problems admit similar solutions, so that past experience can be reused to avoid solving every problem from scratch. In this sense CBR is a form of relaxed, instance-based learning: it defers the bulk of its reasoning to query time and is comparatively knowledge-light, since it requires a case base and a similarity measure rather than a complete set of domain rules or a closed-form model (Aamodt & Plaza, 1994). This property makes CBR attractive in domains where expert knowledge is abundant in the form of concrete examples but difficult to formalise as explicit rules.

Aamodt and Plaza (1994) formalise the approach as a cyclic process, later popularised as the *four Rs* (López de Mántaras et al., 2005), illustrated in Figure 3.4 and summarised step by step in Table 3.1. The cycle separates a *problem space*, over which similarity is assessed, from a *solution space*, in which retrieved solutions are reused and adapted. The *retrieve* and *retain* steps make CBR an inherently incremental, memory-based learner: every successfully solved problem can be added back to the case base, so the system's competence grows with experience rather than being fixed at design time.

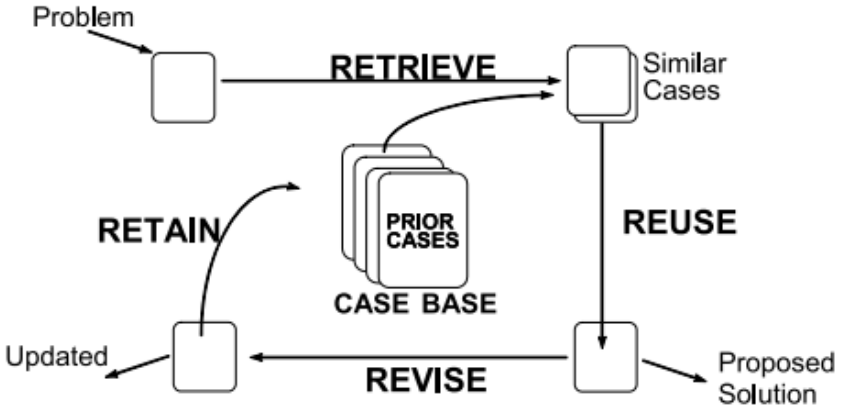


Figure 3.4: The four-step CBR cycle (Aamodt & Plaza, 1994).

Table 3.1: The four-step CBR cycle after Aamodt and Plaza (1994) and López de Mántaras et al. (2005).

Step	Operation
Retrieve	Given a new problem, identify the most similar case(s) in the case base using a similarity measure over the problem features. Nearest-neighbour search over a feature space is the most common retrieval mechanism.
Reuse	Transfer the retrieved solution to the new problem, either directly (copying the solution) or as an initial proposal that serves as a starting point for the new context.
Revise	Evaluate the proposed solution, for example by simulation or execution, and adapt it to account for the differences between the retrieved and the current problem until it is satisfactory.
Retain	Store the validated problem–solution pair as a new case, so that the accumulated experience becomes available for future retrieval.

The retrieval step deserves particular attention, since it determines which prior experience is brought to bear and is the component most directly shared with other memory-based methods. Retrieval reduces to defining a similarity (or distance) measure over the problem features and selecting the closest case or the k closest cases (Aamodt & Plaza, 1994). The quality of a CBR system therefore hinges as much on the design of the feature representation and the similarity metric as on the contents of the case base itself.

The four-step structure maps naturally onto robotic problem solving. A representative example

of the *retrieve* step is given by Fitzgerald et al. (2015), who treat each previously observed skill demonstration as a case and compare several visual similarity measures for retrieving the most relevant source demonstration for a new target situation. Reliable retrieval over the case memory is the prerequisite for reusing a known skill. An early and influential example covering the *retrieve* and *revise* steps in a control setting is the ACBARR system of Ram et al. (1997), in which a CBR module is layered on top of a reactive controller to select behavioural control parameters for the current environment and then continuously adapt them online.

A recurring use of CBR in robotics is to exploit the case base as a *memory* that warm-starts or accelerates an otherwise expensive learning or search process. Bianchi et al. (2018) use a case base as a source of heuristics within a transfer-learning setting to bias and speed up reinforcement learning on two real-robot domains, reporting substantially faster convergence than learning from an uninformed initialisation. Homem et al. (2020) combine CBR with qualitative spatial representations and reinforcement learning so that retrieved cases guide action selection while new cases are acquired at runtime, tightening the retrieve–retain loop during operation. In a manipulation context, Mitrevski et al. (2020) train experience-based, explainable models of robot action execution from past executions, illustrating how a memory of prior outcomes can inform the parameterisation of future action attempts. In each of these works the stored experience does not replace the learner but supplies an informed prior that narrows the search the learner must perform, which is the same role a memory layer plays when seeding an optimization loop.

Beyond control and learning, CBR has a long-standing footprint in industrial robotics. Sun et al. (2014) integrate CBR with explanation-based learning in a human–system collaborative programming platform for industrial robots, reusing the encoded experience of skilled engineers in the form of cases to generate programs for novel tasks and thereby reduce programming effort; the authors explicitly motivate CBR by noting that practical engineering knowledge is often easier to capture as concrete cases than as a complete rule base. CBR has likewise been applied to condition monitoring and fault diagnosis of industrial manipulators from sensor and acoustic signals (Ahmed et al., 2010), again exploiting the availability of past cases in a domain that resists explicit rule-based modelling. Taken together, these applications motivate the use of a case- or memory-based component as a warm-start mechanism within the present work, where prior trials are stored and retrieved by nearest-neighbour similarity to provide informed initial estimates rather than starting each optimisation from an uninformed prior.

Chapter 4 Design

This chapter develops the design of the optimizer agent, proceeding from the behavior the agent must exhibit to the architecture that realizes it, and remaining at the level of design decisions rather than their software realization. The overall goal is to define how the agent arrives at a safe, simulation-verified, energy-minimal parameterization for a queried task.

Section 4.1 first sets out the design requirements, the behavior the agent must satisfy, grouped into five categories and traced to the decisions they induce. Section 4.2 then partitions every parameter that varies across a trial into an observed context, a controlled decision, and the measured outcomes that form the objective and the constraints. On that partition, Section 4.3 casts the task as a constrained, contextual, single-objective Bayesian optimization problem and describes how it is solved under surrogate uncertainty. Section 4.4 specifies how each objective and constraint quantity is measured from a completed execution, and Section 4.5 closes the chapter by composing these elements into an offline characterization phase and an online four-R decision cycle that returns a policy for each queried context.

4.1 Design requirements

This section traces the requirements on the optimizer agent. The requirements are directed at the agent: they specify the behavior the system must exhibit, without committing to an implementation.

The requirements are grouped into five categories. *Functionality* (A) specifies what the agent must do across the trial lifecycle. *Verification* (B) states the conditions under which the system's measurements can be trusted. *Safety and feasibility* (C) fix the non-negotiable boundary conditions the solution must respect and take precedence over all other concerns. *Optimization* (D) names the objective and how it is pursued strictly subject to those conditions. *Adaptivity and generalization* (E) capture how the agent reuses past experience to re-tune cheaply when the context changes, and are organized by the four-R cycle of CBR. Table 4.1 consolidates all requirements and makes the backward and forward traceability explicit.

4.1.1 Functionality

These requirements specify the expected behavior of the agent across a trial.

- **A1 Predictive parameterization.** Given an input context vector (pick and place pose, object mass, friction coefficient), the agent shall output a decision policy $(v_{\text{scale}}, a_{\text{scale}})$ to the user. This policy is the externally visible solution of the agent for a queried context.

- **A2 Parameter handoff.** The agent shall test and validate a candidate policy in simulation before handing it to the user, so that the policy is simulation-verified an not merely predicted.
- **A3 Autonomous trial lifecycle.** The agent shall autonomously execute a complete reset → execute → read cycle in simulation, without intervention.
- **A4 KPI acquisition.** The monitoring node shall be responsible for deriving energy, cycle time, required grip force, and trial success from the joint states and the Inertial Measurement Unit (IMU) during execution.

4.1.2 Verification

These requirements state the conditions under which the acquired measurements faithfully represent the process, and are therefore the basis on which all downstream feasibility and Optimization claims rest.

- **B1 Observational separation.** Monitoring shall be strictly observational: the monitoring node shall read process signals without altering the commanded trajectory or the control loop, so that the measurement reflects the process and not the act of measuring it.
- **B2 Measurement reproducibility.** For a fixed context and policy, repeated trials shall yield metrics with bounded variance.

4.1.3 Safety and feasibility

These are the non-negotiable boundary conditions on the solution space. They are stated before the objective because feasibility is necessary and Optimization only desirable: the agent must first guarantee a safe, executable grasp, and only then optimize it.

- **C1 Static-grasp check.** The agent shall ensure that the maximum grip force constraint $F_{\max}$, defined by the user, is enough to sustain a static hold. This is a hard safety constraint, applied before optimization.
- **C2 Execution feasibility.** The agent shall ensure that the measured cycle time and grip force stay within the user-defined limit $T_{\max}$ and $F_{\max}$. This is a hard feasibility constraint: a policy that exceeds $T_{\max}$ or $F_{\max}$ is rejected regardless of its energy cost.
- **C3 Honest infeasibility.** The agent shall responsibly admit when no feasible policy was found for the queried context, rather than returning an infeasible recommendation, and shall report which constraints are binding so that the user can relax them to realistic values.
- **C4 Input rejection.** The agent shall reject invalid or out-of-scope inputs before any trial budget is spent.

The required grip force F_{req} that C-1 and C-2 bound is obtained from the observed peak acceleration through the Coulomb friction model. Its explicit form, the role of the safety factor S, and the reason it enters the optimization as a modeled outcome constraint are developed where the quantity is measured (section 4.4) and where the constrained problem is posed (section 4.3).

4.1.4 Optimization

These requirements name the objective and the way it is pursued. They are subordinate to the safety and feasibility requirements of the previous subsection: the objective is meaningful only over the feasible region that those constraints define.

- **D1 Constraint-aware acquisition.** Cycle time, grip force, and trial success shall enter the optimization as constraints, and the acquisition shall minimize energy only over the region they admit.
- **D2 Single objective.** Energy consumed per task shall be the sole objective. The constraints take priority over the objective: a policy must first be feasible, and only among feasible policies is energy minimized.
- **D3 Sample efficiency.** Since online trials are expensive, the agent must be sample efficient and converge within an acceptable number of trials.

4.1.5 Adaptivity and generalization

These requirements describe how the agent reuses past experience so that it can re-tune cheaply when the task context changes. They are organized by the four-R cycle of CBR, *Retrieve*, *Reuse*, *Revise*, *Retain* (Aamodt & Plaza, 1994; López de Mántaras et al., 2005) (section 3.3.4), which structures the optimization architecture of section 4.5. Contextual conditioning (E1) is the substrate that makes a context-indexed memory meaningful, and a feasibility screen (E4) precedes the cycle. Within the cycle, a recurring context is answered by *retrieving* and *reusing* a stored solution directly (E5); a novel context instead *retrieves* and *reuses* its nearest offline anchors as a warm-start (E3) and is then *revised* by the online loop to generalize to that context (E2); and every solved context is *retained* for future recall (E6).

- **E1 Contextual conditioning.** The agent shall condition on the eight-dimensional context rather than search it, keeping the online problem two-dimensional over $(v_{\text{scale}}, a_{\text{scale}})$. The context is a per-task input held fixed during the online run.
- **E2 Generalization to unseen contexts.** The agent shall produce feasible and optimized policies for context combinations not present in the offline dataset, by interpolating from neighboring offline observations rather than re-optimizing from scratch.
- **E3 Memory-based warm-start.** Offline anchors retrieved by a K-Nearest Neighbors (KNN) retriever shall seed the online run, providing the first anchor point for the optimization at the queried context.
- **E4 Feasibility gating.** The offline characterization shall screen infeasible contexts before any online trial is spent, so that the limited online budget is committed only to contexts that can plausibly be served.
- **E5 Case recall.** The agent shall recall a previously solved policy for a recurring context and reuse it directly whenever it remains compatible with the current constraints, returning the recommendation without spending any trial budget.
- **E6 Case retention.** The agent shall persist each solved context–policy pair so that recurring contexts are served by recall rather than re-optimization, allowing the system to improve with use.

4.1.6 Requirements traceability

Table 4.1 summarizes the requirements and links each one to the design decision it induces, making the forward traceability into the remainder of the chapter explicit.

Table 4.1: Design requirements and the design decisions they trace to.

ID	Requirement	Traces to
Functionality		
A1	Output policy for a context	Agent output, decision space ($v_{\text{scale}}, a_{\text{scale}}$)
A2	Validate before handoff	In-simulation validation step
A3	Autonomous trial lifecycle	Skill trial runner, `run_cycle()` loop
A4	Derive KPIs from signals	Monitoring node (joint states, IMU)
Verification		
B1	Observational separation	Passive monitoring, decoupled from control
B2	Reproducible measurement	Deterministic planning
Safety and feasibility		
C1	Static-grasp check	Coulomb static-hold constraint
C2	Execution feasibility	Cycle-time $T_{\max}$ and Force $F_{\max}$ limit, hard constraint
C3	Honest infeasibility	Feasibility report, constraint relaxation
C4	Input rejection	Context and policy bounds, pre-execution gate
Optimization		
D1	Constraint-aware acquisition	BO loop with constrained acquisition
D2	Energy as sole objective	Minimize energy within feasible region
D3	Sample efficiency	BO over reinforcement learning, grid, or random search
Adaptivity and generalization (CBR cycle)		
E1	Contextual conditioning	Contextual formulation (8 D context, 2 D decision)
E2	Generalize to unseen contexts	*Revise*: warm-started BO loop
E3	Memory-based warm-start	*Retrieve/Reuse*: k-KNN offline anchors seed BO
E4	Feasibility gating	Offline characterization gate
E5	Case recall	*Retrieve/Reuse*: case-base lookup, instant policy
E6	Case retention	*Retain*: case-base persistence, learning effect

4.2 Parameter taxonomy

Before an optimization problem and the objective can be defined, every parameter that varies across trials must be assigned a role. This section establishes that partition. It separates the quantities the optimizer treats as observed task conditions from those it actively searches over, and it identifies the measured outcomes that define the objective and the constraints. This partition is the structural foundation of the remainder of this work; the choice of which variable belongs in which role is defended here on the grounds of controllability and of the demands the optimization method places on the search space.

4.2.1 Context and decision variables

In its standard form, Bayesian optimization searches over a set of variables that the optimizer fully controls: a surrogate model is fitted over that search space and query points are selected to minimise an objective (Frazier, 2018).

In the problem considered here, however, a subset of the variables that influence the outcome are not controllable. They are fixed by the task and merely observed before the skill is executed. This is the contextual setting of Bayesian optimization (Krause & Ong, 2011), in which the objective depends jointly on a controllable decision vector and an observed context vector.

The variables are therefore partitioned by controllability. The user controls *how* the skill executes through the execution parameters which form the decision vector $\mathbf{x} \in \mathcal{X}$. The task defines *what* is to be handled and *where*: the identity and pose of the object, its physical properties, and the target placement. These conditions are read from the knowledge model, a world model providing the current state of the workspace, at the start of each job and describe the problem space to the optimizer; they form the context vector $\mathbf{z} \in \mathcal{Z}$. The objective is thus a function of the joint input $(\mathbf{x}, \mathbf{z})$:

$$f : \mathcal{X} \times \mathcal{Z} \to \mathbb{R}, \qquad (\mathbf{x}, \mathbf{z}) \mapsto E(\mathbf{x}, \mathbf{z}) \tag{4.1}$$

where E denotes the energy consumed by the execution. This criterion, with user-settable execution parameters on the one hand, and problem-describing conditions on the other, is what assigns each variable to $\mathcal{X}$ or $\mathcal{Z}$, and it is precisely this structure that the contextual formulation supports(Krause & Ong, 2011).

4.2.2 Context Variables

The context vector is eight-dimensional,

$$\mathbf{z} = (x_\mathrm{p},\, y_\mathrm{p},\, \theta_\mathrm{p},\, x_\mathrm{q},\, y_\mathrm{q},\, \theta_\mathrm{q},\, m,\, \mu) \in \mathcal{Z}, \tag{4.2}$$

comprising the planar pick position $(x_\mathrm{p}, y_\mathrm{p})$ and orientation θ_p, the planar place position $(x_\mathrm{q}, y_\mathrm{q})$ and orientation θ_q, the object mass m, and the contact friction coefficient μ.

These dimensions are chosen to span every conceivable source of skill variation that determines feasibility and cost. Two aspects govern whether, and at what cost, a given parameterization succeeds. The first is the geometric workspace and the state of the object in the world, which is determined by the pick and place poses. The second is the peak acceleration permitted to retain

the object without slip, which indirectly gives us the force required for grip. The Coulomb friction model determines this using the friction coefficient, the mass, and the motion-induced acceleration. The context set is exactly the set of task conditions that drive these two phenomena; dimensions with no bearing on feasibility or cost are not admitted.

Both orientations are restricted to the half-open interval $[0°, 180°)$. The object geometry and the parallel-jaw gripper are each invariant under a half-turn about the grasp axis, so orientations differing by 180° are physically equivalent for grasping. Exploiting this symmetry halves the extent of each orientation dimension and reduces the volume of the context space that must be characterized, a meaningful saving, since the effort required to cover a space grows rapidly with its dimension and extent (Shahriari et al., 2016).

4.2.3 Decision variables

The decision vector is two-dimensional,

$$\mathbf{x} = (s_v, s_a) \in \mathcal{X}, \tag{4.3}$$

comprising the velocity scaling s_v and the acceleration scaling s_a applied to the skill's trajectory. These are the execution knobs the user can set without altering the skill itself, and they directly modulate the trade-off between the energy consumed and the cycle time of the trial.

The low dimensionality of the decision space is an important observation. The sample efficiency of Bayesian optimization, which is its capacity to locate an optimum in a few expensive evaluations, degrades as the dimension of the search space grows, and the method is best suited to low-dimensional problems (Frazier, 2018). Because every evaluation here is a complete simulated skill execution, the number of online evaluations must remain small; a two-dimensional decision space is what renders convergence within such a budget attainable.

Another execution parameter that the skill exposes but is deliberately excluded from the decision vector was the force exerted by the gripper. The grip force is not a free variable: given the object mass, the friction coefficient, and the peak acceleration observed during execution, the force required to prevent slip is determined by the Coulomb friction model. It is therefore computed deterministically and enforced as a constraint rather than searched over; the constraint formulation is developed in section 4.3.

4.2.4 Evaluation metrics

The outcomes measured from each execution are assigned either to the objective or to the constraint set. The energy E consumed over the execution is the objective, and is to be minimized. The cycle time T is a constraint: the execution must complete within the deadline $T_{\max}$ specified for the skill. Grasp stability is a constraint: the derived grip force F_{req} must not exceed the user-defined maximum force $F_{\max}$ so that the object is retained without slip. Finally, the success of the execution is a binary constraint that distinguishes feasible parameterizations from those that fail outright. How each quantity is observed during execution is described in section 4.4, and its formal incorporation into the optimization problem follows in section 4.3.

Table 4.2 summarizes the full taxonomy and the source or controlling agent of each variable.

The taxonomy places the problem in the class of constrained, single-objective Bayesian optimization with an observed context: a scalar energy objective, three inequality constraints on cycle time, force,

Table 4.2: Parameter taxonomy: each variable, its symbol, its role in the optimization problem, and the agent that determines its value.

Variable	Symbol	Role	Source / control
Pick position	$x_\mathrm{p}, y_\mathrm{p}$	Context	handling Skill (knowledge model)
Pick orientation	θ_p	Context	handling Skill (knowledge model)
Place position	$x_\mathrm{q}, y_\mathrm{q}$	Context	handling Skill (knowledge model)
Place orientation	θ_q	Context	handling Skill (knowledge model)
Object mass	m	Context	handling Skill (knowledge model)
Friction coefficient	μ	Context	handling Skill (knowledge model)
Velocity scaling	s_v	Decision	user / tunable
Acceleration scaling	s_a	Decision	user / tunable
Energy	E	Objective	Monitoring Node / measured
Cycle time	T	Constraint	Monitoring Node / measured
Grip force	F_req	Constraint	Slip detection / derived from a_peak
Success	—	Constraint	Monitoring Node / measured

and success, and a partition into a controlled decision and an observed context. Its formal statement, the noisy constrained acquisition that solves it, and the supporting literature are developed in section 4.3.

4.3 Problem formulation

For a given optimization run, the context $\mathbf{z}$ is observed and held fixed (section 4.2.2); the optimizer then searches the two-dimensional decision space $\mathcal{X} = [0.05,\, 1.0]^2$ for the velocity and acceleration scalings that minimise energy while meeting the constraints. All quantities below are written as functions of the decision vector $\mathbf{x} = (s_v, s_a)$, with the conditioning on the fixed context left implicit. The problem is stated first in its nominal form (the constrained problem one would pose if every outcome could be evaluated exactly) after which the mechanism by which it is actually solved under surrogate uncertainty is described.

4.3.1 Nominal constrained problem

Feasibility takes precedence over Optimization: a policy must first satisfy the cycle-time deadline, the grasp-stability requirement, and successful completion, and only among those feasible policies is the energy E then minimized. This ordering is expressed directly by the constrained formulation, in which the constraints define the admissible set and the objective acts only within it:

$$\begin{aligned} \min_{\mathbf{x}\in\mathcal{X}} \quad & E(\mathbf{x}) \\ \text{s.t.} \quad & T(\mathbf{x}) \le T_\mathrm{max}, \\ & F_\mathrm{req}(\mathbf{x}) \le F_\mathrm{max}/S, \\ & s(\mathbf{x}) = 1, \end{aligned} \tag{4.4}$$

where T_max is the cycle time deadline specified for the skill, F_max is the maximum holding force the gripper is allowed to supply, $S \ge 1$ a safety factor (section 2.4), and $s(\mathbf{x}) \in \{0, 1\}$ indicates a

successful execution. Energy is reduced only with respect to these constraints; a lower-energy policy that violates any of them is rejected in favour of a feasible one. The cycle time T and the energy E are measured directly from the Monitoring Node described in section 4.4. The required holding force F_{req} is not measured but derived per trial from the observed peak acceleration $a_{\text{peak}}(\mathbf{x})$ through the Coulomb friction model (eq. (4.7), section 4.4). Because the context fixes the object mass, friction, contact count, and safety factor, the decision vector enters the force constraint only through a_{peak}: changing (s_v, s_a) alters the required force solely by altering the peak acceleration, which is the observed kinematic quantity that the surrogate models.

4.3.2 Constraint handling under surrogate uncertainty

The nominal problem cannot be solved directly. The energy, cycle time, and peak acceleration are black-box outcomes of a stochastic simulation, observed with noise and accessible only through the Gaussian-process surrogates fitted during the online phase, and the success indicator is a noisy binary outcome. The constraints can therefore never be verified exactly, only estimated under uncertainty; selecting a candidate on its posterior-mean prediction alone would ignore that uncertainty and admit points whose feasibility is marginal.

Rather than enforcing the constraints through explicit probability thresholds, feasibility is handled inside the acquisition function. In its analytic form, the constrained expected improvement of Gardner et al. (2014) scales the expected improvement in the objective by the posterior probability that the candidate satisfies the constraints,

$$\alpha_{\text{cEI}}(\mathbf{x}) = \text{EI}(\mathbf{x}) \cdot \prod_i \Pr\big(c_i(\mathbf{x}) \le 0\big), \tag{4.5}$$

where $\{c_i\}$ are the constraints of the nominal problem, written in the feasibility form $c_i(\mathbf{x}) \le 0$: the cycle time margin $T(\mathbf{x}) - T_{\max}$, the force margin $F_{\text{req}}(\mathbf{x}) - F_{\max}$, and the success requirement. Each factor $\Pr(c_i(\mathbf{x}) \le 0)$ is the posterior probability, under the surrogate for that outcome, that the corresponding constraint holds. Predicted improvement is thus rewarded only to the extent that the point is also likely to be feasible, and a low satisfaction probability on any single constraint suppresses the whole acquisition. Because the objective and constraints are observed through noisy executions, the noisy expected-improvement formulation of Letham et al. (2019) is used in place of the analytic form: the feasibility weighting is evaluated by quasi-Monte-Carlo sampling over the joint surrogate posterior, with each posterior sample of the objective improvement weighted by whether the constraints are satisfied in that same sample, which avoids assuming independence between the constraints. The implementation is provided by the Ax/BoTorch framework (Balandat et al., 2020).

Feasibility is therefore incorporated as a soft, continuous weighting rather than imposed as a hard cut-off. Candidates with high expected energy savings but uncertain constraint satisfaction are not discarded outright; instead, they are penalized in proportion to their probability of infeasibility. The resulting acquisition criterion thus implements a risk–reward trade-off, favoring regions that combine high expected performance with a high likelihood of satisfying the constraints. In this sense, the approach operationalizes the principle underlying chance-constrained Bayesian optimization (Gelbart et al., 2014): prioritizing points whose constraints are likely to hold, while avoiding the need to specify hard probability thresholds for individual constraints, none of which are configured in this work.

4.4 Monitoring loop

The problem formulation in section 4.3 expresses the optimization target as a single objective subject to three inequality constraints. For this formulation to be actionable, each quantity it references must be *measured* from a completed skill execution and returned to the optimizer as a scalar. The monitoring subsystem fulfills this role: it converts raw simulation telemetry into the per-trial KPIs that constitute one observation $\big(\boldsymbol{x}, f(\boldsymbol{x}), \boldsymbol{c}(\boldsymbol{x})\big)$ in the optimization loop. Here $\boldsymbol{x}$ is the decision vector the optimizer chose for the trial, $f(\boldsymbol{x})$ is the resulting objective value (the measured energy), and $\boldsymbol{c}(\boldsymbol{x})$ is the vector of resulting constraint values (cycle time, required grip force, and success).

A central design principle governs the subsystem: monitoring is strictly *observational*. The monitor nodes subscribe to data streams and accumulate metrics, but they never publish commands, alter the skill, or otherwise influence execution. This separation is deliberate. Because the surrogate models are trained directly on the measured KPIs, any feedback from the monitors into the controlled process would couple the measurement to the quantity being measured and contaminate the training data. Keeping the monitors passive ensures that the variance observed across trials is attributable to the decision and context variables rather than to the act of observing.

Measurement architecture

Measurement is distributed across two dedicated observer nodes that run alongside the skill stack. Each node owns one telemetry domain and exposes a query interface to the Skill Trial Runner, which orchestrates measurement on a per-trial basis. The first node derives the energy and the cycle time from the robot's joint-state stream. The second derives the peak acceleration of the held object from an integrated IMU on the object, gated by the gripper-occupancy state read from the Knowledge model (KM). The Skill Trial Runner itself contributes the two metrics that are intrinsic to the orchestration layer rather than to the simulation, namely the end-to-end wall-clock duration (cycle time) and the binary trial outcome (trial success).

Per-Trial measurement lifecycle

Every trial follows the same three-phase lifecycle, mediated by the Skill Trial Runner:

1. **Reset.** Before execution, the Skill Trial Runner zeroes all accumulators and records a time reference in each monitor node, so that the upcoming measurement window contains no residue from the previous trial.

2. **Execute.** The skill runs. The monitor nodes accumulate their metrics in real time as telemetry arrives, with no interaction with the executing skill.

3. **Read.** After execution completes, the Skill Trial Runner queries each node for its final accumulated values, assembles them with its own orchestration-level metrics, and forwards the resulting KPI tuple to the optimizer.

This reset–execute–read cycle gives each trial a well-defined and mutually disjoint measurement window, which is what allows a single execution to be treated as one independent observation by the surrogate models. Figure 4.1 summarizes the resulting data flow.

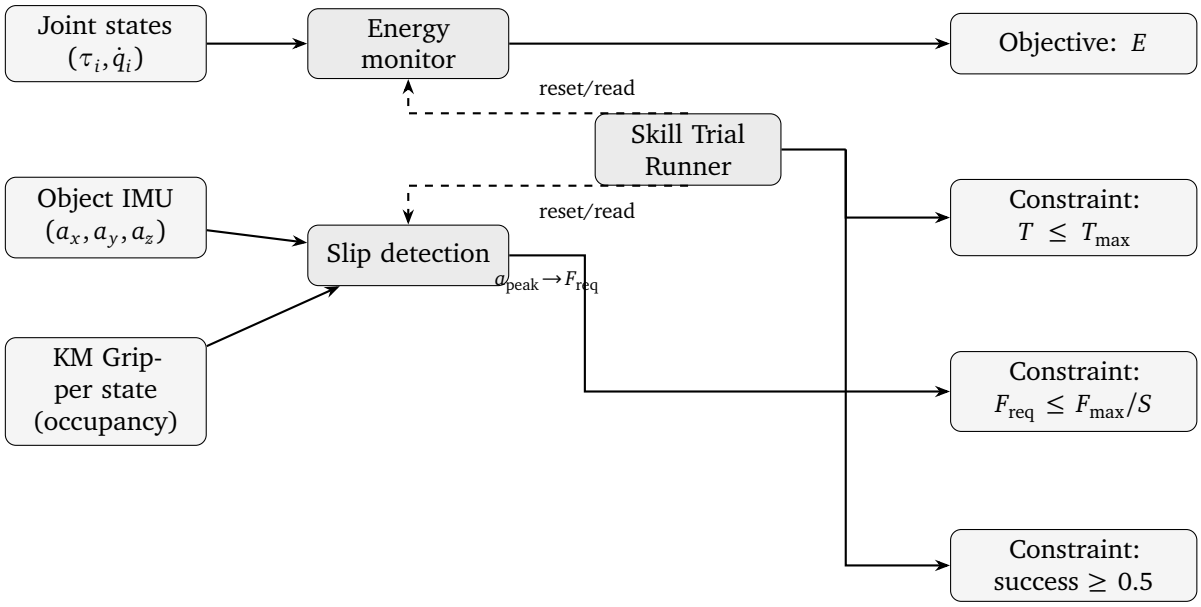


Figure 4.1: Data flow of the monitoring subsystem. Solid arrows denote telemetry and KPI flow; dashed arrows denote the Skill Trial Runner's reset and read calls. Each measured quantity maps to exactly one role in the optimization problem.

Cycle time constraint

Cycle time is the one KPI for which two distinct measurements are available, and the choice between them is a design decision with direct consequences for how faithfully the constraint transfers to hardware. The two notions differ both in the time source they use and in the interval they enclose.

ROS cycle time. The energy monitor reports a cycle time derived from the same window over which it integrates energy. It is timestamped with the Robot Operating System 2 (ROS2) clock which, with simulation time enabled, tracks the *simulation* clock rather than real time. The window opens when the Skill Trial Runner resets the accumulators before the skill is dispatched, and closes at the last joint-state message received before the metrics are queried, which follows a short post-execution settling delay. This measurement therefore, wraps a wider span than the skill itself: a brief setup interval precedes dispatch, and a brief settling interval follows completion, and the elapsed value is denominated in simulation time.

Wall-clock cycle time. The Skill Trial Runner independently measures the real-time duration enclosing *exactly* the skill execution from the instant the skill goal is dispatched to the instant the skill action returns, whether in success or failure. This interval contains the planning latency, all motion phases, gripper actuation, and controller settling, and nothing outside the skill. Figure 4.2 contrasts the two intervals on a single timeline.

Why the wall-clock duration is adopted. The constraint $T \leq T_{\max}$ encodes a user's cycle time requirement, the handling skill must complete within a specified number of seconds. Three considerations make the wall-clock duration the appropriate metric for that constraint:

1. **Deployment realism.** Planning latency, communication overhead, and controller settling are present on real hardware as well; the cycle time bound should reflect the duration a deployment would actually experience. The ROS value either omits these costs or measures them at a simulation-dependent rate that does not transfer to hardware.

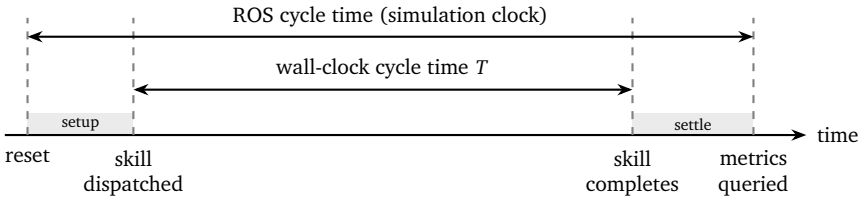


Figure 4.2: The two cycle time measurements. The ROS cycle time spans reset to the final telemetry sample before the query, absorbing a fixed overhead and using the simulation clock. The wall-clock cycle time T encloses the skill execution alone in real time.

2. **Simulation-clock skew.** Under varying GPU load the Isaac Sim physics clock advances at a variable rate relative to real time, so a fixed elapsed simulation time can correspond to different real durations. Anchoring the constraint to wall time keeps it tied to the quantity the cycle requirement is really about.

3. **Tighter measurement bounds.** The ROS interval systematically includes a fixed overhead (the pre-dispatch setup and the post-execution settling delay shaded in fig. 4.2))that is unrelated to the skill. The wall-clock measurement excludes this overhead and isolates the skill execution.

The ROS cycle time is nonetheless retained as a diagnostic: comparing it against the wall-clock value across trials reveals whether the simulation clock is drifting relative to real time. Only the wall-clock value T feeds the optimizer, entering as the inequality constraint $T(\boldsymbol{x}) \leq T_{\max}$ with $T_{\max}$ the user-specified cycle time bound established in section 4.3.

Grip force via peak acceleration Constraint

Of the four monitored quantities, the grip force is the only one that is never observed directly. The simulation models no contact force. The torque sensing at the fingertips and, more fundamentally, the contact interaction that such a sensor would report could not be made reliable in the first place. Establishing why leads to the central modeling decision of this subsection, that slip is evaluated analytically rather than simulated, and that decision in turn dictates how the constraint is measured.

Realizing the grasp through PhysX contact friction proved reproducibly unstable. With correct collision geometry, a physics material carrying appropriate static and dynamic friction, an active `finger_joint` drive (stiffness 3.0, damping 0.0002, `maxForce` 26.0), and confirmed finger to cube contact, the cube nonetheless slips from the fingers as the arm begins its lift. The failure is not specific to this setup: it is documented for the Robotiq 2F-85 across Isaac Sim and Isaac Lab versions throughout the developer community (Isaac Lab, 2026; NVIDIA, 2026; NVIDIA Developer Forums, 2026a, 2026b, 2026c). Because a successful lift is the precondition for measuring anything at all, an unreliable contact grasp would inject failures that reflect the simulator's contact solver rather than the parameterization under test, contaminating every downstream KPI.

The grasp is therefore realized at runtime as a `UsdPhysics.FixedJoint` between the gripper base link and the object, formed once the gripper closes on it. This substitution is benign for exactly the quantities the optimizer depends on. From a rigid-body-dynamics standpoint a perfectly retained grasp and a fixed joint are equivalent: the object's mass and inertia continue to load the kinematic chain, joint torques propagate correctly through all six axes, and the energy, cycle-time, Tool Center Point (TCP) velocity and acceleration, and object-IMU signals all remain physically valid. What the fixed joint discards is contact friction, and with it the emergent slip that a success-or-failure observation would otherwise carry.

That discarded information is precisely what the grip-force constraint must restore, and it motivates recovering slip analytically rather than reintroducing an unreliable contact event. The dynamic load the grasp must withstand is measured directly, and the clamping force required to resist it is computed from the Coulomb friction model (section 2.4); a grasp is then deemed infeasible whenever that required force exceeds the gripper limit. Feasibility is thereby decided deterministically, from an observed kinematic quantity, instead of from a contact outcome the simulator cannot be trusted to produce. The remainder of this subsection makes that measurement precise.

The slip-detection node reads the object's inertial sensor and forms the gravity-corrected acceleration magnitude,

$$a_{\text{corrected}} = \sqrt{a_x^2 + a_y^2 + (a_z - g)^2}, \qquad g = 9.81\,\text{m}\,\text{s}^{-2}, \tag{4.6}$$

and tracks its running maximum a_{peak}. This maximum is updated *only while the object is held*: the gripping window is opened and closed by the gripper-occupancy transitions read from the KM state (`free` → `occupied` opens it, `occupied` → `free` closes it). Gating the peak to this window excludes post-release transients, table impact and bouncing, that would otherwise inflate it with accelerations the grasp never had to resist.

The peak acceleration does not enter the optimizer as a quantity in its own right. It is transformed into the required clamping force through the Coulomb model,

$$F_{\text{req}} = \frac{m\,(g + a_{\text{peak}})}{n\,\mu}, \tag{4.7}$$

with m and μ the object mass and friction coefficient supplied by the context vector and $n = 2$ the number of opposing contact faces of the parallel-jaw gripper. Equation (4.7) maps the measured dynamic outcome to the force a successful grasp requires, which the constraint then bounds as $F_{\text{req}} \leq F_{\max}/S$. The safety factor S is applied at this constraint boundary alone, not inside the measured quantity, so that it can be re-tuned without altering the KPI fed to the optimizer.

Trial success constraint

The final KPI is the binary outcome of the trial, reported by the Skill Trial Runner once the skill terminates. A completed pick-and-place yields success; a failure (e.g. a dropped or unreachable object) yields failure. Treated as a constraint of the form success ≥ 0.5, this outcome lets the optimizer model a probability of feasibility over the decision space and steer sampling away from parameterizations that are physically infeasible, independently of how energy-efficient they might appear.

Energy objective

Energy is derived from the joint-state telemetry, which reports the effort τ_i and velocity $\dot{q}_i$ of each of the six joints at the physics-step rate. The instantaneous mechanical power is the sum of the absolute per-joint powers,

$$P(t) = \sum_{i=1}^{6} \left| \tau_i(t)\,\dot{q}_i(t) \right|, \tag{4.8}$$

and the trial energy follows from numerical integration over the measurement window,

$$E = \sum_{k} P(t_k)\,\Delta t_k, \tag{4.9}$$

with Δt_k the interval between successive telemetry messages. The absolute value is applied *per joint* so that both driving and braking contributions add to the total: the metric reflects the mechanical work the actuators deliver irrespective of its sign, rather than the net work, which would understate the actuation cost of motions that decelerate. The accumulated energy E is the single objective minimized by the optimizer over the decision vector $\boldsymbol{x} = (v_{\text{scale}}, a_{\text{scale}})$.

Summary

Table 4.3 consolidates the mapping from each monitored quantity to its role in the optimization problem. Only the measurement and transformation of the KPIs are the concern of this subsection; how the optimizer incorporates them through surrogate models and a feasibility-weighted acquisition function is the subject of section 4.5.

Table 4.3: Monitored KPIs and their role in the optimization problem.

KPI	Source	Optimizer role	Constraint
Energy E	Energy monitor	Objective (minimize)	—
Cycle time T	Skill Trial Runner	Constraint	$T \leq T_{\max}$
Peak acceleration a_{peak}	Slip detection	Derived $\rightarrow F_{\text{req}}$	$F_{\text{req}} \leq F_{\max}/S$
Trial success	Skill Trial Runner	Constraint	success ≥ 0.5

4.5 Optimization architecture

Section 4.3 specified *what* is optimized: a single energy objective minimized over the two-dimensional decision space $\mathcal{X} = [0.05, 1.0]^2$ subject to the cycle-time, grip-force, and success constraints, solved through a feasibility-weighted acquisition under surrogate uncertainty. This section specifies the *architecture* that solves that problem for each incoming task. Two properties of the setting shape it. First, the objective depends on an eight-dimensional context that is observed rather than searched (section 4.2.2), so the system must re-solve a fresh two-dimensional problem for every distinct context. Second, each online evaluation is a full simulated execution costing on the order of tens of seconds of wall-clock time (section 4.4), so the per-context budget is small and must be spent only where it can pay off.

The architecture answers both demands with a two-phase decomposition. An *offline database* builds a context-indexed memory of past executions at a fixed reference policy. An *online decision phase* then serves each query by reusing that memory. The online phase is organized around the CBR cycle of *Retrieve*, *Reuse*, *Revise*, *Retain* (section 3.3.4), which supplies the control flow of the pipeline. The contextual constrained BO loop of section 4.3 is the engine of the *Revise* step, invoked only for contexts that have not been solved before, and the offline memory is what warm-starts it so that revision converges within the available budget.

4.5.1 Offline dataset

The split into an offline characterization phase and an online decision phase is the most consequential decision in the architecture, and it is forced by the cost structure of the problem. The space spanned

jointly by the eight-dimensional context vector $\mathbf{z}$ and the two-dimensional decision vector (s_v, s_a) cannot be sampled densely: the number of samples needed to cover it at a fixed resolution grows exponentially with dimension, the curse of dimensionality (Bellman, 1961), and each sample is expensive, since a single pick-and-place execution in Isaac Sim takes approximately 30 s to 50 s of time, and cannot be accelerated without disturbing the very physics the measurement depends on. A joint sweep dense enough to serve as a cost surrogate over both spaces is therefore out of reach. The decomposition resolves this by sampling only the context space offline, at a single fixed policy, and deferring the decision-variable search to a short online loop run per query.

Concretely, the offline phase constructs an *experience memory*, a store of context-conditioned outcomes against which online queries are answered by similarity. A QMCS Sobol sequence (section 2.1) generates $N \approx 4000$ points spanning the context space $\mathcal{Z}$, and each point is evaluated once at the fixed reference policy

$$\mathbf{x}_0 = (s_v, s_a) = (0.3,\, 0.3), \tag{4.10}$$

the conservative operating point from which the online search later departs. The roughly 4000-trial budget is already large under the per-trial cost above. Each trial records the three outcomes the optimizer depends on: the energy E, the cycle time T, and the peak TCP acceleration a_{peak} from which the required grip force is derived (section 4.4). After filtering to successful, non-degenerate trials, the retained data form the memory.

$$\mathcal{M} = \left\{ \left(\mathbf{z}_i,\, E_i,\, T_i,\, a_{\text{peak},i} \right) \right\}_{i=1}^{N}, \tag{4.11}$$

indexed spatially for k-nearest-neighbour (k-KNN) retrieval in the scaled context metric.

The benefits of this decomposition come with an important limitation that must be stated explicitly. Because the decision variables remain fixed throughout the parameter sweep, the resulting dataset contains no information about how system performance varies with (s_v, s_a). Consequently, the data *cannot* be used to construct a surrogate model of the cost function over the decision space; any surrogate fitted to them would observe no variation in the decision variables and collapse to a context-only model.

The role of $\mathcal{M}$ is therefore deliberately limited. It acts as a context-feasibility screen and provides a warm-start reference policy, from which the online constrained BO loop performs the actual optimization over (s_v, s_a) for each context. The decomposition makes the offline collection tractable and yields an informed initialization, at the cost of any decision-dependent cost model learned offline: the dataset supports claims about feasibility and initialization, but not about the structure of the cost landscape over the decision variables.

4.5.2 Online decision cycle

A user query supplies an observed context $\mathbf{z}_q$ together with the operating limits $(T_{\max}, F_{\max})$. The query is then processed by the four-R cycle, whose stages are ordered from cheapest to most expensive so that costly simulation is reached only when nothing cheaper resolves the request. Figure 4.3 shows the resulting flow; table 4.4 collects the configuration of the online loop.

Physics and input gate

The first stage is a deterministic, zero-cost screen that enforces the input and static-feasibility requirements (C-1, C-4). It rejects contexts that fall outside the workspace bounds, and it checks the

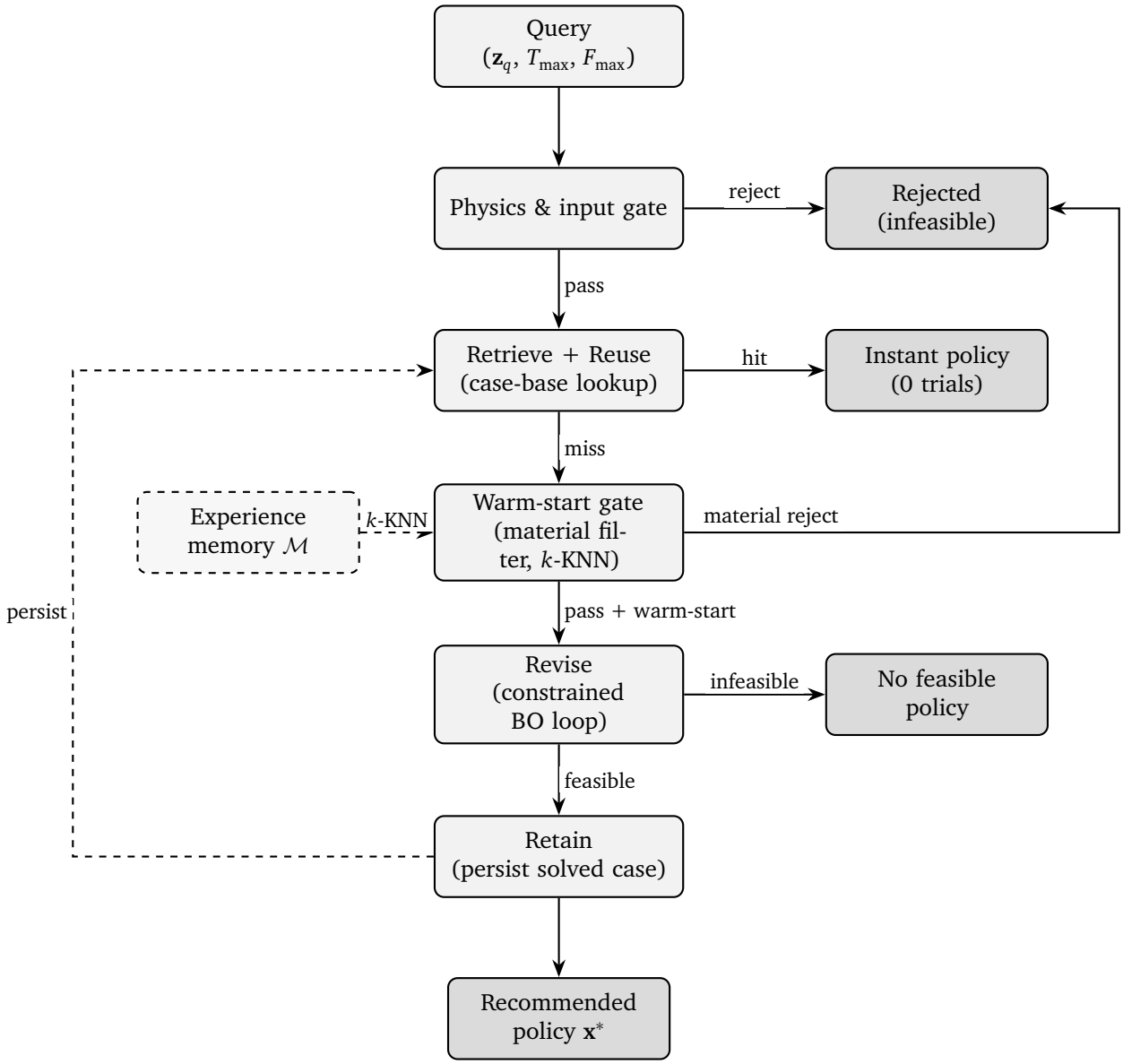


Figure 4.3: The online decision cycle as a four-R loop. Stages are ordered cheapest-first: a deterministic physics gate, an exact-match case-base lookup (*Retrieve* + *Reuse*), a memory-based warm-start gate, the constrained BO loop (*Revise*), and case persistence (*Retain*). Solid arrows are the per-query flow; dashed arrows are the offline-memory lookup and the retain feedback. The material filter is the only hard rejection inside the warm-start gate.

static-hold floor: a grasp that cannot even support the object at rest can never be made feasible by any choice of motion. Setting $a_{\text{peak}} = 0$ in the required grip force of eq. (4.7) and applying the safety factor S gives this floor as

$$\frac{S\, m\, g}{n\, \mu} \;\leq\; F_{\max}, \tag{4.12}$$

with m and μ read from $\mathbf{z}_q$, $n = 2$ contact faces, and S the safety factor. A violation here is definitive: the query is rejected before any trial budget is committed, and the binding constraint is reported back so that the user can relax it to a realistic value (C-2).

Retrieve and reuse: case-based lookup

The case base stores the solved $(\mathbf{z}, \mathbf{x}^*)$ pairs produced by earlier runs. the *Retain* products of past optimizations. *Retrieve* searches it for cases whose context matches $\mathbf{z}_q$ within a tolerance. *Reuse* then filters the retrieved cases by constraint compatibility with the current query, keeping those whose stored cycle time and required force satisfy the present $(T_{\max}, F_{\max})$, and returns the lowest-energy survivor. A hit yields an instant policy recommendation at zero simulation cost: when a context has already been solved under compatible limits, no further search is warranted. A miss passes the query to the warm-start gate.

Warm-start

For a novel context, the experience memory $\mathcal{M}$ is queried to furnish the optimizer with an informed starting point. A *material pre-filter* rejects the query if no offline data lie within the mass/friction tolerance envelope, since outcomes derived at from sampled objects that differ considerably would be uninformative; this is the only hard rejection at this stage. Spatial distance, by contrast, is treated as informational rather than as a gate: geometric novelty does not by itself imply infeasibility, so a spatially distant context is admitted and merely flagged. A context beyond the auto-calibrated threshold $d_{\max}$ (the 95th percentile of in-sample nearest-neighbor distances) raises a *far-from-data* flag, marking its warm-start as less reliable without barring it from optimization. The offline gate thus screens on physical material feasibility, not on spatial proximity.

For an admitted context, the $k = 20$ nearest neighbours $\mathcal{N}_k(\mathbf{z}_q)$ in the scaled context metric supply, for each outcome $y \in \{E,\, T,\, a_{\text{peak}}\}$, a warm-start mean and an empirical noise estimate,

$$\hat{y}(\mathbf{z}_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{z}_q)} y_i, \qquad \hat{\sigma}_y(\mathbf{z}_q) = \operatorname{std}_{i \in \mathcal{N}_k(\mathbf{z}_q)}\big(y_i\big). \tag{4.13}$$

The mean $\hat{y}$ is the expected outcome at the reference policy $\mathbf{x}_0$, and the dispersion $\hat{\sigma}_y$ estimates the outcome variance the surrogates should expect in this region of context space. Both are handed to the *Revise* step as context-adapted priors.

Revise: constrained Bayesian optimization

Revise runs the contextual constrained BO loop of section 4.3 to find the per-context decision vector $\mathbf{x}^* = (s_v, s_a)$. Each of the four outcomes, energy, cycle time, Force required, and success, is modeled by an independent GP surrogate, and the loop selects the next trial by the feasibility-weighted noisy expected-improvement acquisition derived in section 4.3.2, realized through the Ax/BoTorch framework. The loop proceeds in two phases under a total budget of 30 trials.

The *seed phase* (6 trials) initializes the surrogates. The first is a *synthetic anchor*: rather than executing a trial, it injects an observation at the reference policy $\mathbf{x}_0$ whose outcomes are the k-KNN warm-start means retrieved from the offline memory (section 4.5.2), with their empirical dispersion supplied as the observation noise. This gives the GPs a data-grounded estimate at $\mathbf{x}_0$ without spending a simulation on a point the memory already characterizes. The remaining five seed trials are a Sobol space-filling sample of $\mathcal{X}$, each physically executed. Two mechanisms keep the loop within budget and robust to failures. *Early stopping* terminates the run when the best feasible energy improves by less than 1 % over a five-trial window, counted from the first feasible observation rather than from the start, and subject to a minimum of 15 post-feasibility iterations before the check activates so that the budget is not consumed refining an already-converged optimum. However *Failure handling* assigns a failed trial a pessimistic penalty energy, the running maximum feasible energy inflated by a fixed factor, reported with a large standard error, so that the objective GP treats the failure as uninformative for energy while the success-constraint GP learns to steer sampling away from the failure-prone region. On termination, the policy with the lowest energy among all feasible trials is extracted as $\mathbf{x}^*$. If no trial proved feasible, the agent reports honest infeasibility and the binding constraints (C-3) rather than returning an unsafe recommendation.

Retain: case persistence

When a feasible policy is found, the solved pair $(\mathbf{z}_q, \mathbf{x}^*)$ is persisted to the case base. This closes the four-R cycle and produces a learning effect: each solved query makes the system faster on similar future queries, since a later exact match is answered by *Retrieve* + *Reuse* at zero cost. Over time the case base grows to cover the frequently queried regions of context space, shifting the system from expensive simulation-based revision toward cheap memory-based recall, while novel contexts continue to be served efficiently by the warm-started loop. The architecture therefore spans a continuum from instant recall for repeated contexts to informed exploration for unseen ones.

Table 4.4: Configuration of the online constrained BO loop.

Parameter	Symbol	Value
Decision space	$\mathcal{X}$	$[0.05, 1.0]^2$
Reference / anchor policy	$\mathbf{x}_0$	(0.3, 0.3)
Warm-start neighbours	k	20
Seed trials	—	6 (1 anchor + 5 Sobol)
GP-guided trials	—	≤ 24
Total trial budget	—	30
Early-stopping window	—	5 trials, < 1 % gain

Chapter 5 Implementation

This chapter describes how the design of Chapter 4 is realized in software on the ROS2, Isaac Sim, and MoveIt2 stack. The overall goal is a working agent, so the chapter moves from the infrastructure that executes and measures a single trial to the optimization pipeline that turns those trials into a recommended policy.

Section 5.1 introduces the components, the interfaces over which they communicate, the Skill Trial Runner that mediates between the optimization layer and simulation, and the hardware platform being modeled. Section 5.2 describes the pre-existing pick-and-place skill the agent parameterizes and how it is driven, and Section 5.3 details the reset, execute, and read lifecycle of a single trial. Section 5.4 then specifies the energy monitor and the slip-detection node that convert data into the evaluation metrics.

The remaining sections follow the path a query takes through the pipeline. Section 5.5 builds the offline dataset through the offline context sweep, Section 5.6 screens a query at the feasibility gates before any trial budget is spent, Section 5.7 seeds the optimizer from memory, Section 5.8 performs the per-context constrained Bayesian optimization search, and Section 5.9 retains and retrieves solved cases to close the four-R cycle.

5.1 System architecture

The developed system is organized into functional components that communicate with one another over defined interfaces:

- the physics-based simulation environment Isaac Sim (NVIDIA, 2023), which hosts the robot and workspace
- a monitoring layer of independent data nodes that convert raw simulation telemetry into the per-trial metrics
- the motion planner MoveIt2 (Coleman et al., 2014)
- the skill stack, which exposes the pre-existing pick-and-place skill behind a single action server (section 5.2)
- an optimization layer built on BoTorch/Ax (Balandat et al., 2020)

A dedicated orchestration node, the Skill Trial Runner (section 5.3), mediates between the optimization layer and the remaining components, so that the optimization logic is never coupled directly to simulation control, motion planning, or ROS2 orchestration. Figure 5.1 provides an overview of how these components interact.

Isaac Sim hosts the simulated robot and its surrounding workspace. By modeling joint dynamics, inertia, and object physics, it generates the joint states and sensor data that are forwarded to the rest of the system through the ROS2 bridge.

Communication among the components is handled centrally by ROS2, with joint states, control commands, and sensor data flowing across topics, services, and actions. As a standardized framework embedded directly in ROS2, MoveIt2 plans the trajectories that realize the skill, using the velocity and acceleration scalings supplied by the optimizer. The link between Isaac Sim and the ROS2 interfaces is established through the ROS2 bridge together with the ROS2 simulation-control extensions.

Monitoring layer

The metrics that drive the optimizer's next decision are produced not by a single node but by two independent data nodes, launched together and running alongside the skill stack. The *energy monitor* subscribes to the joint-state stream and computes the consumed energy by integrating the instantaneous joint power—the product of joint effort and velocity—over the execution window, while also tracking the cycle time. The *slip-detection* node subscribes to the IMU that is mounted on the manipulated object (the cube) in Isaac Sim, from which it extracts the peak object acceleration during the gripping phase. Because the object is rigidly coupled to the gripper through a fixed joint while it is held (section 4.4), this object acceleration effectively equals the TCP acceleration a_{peak}, the quantity the constraint formulation requires. The required grip force itself is *not* learned: it is derived analytically per trial from a_{peak} through the Coulomb friction model of section 4.4, with the optimization layer receiving F_{req} as an observation and feeding it to the constraint surrogate. Once a trial finishes, the accumulated metrics are queried by the Skill Trial Runner and forwarded to the optimization layer, where they inform the evaluation of the objective and constraints.

Separation of monitoring and optimization

Isolating the monitors in their own nodes is intentional. The monitors must process a continuous telemetry stream, the cube IMU and the joint states arrive at tens to roughly a hundred hertz throughout a trial. In contrast, the optimization layer is blocked inside a single synchronous `run_cycle()` call until the execution finishes. Folding both responsibilities into one class would force the blocking optimization call to coexist with continuous stream handling in the same control flow, weakening the system's modularity and adding needless complexity to the architecture. Keeping the monitors as passive, independently scheduled subscribers avoids this and preserves the observational separation on which the surrogate models depend.

Optimization layer

The BoTorch/Ax optimization layer holds the optimization logic itself. For each trial it determines a parameter configuration, passes it to the Skill Trial Runner together with the current task context, and receives the resulting observations in return. These update the surrogate model, after which the acquisition function selects the next configuration to evaluate. The layer thus operates purely at the level of the optimization loop and is shielded from simulation control, sensor handling, and ROS2 orchestration.

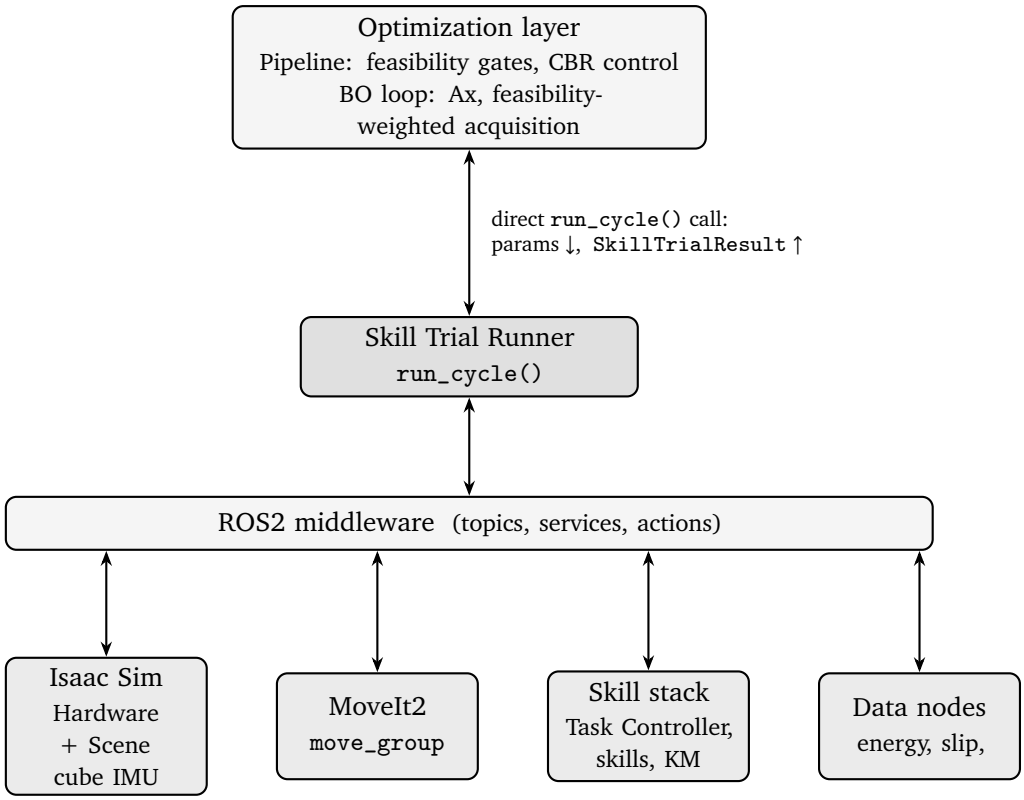


Figure 5.1: Overview of the system architecture. The optimization layer reaches the simulator only through the Skill Trial Runner, by a direct in-process `run_cycle()` call. Every other component communicates over the ROS2 middleware. This keeps the optimization logic fully decoupled from simulation control, motion planning, and sensor handling.

5.1.1 Skill trial runner

The Skill Trial Runner is a dedicated ROS2 node that encapsulates the full lifecycle of a single pick-and-place trial. It first resets the simulation to a defined initial state—pausing the physics, repositioning the arm and the object, injecting the trial's mass and friction into the object in Isaac Sim, and re-seeding the KM accordingly. Then dispatches the pick-and-place skill to the Task Controller action server. The Task Controller assembles the action plan into a state machine and runs it. The skill, in turn delegates the motion planning to MoveIt2 (section 5.2). After execution, the runner collects the resulting metrics from the data nodes through their respective service interfaces. It exposes a single public method that takes a set of context and decision parameters and returns a structured result. In doing so, it decouples the optimization logic from the specifics of simulation control, sensor-data collection, and ROS2 service orchestration: the optimization layer never addresses Isaac Sim or MoveIt2 directly, but calls the Skill Trial Runner once per trial and receives the observations required to update the surrogate model.

5.1.2 Hardware choice

The robotic manipulator modeled in this work is the **UR30** from Universal Robots, paired with the **Robotiq 2F-85** two-finger adaptive gripper. Figure 5.2 shows both components.

UR30 arm

The UR30 is the highest-payload collaborative robot in Universal Robots' line-up, built for heavy-duty pick-and-place, machine tending, and palletizing. Table 5.1 summarizes its key specifications.

Table 5.1: UR30 key specifications (Universal Robots A/S, 2026).

Parameter	Value
Payload capacity	30 kg (up to 35 kg with reduced center-of-gravity offset)
Reach	1300 mm
Degrees of freedom	6 (revolute joints)
Repeatability	±0.1 mm
Weight	63.5 kg
Footprint diameter	∅245 mm
IP rating	IP54 (arm; control box IP44)
Safety functions	17 (EN ISO 13849-1, PLd Cat. 3)
Communication	Modbus TCP, PROFINET, EtherNet/IP

The UR30 is integrated into the ROS2 ecosystem through the official `Universal_Robots_ROS2_Driver`, which exposes the standard `joint_trajectory_controller` interface consumed by MoveIt2.

Robotiq 2F-85 gripper

The Robotiq 2F-85 is an adaptive two-finger parallel gripper for versatile pick-and-place operations. Its under-actuated finger mechanism conforms to a range of object geometries without dedicated fingertips. Table 5.2 lists its key parameters.

Table 5.2: Robotiq 2F-85 key specifications (Robotiq Inc., 2026).

Parameter	Value
Stroke	85 mm
Grip force range	20–235 N
Closing speed (adjustable)	20–150 mm/s
Recommended payload	5 kg
Weight	0.9 kg
Position resolution (fingertip)	0.4 mm
Communication	Modbus RTU (RS-485)
Form factor	Two-finger adaptive parallel

In the simulation environment, the gripper is modeled using a simplified kinematic chain based on the `ros2_robotiq_gripper` package, with a rigid joint (*FixedJoint*) coupling the grasped object to the gripper TCP during the holding phase. The grip force itself is not physically resolved through contact friction; instead, the analytical Coulomb friction model (section 4.4) determines whether the commanded force is sufficient to prevent slip at the observed peak acceleration. This is precisely why the analytical model is needed. since the simulator does not physically resolve grip slippage, it acts as a physics-informed surrogate for the slip constraint (section 4.4).



(a) UR30 robotic arm (b) Robotiq 2F-85 gripper

Figure 5.2: Robotic manipulator components used in this work: (a) the UR30 arm (Universal Robots A/S, 2026) and (b) the Robotiq 2F-85 gripper (Robotiq Inc., 2026).

5.2 The pick-and-place skill

The capability the agent optimizes is a pre-existing compound pick-and-place skill, part of a previous work at the institute and is not a contribution of this work. From the agent's standpoint, it is a black box: it accepts a parameterized action plan, executes a complete handling cycle—approach, grasp, transport, place, release—and reports an outcome. This section describes the skill only at the level the agent needs, namely the single interface it dispatches into and the parameters it sets; the skill's internal control logic is treated as opaque throughout. The mechanism that drives this interface on a per-trial basis, the Skill Trial Runner, follows in section 5.3.

Architecture and contact surface.

The skill is organized as a three-tier hierarchy, and the agent communicates with the top tier alone. A ROS2 *Task Controller* node exposes one action server that accepts a JSON action plan, assembles a state machine from it, and runs that machine to a terminal outcome of success or abort. Below this entry point, a tier of composite skills (the `PickAndPlace` skill, which in turn invokes `Pick` and `Place`) is driven by behaviour trees that handle grasp-frame selection and approach planning with retries, and a lowest tier of primitive skills realizes the motion through point-to-point and linear Cartesian moves planned by MoveIt2 and an open/close command to the Robotiq gripper controller. The two inner tiers execute autonomously and are never addressed by the agent: the agent submits one action plan to the Task Controller and waits for the terminal outcome, so the entire stack reduces, from its perspective, to a single parameterized call (fig. 5.3).

Architectural grounding

This organization follows established practice in skill-based robot control for flexible, small-batch production rather than being purpose-built for this work. Its decomposition into modular, recombinable skills coordinated by a generic task layer is the central idea of the modular skill framework of Heuss et al. (2019), in which a generic task control serves varying incoming tasks by composing the robot's available skills, and which Heuss et al. (2022) extends toward easily reconfigurable industrial applications by treating task control, knowledge management, and planning as modularly integrable components. Coordinating these skills through behaviour trees is supported by the empirical study of Ghzouli et al. (2023), who find that behaviour trees lend mission-level coordination an extensible and reusable structure while keeping the underlying skills architecturally separate, and who likewise situate the state machine that governs the top tier here as the established alternative. The same pattern, modular skills composed into behaviour trees over a shared world model, underlies the SkiROS2 platform of Mayr et al. (2023), whose knowledge base for reasoning about the world state plays the role the KM plays here. Mayr et al. (2023) further demonstrate that such a skill platform is a natural substrate for a learning layer that tunes skill execution. This is precisely the relationship the optimization agent assumes, treating the skill as a fixed, parameterized capability and searching only over the execution parameters it exposes.

Execution interface

The action plan the agent submits carries four execution parameters. Two of them are the decision variables of the optimization, the velocity and acceleration scalings (s_v, s_a) that the agent searches over (section 4.2.3). They propagate unchanged through the composite tier to every motion segment, where they scale the planner's velocity and acceleration limits. The remaining two, the commanded grip force and the Cartesian approach distance, are held fixed at values taken from the physics-informed reference policy and are not searched. The commanded grip force is the clamping force handed to the gripper at grasp time. It is held at a sufficient fixed value rather than optimized, while grasp feasibility itself enters the problem as a deterministic constraint evaluated downstream from the observed peak acceleration, not as a quantity the skill negotiates (sections 4.2.3 and 4.4). Table 5.3 summarizes the four parameters and their propagation.

Table 5.3: Execution parameters carried by the action plan, their role in the optimization, and their effect inside the skill.

Parameter	Role	Effect in the skill
Velocity scaling s_v	Decision (searched)	Scales the planner's velocity limits on all motion segments
Acceleration scaling s_a	Decision (searched)	Scales acceleration limits; sets the peak TCP acceleration a_{peak}
Grip force	Fixed	Clamping force commanded to the gripper at grasp. Held at a reference value
Approach distance	Fixed	Cartesian approach and retreat path length around the grasp point

Coupling to the world model and simulator

Two couplings of the skill are relevant to the agent because they condition what a trial measures, though both are driven by the runner rather than by the optimizer. First, the skill reads object and

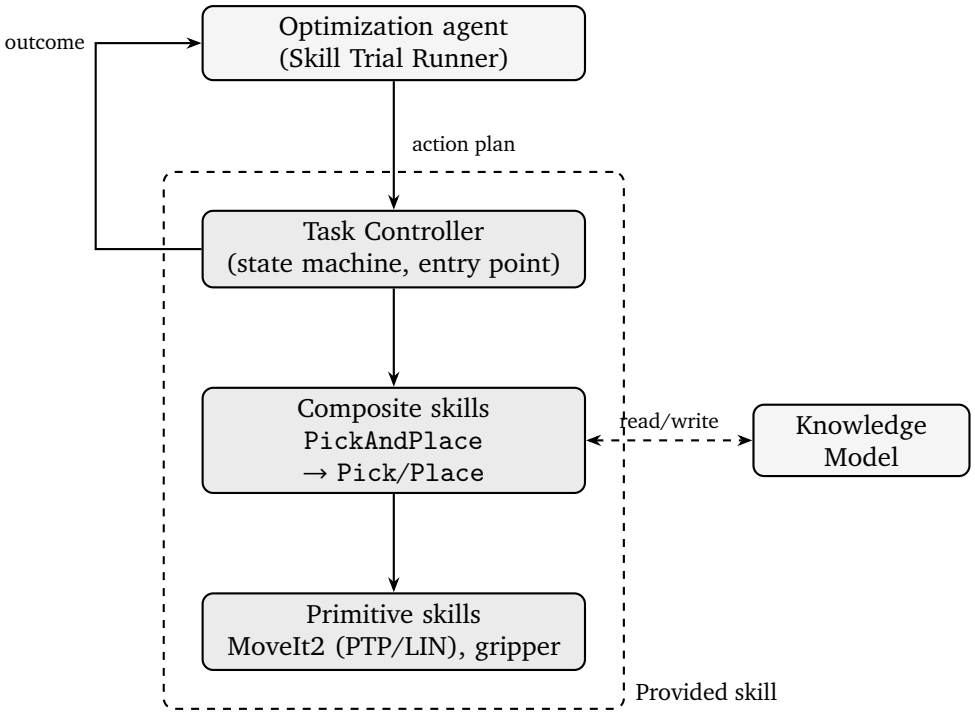


Figure 5.3: The three-tier skill and the agent's single contact surface. The agent submits one action plan to the Task Controller and receives an outcome; the composite and primitive tiers, and the skill's coupling to the KM, execute autonomously and are opaque to the optimizer.

gripper properties—pose, mass, and the grasp frames—from the Knowledge Model (KM), the world model of the workspace, and writes back occupancy and attachment state after each grasp and release. So that every trial starts from an identical world state, the Skill Trial Runner re-seeds the KM before dispatch (section 5.3).

Second, a stable grasp is realized in simulation not through contact friction but through a rigid attachment between the object and the gripper, the deliberate fidelity trade-off argued in section 4.4. Per-trial mass and friction are likewise injected into the simulation by the Skill Trial Runner rather than baked into the skill, which is what lets us vary the context without editing the scene. With these couplings noted, the skill is fully characterized for the agent's purposes as a single parameterized action interface returning one outcome per call.

Workspace and object geometry

The workspace is defined in the robot's `base_link` frame, with the UR30 mounted at the table edge. The table surface sits at $z_{\text{base_link}} = 0\,\text{m}$; in the Isaac Sim world frame this corresponds to $z \approx 0.95\,\text{m}$, so the runner applies a fixed offset $z_{\text{Isaac}} = z_{\text{base_link}} + 0.95\,\text{m}$ for every teleportation (section 5.3.1). Two non-overlapping rectangular zones partition the reachable table area (table 5.4): a *pick zone* on the robot's far side and a *place zone* closer to the base. Both are axis-aligned in xy, and object yaw is sampled independently in $[0°, 180°]$ at each location, exploiting the 180° rotational symmetry of the square cross-section combined with the Robotiq 2F-85's symmetric jaws. The mass, friction, and yaw entries of table 5.4 are the context variables defined in section 4.2.2.

Table 5.4: Pick and place zone bounds (in the `base_link` frame) and context-variable ranges. The zones are the bounding boxes within which the Sobol sampler and the online pipeline place the object.

Parameter	Range	Note
Pick x / y	$[-0.4, 0.05]$ m / $[0.6, 1.1]$ m	Far side of the table, toward the robot's left
Place x / y	$[0.1, 0.4]$ m / $[0.5, 0.8]$ m	Closer to the base, partially overlapping in y
Yaw (pick / place)	$[0°, 180°]$	180° symmetry of the 50 mm cube and Robotiq jaws
Object mass m	$[0.1, 3.0]$ kg	Upper bound near the UR30 wrist payload with gripper
Friction μ	$[0.2, 0.8]$	Analytical Coulomb coefficient, not simulated contact friction

Test object

The object used throughout both phases is a cube of $50 \times 50 \times 150$ mm, standing on its 50 mm square face so that its geometric centre lies at $z = 0.075$ m above the table. Its bounding box ($0.05 \times 0.05 \times 0.15$ m) and mass are registered in the KM at the start of every trial by the runner's KM patch (section 5.3.1). Mass and friction are not intrinsic properties of the object but context variables injected per trial (section 4.2.2), which is what allows the offline sweep to cover the full $[0.1, 3.0]$ kg mass range and the $[0.2, 0.8]$ friction range without a separate scene per combination. A single *top-grasp* frame is defined: the gripper approaches vertically downward along $(0, 0, -1)$ and retreats along $(0, 0, +1)$.

5.3 Skill trial runner

The design chapter specified the trial lifecycle abstractly: requirement A-3 demands an autonomous reset → execute → read cycle in simulation (section 4.4), and the optimization architecture (section 4.5) presupposes a mechanism that can evaluate a single $(\mathbf{x}, \mathbf{z})$ pair and return one consistent observation. This section presents the concrete realization of that mechanism, the Skill Trial Runner: the ROS2 node that encapsulates the full lifecycle of one pick-and-place execution behind a single public method, `run_cycle()`, and serves as the shared evaluation engine for both phases of the pipeline.

5.3.1 Execution life cycle: `run_cycle()`

The public `run_cycle()` method orchestrates the seven sequential phases of fig. 5.4. The ordering ensures that the simulator, the world model, the motion planner, and the data monitors are all in a mutually consistent state before the skill is dispatched, and that the measurement window of each trial is disjoint from its neighbours.

Phase 1: full scene reset

The reset is the most intricate phase. The `_full_scene_reset()` method restores the scene to known initial conditions through the ordered sequence of table 5.5. The ordering is load-bearing

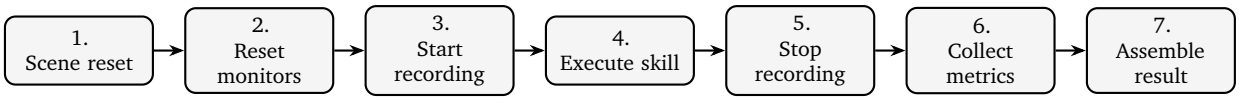


Figure 5.4: The seven phases of `run_cycle()`. The reset, execute, and read groups of section 4.4 map onto phases 1–3, 4, and 5–7 respectively.

rather than incidental: each step depends on the state established by its predecessors, and the most consequential dependency is that any `FixedJoint` from the previous grasp must be released *before* any entity is teleported, since teleporting a still-constrained cube triggers a PhysX constraint violation.

Table 5.5: Ordered steps of the full scene reset, and the dependency that fixes each one's position in the sequence.

#	Action	Why in this position
0	Detach object (publish to `/isaac_sim/detach_object`)	Must precede all teleports; a constrained cube cannot be moved without a PhysX violation
1	Bring arm to ready configuration	The arm must clear the pick zone before the cube is placed
2	Inject mass and friction into Isaac Sim	Physics must be set before execution so torques and IMU reflect the correct mass
2b	Sync slip-detection Coulomb threshold	The node's F_{req} check requires the current m, μ, and grip force
3	Patch KM: cube at pick pose, gripper free	The TaskController plans from KM state, which must be consistent before planning
4	Force planning-scene sync in MoveIt2	Compensates for the KM's asynchronous planning-scene updates
5	Teleport cube to pick pose	Isaac z-offset applied, $z_{\text{Isaac}} = z_{\text{base_link}} + 0.95\,\text{m}$

The arm teleportation of step 1 is itself a multi-step negotiation with the physics engine: pause physics, cancel the active trajectory, command the joints, step the simulation until convergence, sync MoveIt2, resume physics, and re-command the joints. The final re-command after resuming prevents the controller from drifting back toward its pre-teleport trajectory state.

Phases 2–3: reset monitors and start recording

The energy monitor and the slip-detection node are zeroed through their respective service interfaces so that the upcoming window carries no residue from the previous trial. The joint-effort buffer is then cleared and recording is enabled. From this point until phase 5, every `/isaac_joint_states` callback appends the effort vector to an internal buffer for later peak-torque extraction.

Phase 4: skill execution

The `_execute_pick_and_place()` method serializes the decision variables into a `pick_and_place` action plan and dispatches it to the TaskController as an `ExecuteSkill` action goal. Table 5.6 shows the dispatched structure.

Table 5.6: Pick-and-place action plan dispatched to the TaskController. The four execution parameters the optimizer sets are `speed`, `acceleration`, `force_override_N`, and `approach_distance`.

Field	Value
`type`	`"action"`
`name`	`"pick_and_place"`
input_params	
`object`	`"cube"`
`resource`	`"handling_robot"`
`pose`	`{"x": 0.25, "y": 0.65, "z": 0.075, ...}`
`speed`	`0.21`
`acceleration`	`0.87`
`force_override_N`	`150.0`
`approach_distance`	`0.08`

Phases 5–7: stop recording, collect metrics, assemble result

Effort recording is disabled and a 150 ms settle delay allows the final callbacks to propagate. Metrics are then gathered from the two independent nodes by a service call: the energy monitor (`get_task_metrics`) returns cycle time and total energy, and the slip-detection node (`get_slip_summary`) returns peak acceleration. Finally, the per-joint maximum absolute efforts are extracted from the buffer and all values are assembled into the `SkillTrialResult` returned to the caller.

5.3.2 Data contract: `SkillTrialResult`

The runner returns one `SkillTrialResult` record per execution, the typed contract that both callers consume. Table 5.7 lists its principal fields, their units, and the node that produces each. One design point is embedded in the structure: the runner reports the raw peak TCP acceleration a_{peak} and does *not* compute the required grip force itself. The Coulomb transform to F_{req} (section 4.4) is applied downstream in the optimization layer. The runner reports observables, not constraint values, which keeps it independent of the constraint formulation.

Table 5.7: Principal fields of the `SkillTrialResult` record returned by `run_cycle()`.

Field	Unit	Source
`success` / `error`	—	TaskController goal status
`cycle_time`	s	Runner (wall-clock around skill)
`total_energy`	J	Energy monitor (integrated joint power)
`avg_power`	W	Derived, E/T
`peak_accel_ms2`	$\mathrm{m\,s^{-2}}$	Slip-detection node (peak TCP accel.)

5.3.3 Integration with ROS2

The Skill Trial Runner coordinates eight external nodes through the three ROS2 communication patterns, summarized in table 5.8. *Services* carry synchronous control and query traffic: simulation-

state control, entity teleportation, planning-scene sync, KM patches, and the post-execution metric summaries. *Actions* carry the two long-running operations with feedback: stepping physics during arm teleportation, and dispatching the skill itself. *Topics* carry the streaming traffic: joint-state and effort buffering, the cube IMU for grasp verification, slip status, and direct joint commands during reset.

Table 5.8: External interfaces of the Skill Trial Runner, grouped by subsystem. Direction is given relative to the runner.

Interface	**Type**	**Dir.**	**Purpose**
Isaac Sim			
`/set_simulation_state`	srv	out	Pause / resume / playback control
`/set_entity_state`	srv	out	Teleport cube and arm entities
`/simulate_steps`	action	out	Step physics during arm teleport
`/isaac_sim/set_physics`	topic	out	Inject mass and friction
`/isaac_sim/detach_object`	topic	out	Release the grasp `FixedJoint`
`/isaac_joint_states`	topic	in	Joint-state and effort buffering
`/cube_imu`	topic	in	Grasp verification
Motion control			
`/joint_trajectory_controller`	topic	out/in	Direct joint commands and state
`MoveIt2 planning scene`	srv	out	Planning-scene sync
Skill stack			
`TaskController (ExecuteSkill)`	action	out/in	Skill dispatch and status
`/km/modify_state`	srv	out	KM state patch
Data nodes			
`Energy monitor`	srv	out	Energy and cycle time
`Slip detection`	srv/sub	out/in	Peak acceleration

The shared role for both online and offline optimization

That the offline sweep and the online loop both reach the simulator only through `run_cycle()` (the same reset, the same dispatch, the same metric collection) is the architectural property that makes the two-phase decomposition of fig. 4.3 sound. It guarantees that the offline observations the k-KNN retriever interpolates from are measured identically to the online observations the GPs are fitted to, so that warm-start and revision speak a common measurement language and no methodology gap separates the phase that builds the memory from the phase that consumes it.

5.4 Monitoring: data nodes

The design chapter established *what* the monitoring subsystem must deliver—the energy objective, the cycle-time constraint, the grip-force constraint derived from peak acceleration, and the trial-success flag—together with the principle that monitoring stay strictly observational (section 4.4 and table 4.3). The Skill Trial Runner section then specified *when* these quantities are harvested: the runner resets the monitors before dispatch, lets them accumulate during execution, and queries them after a settling delay (section 5.3). This section closes the remaining gap and documents *how* the two observer nodes compute their metric internally—the topics they consume, the equations used, and the services through which the runner reads them back.

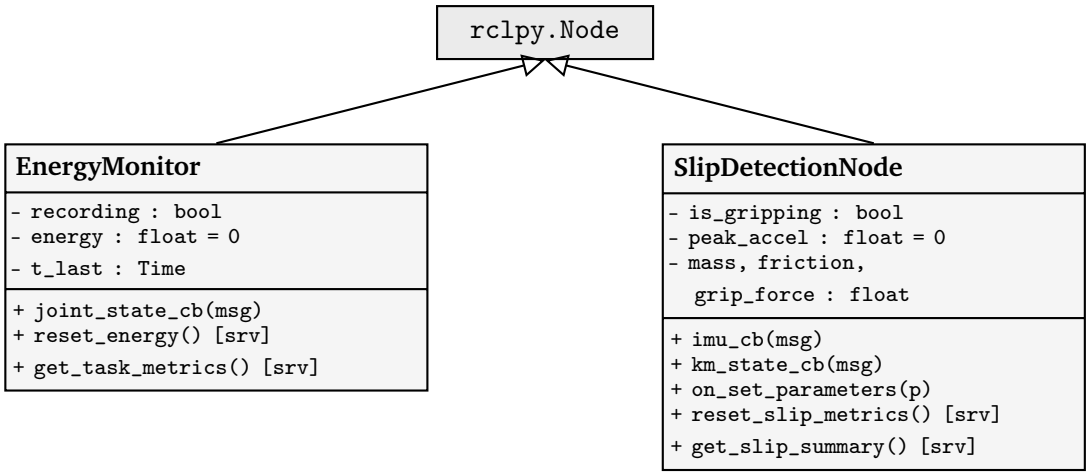


Figure 5.5: Class structure of the two monitoring nodes. Both extend `rclpy.Node`. Attributes hold the running accumulators; the telemetry callbacks (`*_cb`) maintain them, and the service handlers marked `[srv]` are the runner's reset and read entry points.

Both nodes are launched together by `data_nodes.launch.py` and run as independent, passively scheduled subscribers alongside the skill stack. Keeping them out of the optimization process is the architectural decision argued in section 5.1: each owns a single telemetry domain, exposes a query interface, and never publishes a command or writes back into the controlled process. The runner is their only client.

Both are implemented as `rclpy.Node` subclasses with the structure of fig. 5.5: a small set of accumulator attributes, one or two telemetry callbacks that maintain them, and a pair of service handlers through which the runner resets and reads the metric. The remainder of this section describes each class by that structure.

5.4.1 Energy monitor

The energy monitor subscribes to `/isaac_joint_states` (`sensor_msgs/JointState`), which Isaac Sim publishes at the physics-step rate and which carries the per-joint effort τ_i and angular velocity $\dot{q}_i$ of all six UR30 axes. The subscription callback, `joint_state_cb`, advances a single accumulator on every message: it derives the inter-message interval Δt from the ROS2 clock, forms the instantaneous mechanical power as the sum of absolute per-joint products (eq. (4.8)), and adds the increment $P\,\Delta t$ to the running energy total (eq. (4.9)). The accumulation is guarded by a `recording` flag, so the integration window covers exactly the interval between the runner's reset and read calls and matches the trial.

The node exposes the two services of table 5.9. The `reset_energy` handler zeros the accumulator and opens a fresh window. `get_task_metrics` returns the accumulated energy together with a cycle-time field and then disarms recording. That cycle-time field is the ROS2-clock duration of the measurement window and is retained only as a diagnostic cross-check. The constraint the optimizer enforces is the runner's wall-clock measurement, for the reasons given in section 4.4. The returned `total_energy` is the objective value E stored in the trial result (table 5.7).

Table 5.9: Service interface of the energy monitor.

Service	Type	**Function**
`reset_energy`	`std_srvs/Empty`	Zero the accumulator, open a new window
`get_task_metrics`	`isaac_ros2_messages/ GetTaskMetrics`	Return `total_energy` [J] and the diagnostic `cycle_time` [s], then stop recording

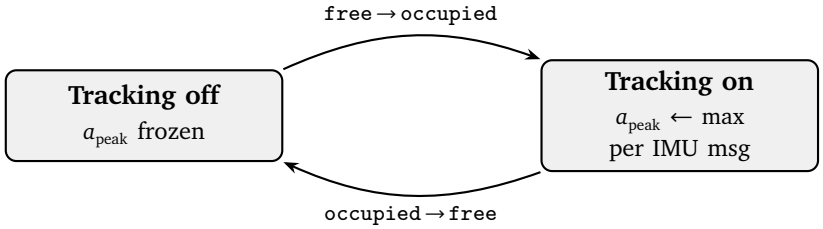


Figure 5.6: The slip-detection node's peak-tracking gate. Occupancy transitions read from the KM state topic switch the running maximum on and off, so that a_{peak} reflects only the interval during which the object is held.

5.4.2 Slip-detection node

The slip-detection node derives the peak TCP acceleration a_{peak} that the grip-force constraint depends on. It subscribes to `/cube_imu` (`sensor_msgs/Imu`), the inertial sensor mounted on the manipulated cube, published at the Isaac Sim physics rate. Its callback, `imu_cb`, removes the gravity bias from the z-axis, forms the corrected Euclidean magnitude (eq. (4.6)), and updates the running maximum `peak_accel`—but only while the `is_gripping` flag is set. Gating the maximum to the gripping window is what keeps post-release transients out of a_{peak}; the justification for this is given in section 4.4 and is not repeated here.

Grip-phase gating

The gripping window is opened and closed by observing the KM, not by any coupling into it. The node subscribes to the KM state topic `/km/state`, which publishes the full world-model state as JSON roughly every 100 ms. Its callback, `km_state_cb`, parses the gripper's occupancy field `resources.robotiq_gripper.status.occupancy` and toggles `is_gripping` on its transitions: a `free` → `occupied` change opens the window and arms peak tracking, an `occupied` → `free` change closes it. The node has no code dependency on the KM and never writes to it, so the gating preserves the observational separation. Figure 5.6 shows the resulting two-state behaviour.

Per-trial parameter synchronization

Because the context sweep vary object mass and friction between trials, the runner synchronizes the node's physics parameters before each execution through the standard ROS2 parameter service `/slip_detection/set_parameters`; this is step 2b of the reset sequence (table 5.5). The three synchronised parameters are listed in table 5.10. On change, the registered `on_set_parameters` callback recomputes an internal Coulomb threshold from the current mass, friction, and commanded grip force and classifies each trial as slipping or holding in real time. This boolean verdict does not enter the *online* constraint evaluation: the BO loop consumes the raw a_{peak} alone and applies the

Coulomb transform to F_{req} downstream (section 4.4, section 5.3.2). Offline, however, it is recorded per trial and serves as one conjunct of the feasibility label (eq. (5.1)) that admits a row into the experience memory.

Table 5.10: Physics parameters synchronised into the slip-detection node before each trial via `/slip_detection/set_parameters`.

Parameter	Source	**Effect**
`object_mass_kg`	`context` m	Mass in the diagnostic Coulomb threshold
`static_friction_cube`	`context` μ	Friction coefficient in the threshold
`grip_force_N`	`fixed policy`	Commanded clamping force

The node exposes the two services of table 5.11. `reset_slip_metrics` zeros the peak accumulator and the cycle counters; `get_slip_summary` returns a JSON payload whose `peak_accel` field the runner stores as the trial result's `peak_accel_ms2` (table 5.7). The runner reports this raw observable and leaves the transform to F_{req} to the optimization layer, which is the contract documented in section 5.3.2.

Table 5.11: Service interface of the slip-detection node.

Service	Type	**Function**
`reset_slip_metrics`	`std_srvs/Empty`	Zero the peak accumulator and cycle counters
`get_slip_summary`	`std_srvs/Trigger`	Return JSON with `peak_accel` [$\mathrm{m\,s^{-2}}$] and the diagnostic `slip_events`, `is_gripping`, `is_slipping` flags

5.4.3 Summary

The two nodes together realise the passive measurement layer of table 4.3: the energy monitor integrates joint power into E from `/isaac_joint_states`, and the slip-detection node tracks the gravity-corrected, grip-phase-gated peak acceleration a_{peak} from `/cube_imu`. Each owns one telemetry domain, computes its scalar through a numerical operation with no learned component, and returns it on demand through a service. The runner's wall-clock cycle time and the binary trial outcome (section 5.3) complete the four-element observation that one `run_cycle()` call hands to the optimizer.

5.5 Offline context sweep

The design chapter established the offline phase abstractly: a single fixed policy is evaluated across the context space to build the experience memory $\mathcal{M}$ that the online pipeline retrieves from (section 4.5.1). This section documents how that sweep is realized. The procedure is implemented in `collect_context_sweep.py`, which samples the eight-dimensional context space, evaluates each sample, and writes one row per trial to a CSV file that constitutes $\mathcal{M}$.

5.5.1 Sobol sampling

The context samples are drawn from a scrambled Sobol sequence (section 2.1) using SciPy's Sobol sequence generator,, configured with dimension $d = 8$, Owen scrambling enabled, and a fixed seed. Three implementation details govern how the raw sequence is turned into the 4000 sampled contexts.

First, the engine is generated in a power-of-two block and then truncated. The Sobol generator selects 2^m points, so for a target of $N = 4000$ samples the implementation takes $m = \lceil \log_2 N \rceil = 12$, draws the $2^{12} = 4096$-point block, and retains the first 4000.

Second, Owen scrambling (`scramble=True` flag) randomizes the sequence while preserving its equidistribution, and the fixed seed makes the draw deterministic: the identical sequence is regenerated on every run, which is what allows the sweep to be resumed even after an interruption.

Third, the raw samples $u_j \in [0, 1]$ are scaled onto the physical bounds of each context dimension by the range listed in table 5.4. Yaw angles are then rounded to the nearest integer degree, to match the discrete orientation representation the KM stores.

5.5.2 Context sweep execution

Each sampled context is evaluated once at the fixed reference policy $\mathbf{x}_0 = (0.3, 0.3)$ (section 4.5.1). The per-trial control flow of fig. 5.7 screens each context before any simulation is committed and then dispatches the valid ones to the Skill Trial Runner.

Physics prior screen

Before a trial is launched, the context is passed to `check_static_feasibility()` in `physics_priors.py`. This is the zero-cost static-hold floor of eq. (4.12), evaluated at rest ($a_{\text{peak}} = 0$). The minimum clamping force needed to hold the object against gravity is compared with the commanded 150 N. A context that fails this floor can never be made feasible by any motion, so it is logged with `executed = False` and `failure_category = PHYSICS_INFEASIBLE` and the simulation is skipped entirely. The screen removes the high-mass, low-friction corner of the context space and spends the expensive simulation budget only where a static grasp is physically admissible.

Simulation trial and feasibility label

A context that clears the screen is dispatched to `run_cycle()`, which returns the observation tuple (E, T, a_{peak}, success) measured identically to the online phase (section 5.3). Each completed trial is assigned a binary feasibility label that conjoins the three conditions the offline data must satisfy to be admissible,

$$\text{feasible} = \text{success} \wedge \neg\,\text{slip} \wedge \big(T < 60\,\text{s}\big), \tag{5.1}$$

combining the runner's success flag, the absence of a detected slip event, and a cycle-time ceiling. The data established that most runs took 15 s to 45 s to complete, while runs exceeding 60 s were typically caused by planner failures or simulation crashes. The resulting row is appended to the CSV and the file handle is flushed immediately, so that the dataset is durable even if the multi-hour sweep is interrupted midway through.

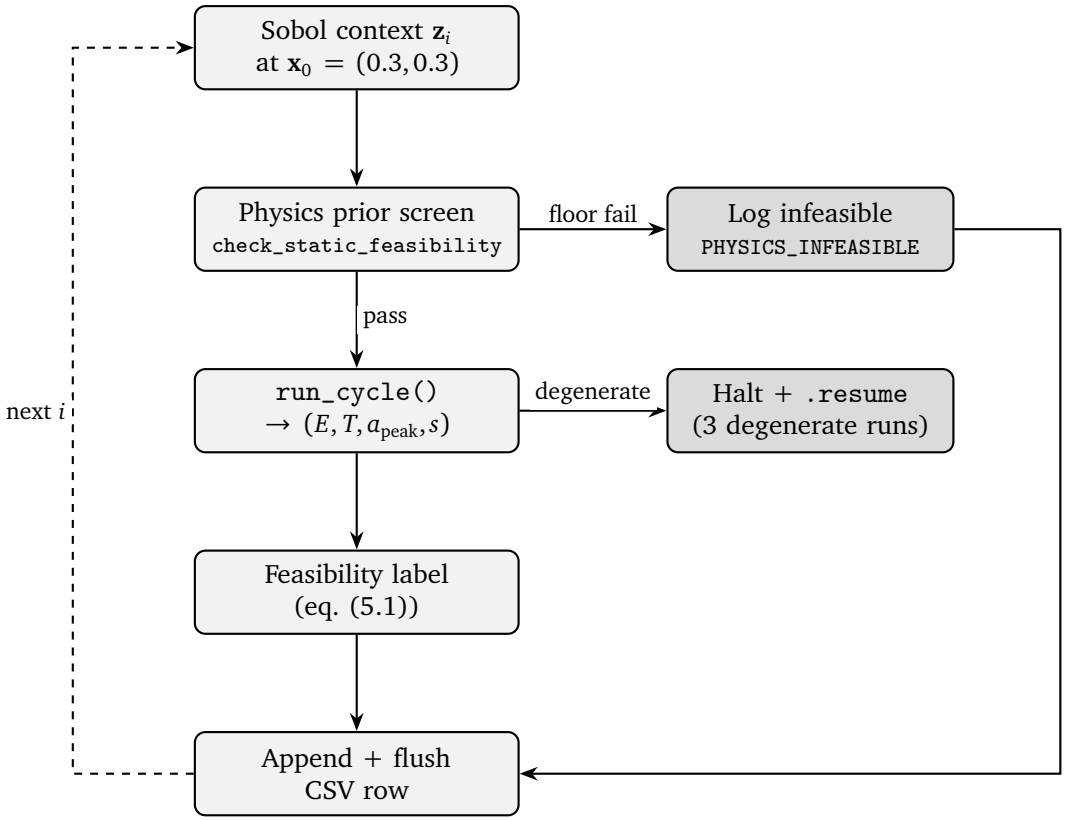


Figure 5.7: Per-trial control flow of the offline context sweep. Each Sobol context is screened by the static-hold floor before any simulation is run; survivors are evaluated through `run_cycle()`, labelled by eq. (5.1), and written to the experience memory. Physics-infeasible contexts are logged without simulation, and three consecutive degenerate trials halt the sweep for resumption.

Degenerate-trial and simulation stall detection

Since the simulation stalling was a common occurrence, two methods were designed to prevent the stall from corrupting the dataset. A *fast-fail* degenerate is a `SKILL_ABORTED` outcome where the planner rejects the motion outright. Second, a *frozen-sim* degenerate is an execution timeout (a 120 s hard limit) validated by a near-zero peak acceleration ($< 0.01\,\mathrm{m\,s^{-2}}$) and near-zero energy ($< 50\,\mathrm{J}$), the clear signs of an unresponsive Isaac Sim. Three consecutive degenerate trials halt the sweep and write a `.resume` record holding the current trial index. On restart, the implementation reads the companion `.meta` record to recover the original N, seed, trims the degenerate tail from the CSV, and continues from the last valid index, so that no completed trial is repeated.

5.5.3 Recording the data schema in CSV

The CSV records every sampled context together with its physics-prior verdict and, for executed trials, the measured outcomes. Table 5.12 lists only the fields read by downstream pipeline components: the columns the context feasibility gate uses to filter and to compute distances, and the three outcome columns the warm-start aggregates. The remaining recorded fields (timestamps, the diagnostic derived scalars, and the per-joint effort maxima) are retained for analysis but are not consumed online.

At load time the context feasibility gate reduces the raw sweep to a *clean* dataset by intersecting

four masks,

$$\texttt{success} \wedge (\texttt{failure_category} = \texttt{OK}) \wedge (\texttt{avg_power_W} \geq 3.0\,\text{W}) \wedge \texttt{feasible},$$

so that only successful, genuinely feasible trials enter the memory. The `avg_power_W` floor is the one condition not already implied by the feasibility label since it serves the purpose of discarding trials that reported success but performed negligible mechanical work, which indicates a physics glitch rather than a genuine handling cycle.

Table 5.12: Sweep CSV fields consumed by downstream pipeline components, grouped by consumer.

Column	Type	Role downstream
Context feasibility gate		
`pick_x, pick_y`	float	Spatial k-KNN distance
`place_x, place_y`	float	Spatial k-KNN distance
`pick_yaw_deg, place_yaw_deg`	int	Angular distance (wrap-around)
`mass_kg`	float	Material pre-filter
`friction`	float	Material pre-filter
`success`	bool	Clean-dataset mask
`feasible`	bool	Clean-dataset mask
`failure_category`	str	Clean-dataset mask (= `OK`)
`avg_power_W`	float	Clean-dataset mask ($\geq 3.0\,\text{W}$)
k-KNN warm-start		
`total_energy_J`	float	Warm-start mean/std for energy E
`cycle_time_wall_s`	float	Warm-start mean/std for cycle time T
`peak_accel_ms2`	float	Warm-start mean/std for a_{peak}

5.6 Feasibility gates

A user query entering the online pipeline passes two deterministic gate layers before any optimization begins. They realize the two screens specified in the design: the physics-and-input gate of section 4.5.2 and the material screen of the warm-start gate of section 4.5.2.

5.6.1 Physics and input gate

The first gate is implemented in `feasibility_gate.py` and applies two checks in sequence.

The *workspace check* verifies that every one of the eight context dimensions lies inside the bounds declared in `config.py`. A dimension whose value falls outside this bound is rejected as out of workspace, and a missing dimension key is likewise rejected, which catches malformed queries from the upstream system before they reach the optimizer.

The *static-Coulomb check* delegates to the same `check_static_feasibility()` used by the offline screen, but with the worker's grip-force limit $F_{\max}$ as the constraint. It evaluates the static-hold floor of eq. (4.12) with safety factor $S = 2.0$ and $n = 2$ contact faces. If the required static force exceeds $F_{\max}$, the query is rejected as physics-infeasible.

5.6.2 Context feasibility gate

The second gate is the `ContextFeasibilityGate` class in `context_feasibility_gate.py`. It performs the data-proximity lookup against the cleaned offline data set and produces the warm-start estimates the BO loop consumes (section 5.7). On construction, it loads the sweep CSV, applies the clean-dataset mask of section 5.5.3, and pre-extracts the spatial, yaw, mass, friction, and outcome columns into `NumPy` arrays.

The single hard rejection at this stage is the *material pre-filter*. The query's mass and friction are matched against the memory within a fixed tolerance envelope, $|m_i - m_q| \leq 0.3\,\text{kg}$ and $|\mu_i - \mu_q| \leq 0.4$, the tolerances being configured in `config.py`. If no stored context lies within this envelope, the query is rejected, since outcomes measured on materially dissimilar objects would be uninformative as a warm-start. Every context that clears the material filter is admitted: spatial distance is treated as informational, never as a rejection condition.

Spatial proximity is nonetheless reported. The nearest-neighbor distance to the admitted subset is compared against an auto-calibrated threshold $d_{\max}$ to raise a `far_from_data` flag, which marks the warm-start as less reliable without barring the query from optimization. Rather than tuning $d_{\max}$ by hand, the gate computes it once at construction, in `_calibrate_d_max()`, as the 95th percentile of the in-sample one-nearest-neighbor distances over the clean rows, using the same scaled metric as the query-time lookup (section 5.7.1). This sets the flag so that only the most isolated few percent of the training distribution register as far from data.

5.7 Warm start

The warm-start is produced inside the context feasibility gate's `query()` method, immediately after the material filter admits a query. It realizes the k-KNN warm-start of section 4.5.2: the gate locates the nearest stored contexts and aggregates their recorded outcomes into the mean and noise estimates of eq. (4.13), which it hands to the BO loop. Figure 5.8 traces the path from the clean memory to the returned record.

5.7.1 k-Nearest-neighbor lookup

Over the material-filtered subset, the gate selects the $k = 20$ nearest stored contexts under a scaled six-dimensional metric. Only the four spatial and two yaw dimensions enter the distance; mass and friction are deliberately excluded, since they are already handled by the material filter and govern the grasp force rather than the kinematic path. The distance between the query $\mathbf{z}_q$ and a stored context $\mathbf{z}_i$ is

$$d(\mathbf{z}_q, \mathbf{z}_i) = \sqrt{\sum_{j \in \mathcal{S}} \left(\frac{z_{q,j} - z_{i,j}}{r_j} \right)^2 + \sum_{j \in \mathcal{Y}} \left(\frac{\delta_j}{90} \right)^2}, \qquad \delta_j = \min\big(|\Delta\theta_j|,\ 180 - |\Delta\theta_j|\big), \tag{5.2}$$

where $\mathcal{S}$ are the four spatial dimensions, $\mathcal{Y}$ the two yaw dimensions, $\Delta\theta_j = z_{q,j} - z_{i,j}$, and $r_j = \max_i z_{i,j} - \min_i z_{i,j}$ is the observed range of dimension j. Normalizing each spatial term by its range puts the four positional axes on a common unit scale, so no single axis dominates the distance. The yaw terms use the wrap-around angular distance δ_j, which recognizes 179° and 1° as 2° apart, consistent with the 180° grasp symmetry exploited throughout (table 5.4). Dividing by 90° makes

a 90°-turn contribute the same as a full-range positional displacement. All pairwise distances are computed in a single vectorized `NumPy` pass, and the k smallest are taken by `argsort`. The count is clamped to $k_{\text{actual}} = \min(k, n_{\text{subset}})$, so that a query with fewer than 20 contexts inside its material envelope still returns a warm-start from whatever neighbors exist.

5.7.2 Warm-start computation

The k_{actual} neighbours yield two products, both formed column-wise over the three outcomes $y \in \{E, T, a_{\text{peak}}\}$. The *warm-start mean* $\hat{y}$ is their arithmetic mean (eq. (4.13)). Since the entire memory was collected at the single reference policy $\mathbf{x}_0$, this mean is a direct interpolation of observed outcomes at that operating point, not an extrapolation across the decision variables. The *noise estimate* $\hat{\sigma}_y$ is the sample standard deviation of the same neighbors, and quantifies the context-conditioned spread of each outcome in this region of the memory.

Both products are packaged, with the proximity diagnostics, into the `GateResult` record of table 5.13, which the gate returns to the pipeline. The pipeline forwards the `warmstart` and `noise_estimate` dictionaries to the online optimizer's entry point (`online_bo.run`). How the seed phase of the BO loop consumes them is described in section 5.8.

Table 5.13: Principal fields of the `GateResult` record returned by the context feasibility gate.

Field	**Type**	**Meaning**
`accepted`	bool	Material filter verdict
`reason`	str	Reject reason when not accepted
`warmstart`	dict	Per-outcome means $\hat{y}$
`noise_estimate`	dict	Per-outcome std $\hat{\sigma}_y$
`neighbor_indices`	list[int]	Rows of the k selected neighbours
`nearest_distance`	float	Distance to the closest neighbour
`k_actual`	int	Neighbours actually used
`far_from_data`	bool	Set when nearest distance $> d_{\max}$

5.8 Constrained Bayesian optimization loop

The design chapter specified the *Revise* step abstractly: a contextual constrained BO loop that searches the two-dimensional decision space for the per-context policy $\mathbf{x}^* = (s_v, s_a)$. (section 4.5.2). This section documents how that loop is realized. The procedure is implemented in `online_bo.py`. It iterates over a fixed trial budget to return the best constrained feasible policy. Figure 5.9 traces the control flow from the warm-start through the two trial phases to policy extraction.

The nominal constrained problem laid out in section 4.3, and the noisy expected-improvement acquisition that handles those constraints is derived in section 4.3.2. What follows is the implementation that puts them to work.

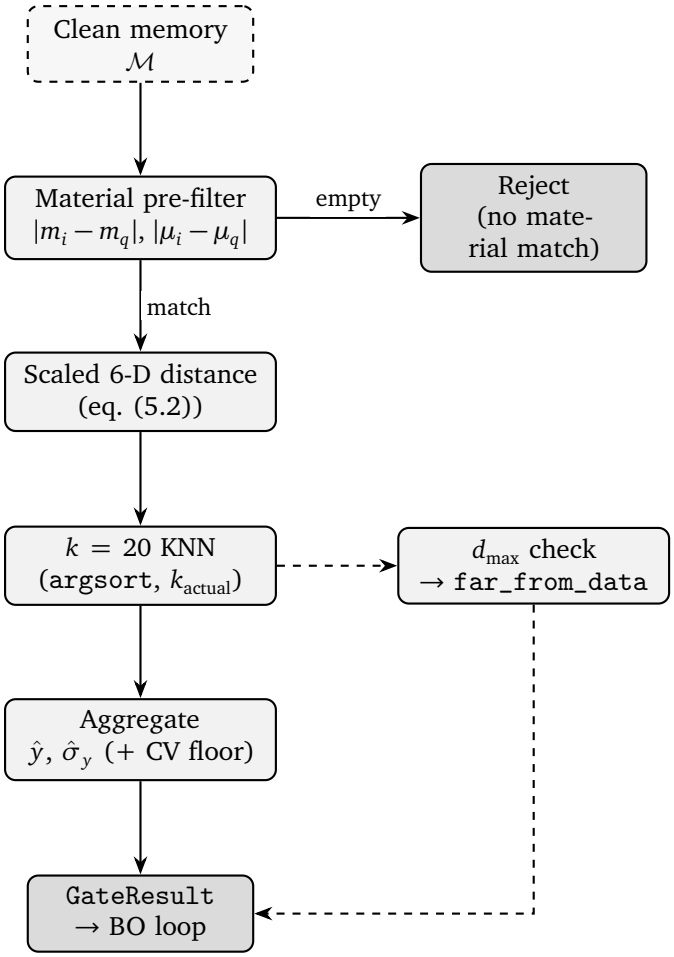


Figure 5.8: Data flow inside the context feasibility gate. The material pre-filter is the only hard rejection; admitted queries proceed through the scaled distance and k-KNN selection to the outcome aggregation, and the nearest-neighbour distance sets the diagnostic `far_from_data` flag. The assembled `GateResult` carries the warm-start means and noise estimates to the BO loop.

5.8.1 Ax/BoTorch integration

The loop is built on the Ax platform's service API (`AxClient`), which wraps the BoTorch GP infrastructure (Balandat et al., 2020). The experiment is constructed once per query by `_setup_ax_experiment()`, which registers the two decision parameters, the single objective, and three outcome constraints, the four registered metrics of table 5.14.

The decision space is the two continuous scalings of section 4.2.3, each registered as a range parameter over $[0.05, 1.0]$. The lower bound of 0.05, rather than 0, excludes degenerate near-zero motion in which the robot would barely move, which would spend a trial on an uninformative observation. The objective registers energy E with `minimize=True`. The three constraints are registered in Ax's outcome-constraint form, $T \leq T_{\max}$, $F_{\text{req}} \leq F_{\max}/S$, and a success constraint `trial_succeeded` ≥ 0.5 on the binary outcome.

The safety factor $S = 2.0$ enters only at the force-constraint bound, $F_{\max}/S$, and never inside the quantity the surrogate models: the GP target is the raw Coulomb force F_{req} of eq. (4.7). This is the separation already argued in the design (section 4.4). In the implementation it has the concrete benefit that S can be retuned without refitting any surrogate, since it reparametrizes only the constraint threshold.

Given this, Ax's default generation strategy fits one independent GP surrogate per metric, one each for E, T, F_{req}, and `trial_succeeded`, and constructs the feasibility-weighted noisy expected-improvement acquisition of section 4.3.2 over them. Separate surrogate models are used for each metric rather than a single correlated multi-output model. This is sufficient because each constraint is evaluated independently using its own GP posterior. The Monte Carlo acquisition function then combines the resulting feasibility estimates with the predicted improvement. Therefore, there is no need to explicitly learn correlations between metrics. The `trial_succeeded` metric is a binary indicator (0.0 or 1.0) carried by a regression GP; the constraint `trial_succeeded` ≥ 0.5 turns its posterior into a feasibility weight that down-weights regions of $\mathcal{X}$ where simulated executions are likely to fail.

Table 5.14: The four metrics registered with the `AxClient`: one objective and three outcome constraints, each backed by an independent GP surrogate.

Metric	**Role**	**Source of the reported value**
`E`	Objective	Energy monitor, integrated joint power (table 5.7)
`T`	Constraint	Runner wall-clock cycle time, $T \leq T_{\max}$
`F_req`	Constraint	Coulomb transform of a_{peak} (eq. (4.7)), $F_{\text{req}} \leq F_{\max}/S$
`trial_succeeded`	Constraint	Binary success flag from the runner, ≥ 0.5

5.8.2 Seed phase

The seed phase populates the surrogates with initial observations before model-guided exploration begins. The total budget is partitioned as the $6 + 24 = 30$ trials of table 4.4: six seed trials followed by up to twenty-four GP-guided trials. The six seed trials comprise one anchor and five Sobol points.

The synthetic anchor

The first seed trial is the synthetic anchor, and it is the point at which the offline memory enters the online loop. Rather than executing a simulation, the loop attaches a manually specified observation at the reference policy $\mathbf{x}_0 = (0.3, 0.3)$ through `attach_trial()`, which bypasses Ax's generation strategy, and immediately reports its outcomes through `complete_trial()`. The reported outcomes are exactly the k-KNN warm-start estimates the gate computed at the queried context (section 5.7.2): the warm-start means $\hat{E}$, $\hat{T}$, and $\hat{a}_{\text{peak}}$ become the metric values, the latter transformed to F_{req} through the same Coulomb model of eq. (4.7), and the per-outcome noise estimates $\hat{\sigma}_y$ are reported as the standard error of the mean so that the surrogate assigns the anchor a calibrated, finite uncertainty rather than treating it as noise-free. The success metric is reported as 1.0.

Since the entire dataset was collected at the same reference policy $\mathbf{x}_0$, the synthetic observation sits at exactly the operating point it represents, so the GP begins from a data-grounded estimate of the outcomes at $\mathbf{x}_0$ without spending a simulation on a point whose behaviour the memory already characterizes. The five Sobol trials then supply the variation across $\mathcal{X}$ that the offline datset cannot.

Sobol seed trials and trial execution

The remaining five seed trials are drawn from Ax's built-in Sobol generator through `get_next_trial()` and are each physically executed. Together with the anchored point at $\mathbf{x}_0$, they give the surrogates a space-filling initial coverage of $[0.05, 1.0]^2$ before acquisition takes over.

Every physically executed trial, the five Sobol seeds and all subsequent GP-guided trials alike, follows one path through `run_single_trial()`. On a successful return, it reads the observation tuple $(E, T, a_{\text{peak}}, \text{success})$ from the trial result (table 5.7), computes F_{req} from the observed a_{peak} through eq. (4.7), and reports the four metrics through `complete_trial()` with their measured values; a successful trial is reported with no added noise. A failed return is handed instead to the failure penalty scheme of section 5.8.3. The optimizer never addresses the simulator directly: it produces a decision vector, hands it to the runner, and receives the observation back, exactly the single contact surface established in section 5.1.

5.8.3 Failure penalty scheme

When a trial fails, no valid (E, T, F_{req}) observation exists. The implementation neither drops the trial nor fabricates a plausible value for it, instead `_get_penalty_values()` reports the failure in a way that gives each surrogate the signal it needs. Two reasons motivate reporting a failed trial rather than discarding it.

The first is mechanical: Ax's state machine expects a matching `complete_trial()` call, so a trial cannot be silently abandoned. The second, and the real design reason, is that the success-constraint surrogate can only learn where failures occur if it sees those data points too. Were failures discarded, the `trial_succeeded` GP would never observe a 0.0, the acquisition would carry no evidence that a region of (s_v, s_a) is failure-prone, and it would keep proposing points there.

The scheme reports each metric so that the right GP is steered and the others are not. The `trial_succeeded` metric receives a precise 0.0. This clean value lets its surrogate learn a failure boundary, so the acquisition function down-weights the region. The objective and the two metric constraints E, T, and F_{req} are handled differently: each receives a deliberately uninformative observation built from two mechanisms. First, its penalty mean is the running maximum of that metric over all successful trials so far, inflated by a fixed factor of 1.5. The penalty therefore lies beyond anything yet observed, in the undesirable region of the metric's posterior. Second, its reported standard error is set equal to that mean. This gives a coefficient of variation of roughly 100 %, so the surrogate reads the point as "no information here" rather than as a precise high value. As a result, the energy and constraint landscapes are not distorted by a fabricated observation. All directional work, steering sampling away from failures, is done by the clean 0.0 on the success constraint. The inflated, high-variance penalties on the other three metrics simply satisfy Ax's interface without corrupting their surrogates.

5.8.4 Early stopping

The GP-guided phase can terminate before exhausting its twenty-four-trial budget once the best feasible energy has converged. Convergence is measured from the first feasible observation, not from the start of the phase. This is the critical choice: a run may spend its opening iterations exploring infeasible regions before it finds any constraint-satisfying policy, and a window anchored at the start

of the phase would risk stopping during that exploration, before the optimizer has had a chance to improve on a feasible point.

The loop maintains a running best policy history and the index of the first feasible trial. The stopping test fires only when three predicates hold jointly: at least fifteen trials have elapsed since the first feasible observation, the history holds more than five entries, and the relative improvement of the best feasible energy across the last five entries falls below 1 %,

$$\frac{E_{\text{best}}^{(t-5)} - E_{\text{best}}^{(t)}}{E_{\text{best}}^{(t-5)}} < 0.01.$$

The fifteen-iteration floor prevents a premature stop, and the five-entry window makes the test insensitive to single-trial noise. If no feasible point is ever found, the test never activates and the loop runs to its full budget. The three parameters, the window length, the relative tolerance, and the post-feasibility minimum, are fixed in `config.py` and listed in table 5.15.

Table 5.15: Early-stopping parameters of the GP-guided phase, defined in `config.py`.

Parameter	Value	**Role**
`BO_EARLY_STOP_WINDOW`	5	Look-back window for relative improvement
`BO_EARLY_STOP_REL_TOL`	0.01	Minimum relative improvement threshold
`BO_EARLY_STOP_MIN_ITERS`	15	Feasible iterations required before the test activates

5.8.5 Policy extraction

When the loop terminates, by convergence or by budget exhaustion, the recommended policy is extracted by `_extract_best_feasible()`. The function scans the full trial history and keeps every trial that succeeded and that observed a cycle time within $T_{\max}$ and a required force within $F_{\max}/S$; among those it returns the one with the lowest observed energy.

The extraction is a post-hoc filter over the raw observations, not a query of any GP posterior. It uses the measured constraint values rather than the surrogates' predicted feasibility probabilities, so the recommended policy is one that satisfied every constraint in at least one real trial. The surrogates and their acquisition steer the search toward promising regions; the final recommendation is nonetheless grounded in an observed feasible execution rather than in a model prediction.

If no trial proved feasible, the function returns nothing and the loop reports honest infeasibility, the outcome already required by the design (section 4.5.2). The pipeline sets its status to `no_feasible_policy` and logs diagnostics, the fastest cycle time and the lowest energy achieved among the infeasible trials, together with a suggestion to relax the binding limit, so that the upstream system can act on the failure rather than receive an unsafe recommendation. On success the recommended policy and its achieved (E, T, F_{req}) are serialized to a results file alongside the full trial history and the best-energy convergence trace, and are handed to the *Retain* step of section 5.9.1.

5.9 Case-based reasoning: retain and retrieve

The case-based reasoning layer provides instant policy recall for contexts that have already been solved. When the pipeline has previously optimized a given context, the stored solution is returned immediately, bypassing the warm-start gate, the seed phase, and the entire BO loop, at zero simulation cost. This is the *Retrieve* and *Reuse* of the online cycle, served from the products of past *Retain* steps (section 4.5.2). *Retain* is implemented in `case_base.py`, and fig. 5.10 shows its two classes.

5.9.1 Case-based storage

A solved context is represented by the `SolvedCase` dataclass, which holds the full eight-dimensional context that was optimized, the recommended two-dimensional policy (s_v, s_a), and the three achieved metric values (E, T, F_{req}), together with a creation timestamp and the originating run's output directory for provenance. The eight context keys are the case's identity: two contexts are the same case if and only if those keys agree.

The `CaseBase` class manages a flat list of `SolvedCase` instances, persisted as a single JSON file (`solved_cases.json`) beside the optimization module. The store is read into memory once on construction and flushed to disk after every `add()`, so the case base survives across pipeline invocations. No locking is provided, the pipeline is single-threaded, so a case base is only ever read or written by one process at a time.

The *Retain* trigger lives in the pipeline rather than in the class: when the BO loop returns a feasible recommendation, the pipeline constructs a `SolvedCase` from the query context and the recommended policy and calls `add()`. Retention fires only when the run's status is success; a run that found no feasible policy is never stored, so the case base contains exclusively verified solutions.

5.9.2 Lookup logic

Lookup runs early in the online cycle, after the physics and input gate (section 5.6.1) but before the context feasibility gate and the BO loop, exactly the *Retrieve / Reuse* position of the design pipeline (fig. 4.3). Placing it there means a hit returns in sub-millisecond time, without loading the offline sweep CSV or initializing any ROS2 infrastructure. The `lookup()` method proceeds in three stages.

Stage one, exact context match. The query context is compared against every stored case key-by-key, accepting a match when all eight context values agree within a floating-point tolerance. The comparison is the static helper `_context_matches()`, shared with duplicate detection. The match is exact, not approximate: the case base answers *repeat* queries with an identical context vector, whereas approximate, nearest-neighbour interpolation is the separate role of the k-KNN warm-start (section 5.7.1). Exact matching guarantees that a recalled policy was optimal for the precise context now being asked.

Stage two, constraint re-filtering. Among the context matches, only those whose achieved cycle time and required force satisfy the *current* query's limits, $T \leq T_{\max}$ and $F_{\text{req}} \leq F_{\max}/S$, are retained. This step is necessary because the same context may be queried under different limits across deployments: a policy that was optimal under a generous $T_{\max}$ may violate a tighter one, and re-filtering ensures only presently-feasible cases survive.

Stage three, best-energy selection. Among the surviving cases the one with the lowest achieved energy is returned as the instant recommendation. If no case matches the context, or if every match fails the current limits, `lookup()` returns nothing and the pipeline falls through to the warm-start gate and the BO loop, exactly the *miss* branch of the design pipeline.

5.9.3 Duplicate detection

A new case is a duplicate of an existing one when they share both the same eight-dimensional context (again within $\varepsilon = 10^{-6}$ through `_context_matches()`) and the same two-dimensional policy. On a duplicate, a dominance comparison on achieved energy decides the outcome: the lower-energy case is kept, replacing the stored entry if the newcomer improves on it and discarding the newcomer otherwise.

Crucially, the identity for duplicate detection is the (context, policy) pair, not the context alone. Two different policies for the same context, produced by separate runs under different constraint bounds, are therefore stored as distinct entries rather than collapsed. This is what lets the constraint re-filtering of section 5.9.2 select the appropriate policy at query time: the store retains at most one entry per unique (context, policy) pair, always at its best observed energy, while still holding several admissible policies per context when they exist.

5.9.4 Closing the four-R cycle

With the optimization loop and the case base in place, the implementation realizes the four-R cycle of section 4.5 end to end, each R mapped to a concrete component of this chapter.

Retrieve and *Reuse* appear in two forms, ordered cheapest first. For a context already solved under compatible limits, the case-base `lookup()` of section 5.9.2 retrieves the stored solution and reuses it directly, returning an instant policy at zero simulation cost. For a novel context, retrieval and reuse instead operate over the offline experience memory: the context feasibility gate retrieves the k nearest offline anchors and aggregates them into the warm-start (section 5.7). *Revise* is the constrained BO loop of section 5.8. *Retain*is the case-base `add()` of section 5.9.1, invoked only on a verified feasible recommendation, which writes the solved pair back to the store and so makes the next identical query a Stage-one lookup hit.

The cycle is therefore a learning loop, not merely a control flow: each solved query enlarges the case base, shifting future work from expensive simulation-based revision toward cheap memory-based recall, while novel contexts continue to be served efficiently by the warm-started loop. The implementation spans the same continuum the design promised, from instant recall for repeated contexts to informed exploration for unseen ones.

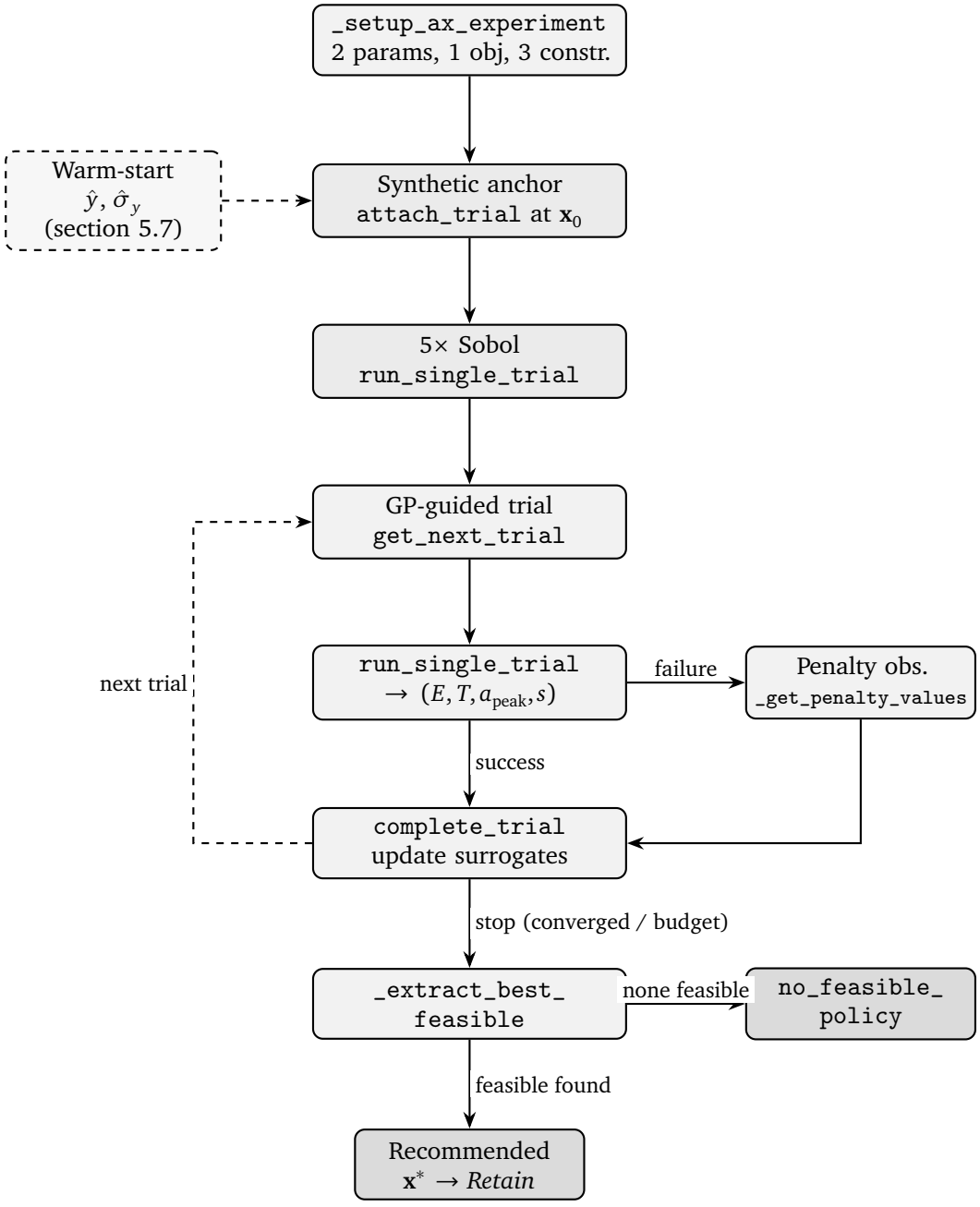


Figure 5.9: Control flow of the constrained BO loop. The experiment is set up once. The seed phase attaches the synthetic warm-start anchor at $\mathbf{x}_0$ and then executes five Sobol points, after which the GP-guided phase proposes, executes, and reports trials until early stopping or budget exhaustion. Failed trials are reported through the penalty scheme rather than dropped, and policy extraction returns the lowest-energy observed-feasible trial or reports honest infeasibility.

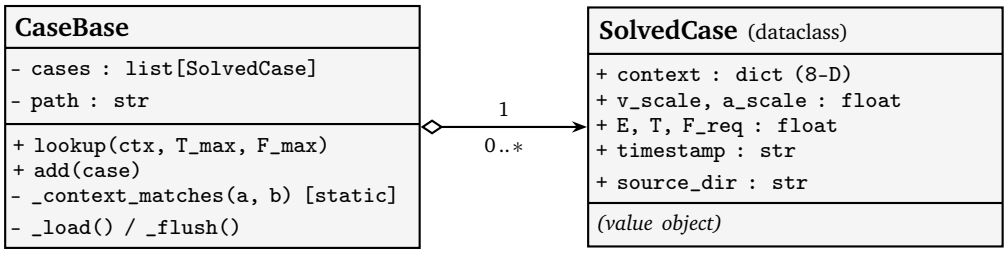


Figure 5.10: Class structure of the case-based reasoning layer. `CaseBase` aggregates a list of `SolvedCase` value objects and persists them to a single JSON file. `lookup` serves the *Retrieve* / *Reuse* query and `add` performs *Retain* with duplicate detection; `_context_matches` is the shared identity test used by both.

Chapter 6 Evaluation

The Design chapter set out the optimizer agent, its requirements, problem formulation, and pipeline architecture, and the Implementation chapter realized that design in software. This chapter evaluates the result. The agent's behavior is defined less by a single output than by how it routes each task through the sequence of gates and decision stages of fig. 4.3. The Evaluation is done by running a set of test cases with varying contexts and observing how the pipeline handles each. Every test case is built to drive the pipeline down one specific branch, isolating a single decision: whether an out-of-scope input is rejected before any trial budget is spent, whether a statically infeasible grasp is refused, whether an admissible context is warm-started and optimized, and whether the returned policy respects the user-defined limits.

Because the requirement categories of table 4.1 are themselves stated as behaviors the agent must exhibit, a context that exercises a given branch is at the same time a test of the requirement that branch realizes. The test cases map onto the pipeline: a rejected query exercises input rejection and the static-grasp check (C-4, C-1). An admitted query that is solved exercises the warm-started, constrained optimization (D-1, D-2, E-2, E-3), and an admitted query for which the loop finds nothing feasible exercises honest infeasibility (C-3).

The remainder of the chapter first consolidates all test cases in a single overview (section 6.1), then treats in turn each group of contexts that share a pipeline branch: input and static rejection (sections 6.2 and 6.3), the gate's infeasibility transparency to the cycle-time deadline (section 6.4), and constrained optimization on admissible contexts (section 6.5).

6.1 Evaluation overview

Table 6.1 lists the ten test cases, the context condition, the furthest gate each reaches in the pipeline, and the observed outcome.

The ten test cases fall into three behavioral groups. Five exercise the deterministic rejection paths, four on the workspace bounds (T3–T6) and one on the static-hold floor (T8). Two share an identical context and differ only in the deadline, isolating what the gate does and does not check (T1, T2). The remaining three carry an admissible context through the full constrained optimization (T7, T9, T10).

For reference, table 6.2 records the concrete parameter values behind each test case: the full eight-dimensional context vector together with the operating limits supplied to the pipeline.

Table 6.1: Test cases, the condition each exercises, the furthest gate each reaches in the pipeline, and the observed outcome.

Test case	Condition tested	Furthest gate	Outcome
T1	Nominal in-bounds context	Pass + warm-start → BO	Solved
T2	As T1, deadline $T_{\max} = 18\,\mathrm{s}$	Pass + warm-start → BO	No feasible policy
T3	Mass 5 kg, above bound	Input rejection	Rejected (mass)
T4	Friction 1.5, above bound	Input rejection	Rejected (friction)
T5	Mass 4.5 kg and friction 0.05	Input rejection	Rejected (mass and friction)
T6	Pick pose outside reach	Input rejection	Rejected (pick-x)
T7	Light object, generous deadline	Pass + warm-start → BO	Solved
T8	Heavy object, low $F_{\max}$	Static-hold rejection	Rejected (static)
T9	Extreme boundary context	Pass + warm-start → BO	Solved
T10	Tight $F_{\max}$ and deadline	Pass + warm-start → BO	Solved

Table 6.2: Full context of each test case: the eight-dimensional context vector $\mathbf{z}$ (first eight value columns) together with the user-specified operating limits. Positions x, y in m, orientations θ in °, mass m in kg, friction μ dimensionless, deadline $T_{\max}$ in s, force limit $F_{\max}$ in N; subscripts p and q denote the pick and place poses.

Test case	x_p	y_p	θ_p	x_q	y_q	θ_q	m	μ	$T_{\max}$	$F_{\max}$
T1	−0.20	0.90	0	0.25	0.65	0	2.0	0.30	30	70
T2	−0.20	0.90	0	0.25	0.65	0	2.0	0.5	18	200
T3	−0.20	0.90	0	0.25	0.65	0	5.0	0.50	30	200
T4	−0.20	0.90	0	0.25	0.65	0	1.0	1.50	30	200
T5	−0.20	0.90	0	0.25	0.65	0	4.5	0.05	30	200
T6	2.00	3.00	0	0.25	0.65	0	1.0	0.50	30	200
T7	−0.30	1.00	0	0.15	0.55	0	0.5	0.60	60	200
T8	−0.20	0.90	0	0.25	0.65	0	2.5	0.30	30	70
T9	0.04	1.09	0	0.39	0.79	0	2.9	0.25	35	200
T10	−0.20	0.90	0	0.25	0.65	0	2.5	0.40	25	70

6.2 Input rejection

The first stage of the online pipeline is the deterministic physics-and-input gate (section 5.6.1), whose workspace check verifies that all eight context dimensions lie within the bounds of table 5.4 before any trial budget is committed (C-4). Four test cases drive this check, each with one or more out-of-bounds dimensions.

T3 sets the object mass to 5 kg, above the 0.1 kg to 3.0 kg payload bound, and is rejected on the mass dimension. T6 places the pick pose at (2.0 m, 3.0 m), far outside the reachable pick zone $x_\mathrm{p} \in [-0.4, 0.05]$ m, and is rejected on the pick-x dimension. Both rejections occur at zero simulation cost, as required, and name the binding dimension so that the caller can correct the query.

T4 and T5 exercise the same check from the friction side. T4 sets the friction coefficient to $\mu = 1.5$, above the $[0.2, 0.8]$ bound, and is rejected on the friction dimension. T5 violates two bounds at once, a 4.5 kg mass together with a friction of 0.05, and is rejected with both the mass and the friction dimension flagged.

6.3 Static-hold rejection

A query that clears the workspace check then faces the static-Coulomb check, the second half of the physics-and-input gate (C-1). It evaluates the static-hold floor of eq. (4.12), $S\,m\,g/(n\,\mu) \leq F_{\max}$, with safety factor $S = 2.0$ and $n = 2$ contact faces, and refuses any grasp that the gripper could not sustain even at rest, since no choice of motion can rescue a grasp that already fails under gravity alone.

T8 is constructed to fail precisely here: a 2.5 kg object at friction $\mu = 0.3$ with a gripper limit of only $F_{\max} = 70\,\text{N}$. The required static force is $S\,m\,g/(n\,\mu) = 2.0 \cdot 2.5 \cdot 9.81/(2 \cdot 0.3) \approx 81.8\,\text{N}$, which exceeds the 70 N limit, so the query is rejected as physics-infeasible before any trial is run. The contrast with the nominal context T1 is instructive: T1 carries the same friction and gripper limit but a lighter 2.0 kg object, giving a static requirement of 65.4 N that clears the 70 N limit and lets T1 pass the gate (section 6.4). The two contexts are separated on exactly the physical quantity that determines whether a static hold is possible at all.

6.4 Infeasibility transparency due to cycle time constraint

A design choice that warrants explicit verification is what the physics-and-input gate is intentionally unable to check. The cycle-time deadline $T_{\max}$ and the dynamic force limit $F_{\max}$ are constraints of the optimization problem (D-1). They can only be evaluated from measured outcomes during execution and are therefore assessed inside the constrained BO loop. Consequently, they are not part of the gate, which checks only static feasibility and input validity. A context with an impossible deadline should therefore pass the gate and be rejected later by the optimization loop on the basis of genuine infeasibility (C-3), rather than by the gate itself.

T2 is an in-bounds, statically feasible context with a tight 18 s deadline. As expected, it passes the gate. However, none of the evaluated parameter settings reduce the measured cycle time below 18 s, meaning that no feasible policy exists. The optimization loop therefore returns the honest-infeasibility terminal (C-3) rather than a gate rejection. This contrasts with T1, a nominal context that is solved comfortably within its more relaxed 30 s deadline. Both contexts pass the same gate, but they receive different outcomes because of a constraint that is evaluated only within the optimization loop.

The optimization trace provides further insight. As cycle time decreases when the motion is executed more aggressively, the optimizer responded to the early results by increasing the velocity and acceleration scales in an attempt to satisfy the deadline. Toward the end of the run, most evaluations are concentrated in the high-velocity, high-acceleration region of the decision space, visible as the dense cluster in the upper-right corner of fig. 6.1. Even there, the fastest execution achieved a cycle time of approximately 18.5 s, which still did not meet the requirement. Moreover, the extreme corner of the search space was not executable: the trial with both scales at their maximum values failed in the motion planner. Since the deadline remained unattainable even at the limits of the decision space, the loop correctly declines to return a policy and reports the context as infeasible.

This layering keeps each feasibility question at the stage that can actually measure it. Static feasibility is decided analytically and cheaply at the gate, whereas the deadline, which depends on a measured cycle time, is decided only once the loop has run trials that produce that measurement.

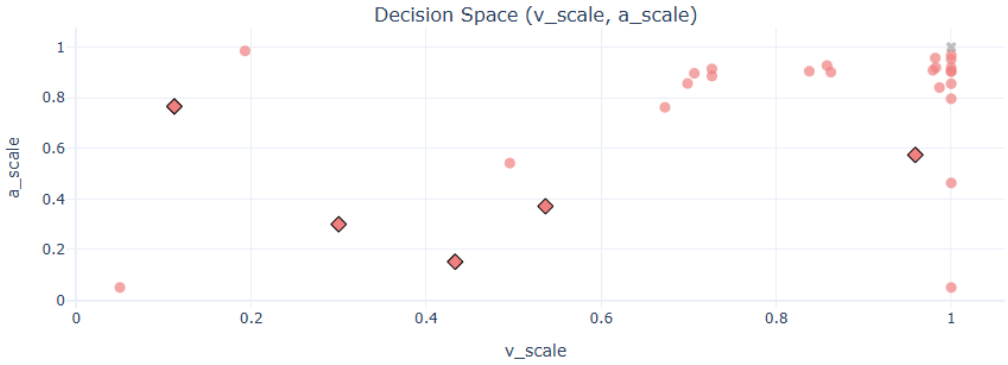


Figure 6.1: Decision-space exploration for T2 under the 18 s deadline. Each marker is one trial in the $(v_{\text{scale}}, a_{\text{scale}})$ plane. Seeking to satisfy the binding cycle-time constraint, the loop concentrates its later trials in the high-velocity, high-acceleration corner.

6.5 Constrained optimization on admissible contexts

Three further test cases, T7, T9, and T10, carry an admissible context all the way through the *Revise* step, the constrained BO loop (section 5.8). They exercise the agent's core function most directly: returning a simulation-verified, energy-minimized policy that respects the user-defined limits (D-1, D-2).

T7 pairs a light 0.5 kg object with a generous 60 s deadline, an easily feasible context that passes the gate and is solved to a successful policy by the loop. It establishes that the full path from gate through warm-start and optimization to case retention (section 5.9) operates end to end.

T9 is the one extreme case in the set: it places several context dimensions near the edge of their range at once, a near-maximal payload of 2.9 kg, a low friction of 0.25, and pick and place poses at the upper end of the reachable workspace. It nonetheless clears both halves of the gate, its static requirement of $S\,m\,g/(n\,\mu) \approx 113.8$ N sitting well within the 200 N limit, and the loop solves it cleanly: all 21 trials succeed, and the optimizer converges to a feasible policy at a velocity scale of 0.284 and an acceleration scale of 0.362, costing 337.1 J, about 5.5 % below the best warm-started seed. Neither outcome constraint binds: the cycle time of 21.7 s stays well inside the 35 s deadline, and the grip-force requirement of 62.1 N remains within its limit, leaving the optimizer free to minimize energy in a low-velocity, low-acceleration region. A context assembled from individually extreme but in-bounds values is thus handled no differently from a central one.

T10 tightens both active constraints at once. It carries the same 2.5 kg mass and 70 N limit as T8, but at a higher friction $\mu = 0.4$. The higher friction lowers the static requirement to $S\,m\,g/(n\,\mu) \approx 61.3$ N, below the 70 N limit, so unlike T8 the context clears the static gate and is admitted. The narrow remaining margin, however, leaves little dynamic headroom: the force constraint, derived from the peak acceleration through eq. (4.7), and the tight 25 s deadline both press on the loop at once. The optimizer must therefore trade energy against two binding outcome constraints rather than one, and converges to a feasible policy at 270.8 J, the energy cost reflecting the narrower feasible region left by the two active constraints.

Together, T7, T9, and T10 trace the loop across a range of admissible contexts, from an easily feasible one, through a context built from extreme but in-bounds values, to one where two constraints bind simultaneously. In each case the agent returns a feasible, simulation-verified policy and retains the solved context–policy pair for future recall, closing the four-R cycle of section 5.9.

Chapter 7 Conclusion and future outlook

This thesis set out to develop an autonomous agent that, given an observed task context and the user's operating limits, returns a simulation-verified, optimized parameterization for a pick-and-place skill. The Design chapter (chapter 4) posed this as a constrained, contextual, single-objective Bayesian optimization problem and provided the solution (fig. 4.3): a deterministic feasibility gate, a case-based recall stage, a memory-based warm-start, the constrained BO loop, and case retention. The Implementation chapter realized that pipeline in ROS2 and Isaac Sim, and the Evaluation chapter exercised it through ten test cases, each exploring a different branch of the pipeline. This chapter assesses what was achieved against the requirements of chapter 4, judges where the agent succeeds and where it falls short, and outlines the work that would follow.

7.1 Requirements fulfillment

The requirements of chapter 4 were stated as behaviors the agent must exhibit. The pipeline branches and the test cases that exercise them map back onto those requirements, so the evaluation is at the same time a check of whether each requirement was met. This section revisits the five categories in turn. Table 7.1 consolidates the outcome.

Functionality (A) is met across the trial lifecycle. The online cycle returns a decision policy $\mathbf{x}^* = (s_v, s_a)$ for a queried context (A-1), demonstrated by the solved cases TC7, TC9, and TC10 (section 6.5). The returned policy is never merely predicted: it is the lowest-energy policy among the trials actually executed and verified feasible in simulation (A-2). The Skill Trial Runner performs the autonomous reset–execute–read cycle without intervention (A-3), and the two monitor nodes derive energy, cycle time, the required grip force, and the success flag from the joint states, the object IMU, and the gripper-occupancy state (A-4).

Verification (B) is satisfied by construction. The monitor nodes are passive subscribers, decoupled from the control loop, so measurement does not perturb the process it measures (B-1). Reproducibility (B-2) is verified by the success of the optimization loop, which would have suffered under high measurement variants between different runs.

Safety and feasibility (C) is the category that the evaluation probes most directly. The static-hold floor of eq. (4.12) refuses a grasp that could not be sustained even at rest (C-1), verified by TC8 (section 6.3). The cycle-time and grip-force limits enter the loop as hard outcome constraints (C-2): TC10 solves a context in which both press at once, while TC2 shows the deadline binding to the point of infeasibility. When no feasible trial exists, the loop returns the honest-infeasibility terminal and names the binding constraint rather than an unsafe recommendation (C-3), verified by TC2 (section 6.4). Out-of-scope inputs are rejected at the gate at zero trial cost (C-4), verified by TC3–TC6 (section 6.2).

Optimization (D) is realized by the constrained BO loop. Cycle time, grip force, and success enter

the acquisition as constraints through the feasibility-weighted noisy expected-improvement criterion over the four GP surrogates (D-1), with energy as the sole objective minimized strictly within the feasible region (D-2). Sample efficiency (D-3) follows from the two-dimensional decision space, the KNN warm-start anchor, and early stopping, which together let the loop converge within the thirty-trial budget. TC9, for instance, is solved in 21 trials.

Adaptivity and generalization (E) is provided by the contextual formulation and the four-R cycle. The agent conditions on the eight-dimensional context and searches only the two-dimensional decision space (E-1). For an unseen context the warm-started loop interpolates from neighboring offline observations rather than re-optimizing from scratch (E-2), demonstrated by TC9, whose individually extreme but in-bounds context is handled no differently from a central one. The warm-start itself aggregates the k nearest offline anchors into the synthetic anchor that seeds the loop (E-3), and the offline characterization together with the material pre-filter screens infeasible contexts before any online budget is spent (E-4). Finally, the case base recalls a compatible stored policy at zero cost (E-5) and persists each solved context–policy pair (E-6), closing the cycle so that the system shifts from simulation-based revision toward memory-based recall as it is used.

7.2 Assessment of the agent

Taken together, the requirements review and the test cases support a clear judgment: the agent succeeds at its design goal, which is to be a safe, honest, and sample-efficient recommender for a constrained optimization problem. Its most valuable property is trustworthiness. The agent refuses rather than guesses: it rejects out-of-scope and statically infeasible queries at zero cost (TC3–TC6, TC8), and it reports honest infeasibility with the binding constraint named when the loop can find no feasible policy (TC2). For an agent that is meant to hand a parameterization to a human operator, knowing when *not* to answer is as important as the answer itself, and the layered feasibility design delivers this by deciding each feasibility question at the cheapest stage that can actually measure it.

On the optimization side the agent also performs as intended. On every admissible context it converged within the trial budget to a feasible, simulation-verified policy that respects the user limits, and it reduced energy while doing so: TC9 converged to a policy roughly 5.5 % below its best warm-started seed, and TC10 found a feasible policy under two simultaneously binding constraints. The contextual formulation generalized as designed, serving an individually extreme but in-bounds context (TC9) no differently from a central one, and the decoupled architecture of passive monitors behind the Skill Trial Runner mediator gave clean, reproducible measurements without coupling the optimizer to simulation control.

The agent's limitations are equally clear, and both stem from deliberate design compromises rather than from defects. The first concerns the grasp. Because the PhysX contact grasp for the Robotiq 2F-85 could not be made reliable (section 4.4), the grasp is realized as a fixed joint and slip is never actually simulated. Feasibility of the hold rests entirely on the analytical Coulomb model applied to the observed peak acceleration (eq. (4.7)). This kept the measurement chain trustworthy and reproducible, but it means the grip-force constraint is a first-order model rather than a measured contact outcome, and effects such as finger compliance, partial slip, and torsional friction lie outside it. The second concerns the offline–online decomposition. Because the offline sweep was collected at a single fixed policy, the experience memory carries no information about how outcomes vary with the decision variables. It therefore serves only as a feasibility screen and a warm-start anchor, and the online loop must rediscover the decision landscape for each context rather than inheriting it from memory. Both compromises were the right ones for making the problem tractable and the measurements trustworthy, and both define the directions in which the agent would most naturally

be strengthened.

7.3 Future outlook

Four extensions follow directly from the assessment above.

Contact-based slip detection. The analytical slip model is a direct consequence of the unreliable PhysX contact grasp for the 2F-85. Should that contact solver be resolved in a future Isaac Sim release, the grasp could be realized through genuine contact friction rather than a fixed joint, and the success constraint would then carry the emergent slip that the present formulation has to reconstruct. The Coulomb-derived grip-force constraint could in turn be validated against a simulated contact outcome rather than standing in for it, and richer phenomena, partial slip, rolling, and in-hand rotation, would become observable to the optimizer.

Sampling the decision space. The decomposition's main cost is that the experience memory carries no information about how the outcomes vary with the decision variables. This is not a shortcoming of the surrogate model but of the sampling scope. Closing this gap requires sampling the decision variables jointly with the context during the offline phase, so that the memory captures the decision-dependent structure and can inform the online search itself rather than only feasibility screening and initialization. The obstacle is dimensionality. Adding the two decision dimensions to the eight context dimensions turns the sweep into a search over a ten-dimensional space, and at the per-trial simulation cost an exhaustive sweep at that dimension is intractable, the curse of dimensionality that motivated fixing the policy in the first place. The open question is therefore not which model to fit but how to sample. A space-filling design over the joint space, or an adaptive scheme that concentrates trials where the decision landscape actually varies and leaves flat regions coarsely covered, would let the offline phase capture decision-dependent structure within a feasible trial budget, without the exhaustive coverage that the full joint space would otherwise demand.

Sim-to-real transfer. The entire evaluation is conducted in simulation. The clearest next step is deployment on the physical UR30 and 2F-85, where energy would be measured from real electrical draw rather than integrated joint power, the wall-clock cycle-time constraint could be checked against the duration a deployment actually experiences, and the analytical grip-force constraint could be validated against real slip. Domain randomization over the contact and dynamics parameters during the offline sweep would help the warm-start transfer to hardware.

Benchmarking sample efficiency. The agent converges within its trial budget, but the evaluation establishes this behaviorally rather than comparatively. A controlled study against baselines, random search, a fixed grid, and a fixed nominal policy, would quantify the sample-efficiency advantage that motivates the choice of Bayesian optimization (D-3), and would measure how much the KNN warm-start and the growing case base reduce the trials needed as the system accumulates experience.

Table 7.1: Requirements of chapter 4 and how each was handled in the realized agent.

ID	Requirement	How it was handled
Functionality		
A1	Policy output per context	Online cycle returns $\mathbf{x}^* = (s_v, s_a)$, shown in TC7, TC9, TC10
A2	Validate before handoff	Policy extracted from executed feasible trials, not predicted
A3	Autonomous lifecycle	Skill Trial Runner `run_cycle()`, reset–execute–read
A4	KPI acquisition	Energy and slip monitor nodes derive E, T, F_{req}, s
Verification		
B1	Observational separation	Passive monitor nodes, decoupled from control loop
B2	Reproducible measurement	Deterministic motion planning, settled fixed-joint attach
Safety and feasibility		
C1	Static-grasp check	Static-hold floor at physics gate (eq. (4.12)), verified by TC8
C2	Execution feasibility	$T_{\max}$ and $F_{\max}/S$ as hard constraints, verified by TC2, TC10
C3	Honest infeasibility	Infeasibility terminal reports the binding constraint, verified by TC2
C4	Input rejection	Workspace and context bounds at zero-cost gate, verified by TC3–TC6
Optimization		
D1	Constraint-aware acquisition	Feasibility-weighted noisy EI over four GPs (Ax/BoTorch)
D2	Single objective	Energy minimized within the feasible region, shown in TC7, TC9, TC10
D3	Sample efficiency	2 D search, warm-start, early stop. Converges in budget (TC9: 21 trials)
Adaptivity and generalization		
E1	Contextual conditioning	8 D context held fixed, 2 D online search
E2	Generalize to unseen contexts	Warm-started loop interpolates offline neighbors, shown in TC9
E3	Memory-based warm-start	k-KNN offline anchors form the synthetic anchor
E4	Feasibility gating	Offline characterization and material pre-filter
E5	Case recall	Case-base lookup returns a compatible stored policy at zero cost
E6	Case retention	Solved $(\mathbf{z}, \mathbf{x}^*)$ pairs persisted, closing the four-R cycle

Bibliography

A. Björkelund, L. Edström, M. Haage, J. Malec, K. Nilsson, P. Nugues, S. G. Robertz, D. Störkle, A. Blomdell, R. Johansson, M. Linderoth, A. Nilsson, A. Robertsson, A. Stolt, & H. Bruyninckx, (2011). On the integration of skilled robot motions for productivity in manufacturing. *2011 IEEE International Symposium on Assembly and Manufacturing (ISAM)*, 1–9. https://doi.org/10.1109/ISAM.2011.5942366

Aamodt, A., & Plaza, E., (1994). Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI Communications*, *7*(1), 39–59. https://doi.org/10.3233/aic-1994-7104

Abu Alfeilat, H. A., Hassanat, A. B. A., Lasassmeh, O., Tarawneh, A. S., Alhasanat, M. B., Eyal Salman, H. S., & Prasath, V. B. S., (2019). Effects of distance measure choice on K-nearest neighbor classifier performance: A review. *Big Data*, *7*(4), 221–248. https://doi.org/10.1089/big.2018.0175

Ahmad, F., Mayr, M., Suresh-Fazeela, S., & Krüger, V., (2024). Adaptable recovery behaviors in robotics: A behavior trees and motion generators (btmg) approach for failure management, 1815–1822. https://www.scopus.com/inward/record.uri?eid=2-s2.0-85208238193&doi=10.1109%2FCASE59546.2024.10711715&partnerID=40&md5=c5f57f4ec2a86fb63ad97aeaae9b19ca

Ahmad, F., Mayr, M., & Krueger, V., (2023). Learning to adapt the parameters of behavior trees and motion generators (btmgs) to task variations. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 10133–10140. https://doi.org/10.1109/iros55552.2023.10341636

Ahmed, M. U., Begum, S., Olsson, E., Xiong, N., & Funk, P., (2010). Case-based reasoning for medical and industrial decision support systems. *Successful Case-based Reasoning Applications - I*, *305*, 7–52. https://doi.org/10.1007/978-3-642-14078-5{\textunderscore}2

Albore, A., Doose, D., Grand, C., Guiochet, J., Lesire, C., & Manecy, A., (2023). Skill-based design of dependable robotic architectures. *Robotics and Autonomous Systems*, *160*, 104318. https://doi.org/10.1016/j.robot.2022.104318

Andersen, Solund, & J. Hallam, (2014). Definition and initial case-based evaluation of hardware-independent robot skills for industrial robotic co-workers. *ISR/Robotik 2014; 41st International Symposium on Robotics*, 1–7.

Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E., (2020). BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization. *Advances in Neural Information Processing Systems 33*. http://arxiv.org/abs/1910.06403

Bédard, C., Lütkebohle, I., & Dagenais, M., (2022). Ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ros 2. *IEEE Robotics and Automation Letters*, *7*(3), 6511–6518.

Bellman, R. E., (1961). *Adaptive control processes: A guided tour*. Princeton University Press.

Berkenkamp, F., Krause, A., & Schoellig, A. P., (2021). Bayesian optimization with safety constraints: Safe and automatic parameter tuning in robotics. *Machine Learning*, *112*(10), 3713–3747. https://doi.org/10.1007/s10994-021-06019-1

Berkenkamp, F., Krause, A., & Schoellig, A. P., (2023). Bayesian optimization with safety constraints: Safe and automatic parameter tuning in robotics. *Machine learning*, *112*(10), 3713–3747. https://doi.org/10.1007/s10994-021-06019-1

Bianchi, R. A. C., Santos, P. E., Da Silva, I. J., Celiberto, L. A., & LOPEZ DE MANTARAS, R., (2018). Heuristically accelerated reinforcement learning by means of case-based reasoning and transfer learning. *Journal of Intelligent & Robotic Systems*, *91*(2), 301–312. https://doi.org/10.1007/s10846-017-0731-2

Blatman, G., Sudret, B., & Berveiller, M., (2007). Quasi random numbers in stochastic finite element analysis. *Mécanique & Industries*, *8*(3), 289–297. https://doi.org/10.1051/meca:2007051

Bottin, M., Boschetti, G., & Rosati, G., (2022). Optimizing cycle time of industrial robotic tasks with multiple feasible configurations at the working points. *Robotics*, *11*(1), 16. https://doi.org/10.3390/robotics11010016

Coleman, D., Sucan, I., Chitta, S., & Correll, N., (2014). Reducing the barrier to entry of complex robotic software: A MoveIt! case study. *Journal of Software Engineering for Robotics*, *5*(1), 3–16. https://doi.org/10.6092/JOSER_2014_05_01_p3

Cover, T., & Hart, P., (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, *13*(1), 21–27. https://doi.org/10.1109/tit.1967.1053964

De Blasi, S., Bahrami, M., Engels, E., & Gepperth, A., (2024). Safe contextual Bayesian optimization integrated in industrial control for self-learning machines. *Journal of Intelligent Manufacturing*, *35*(2), 885–903. https://doi.org/10.1007/s10845-023-02087-3

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T., (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182–197. https://doi.org/10.1109/4235.996017

Dige, N., & Diwekar, U., (2018). Efficient sampling algorithm for large-scale optimization under uncertainty problems. *Computers & Chemical Engineering*, *115*, 431–454. https://doi.org/10.1016/j.compchemeng.2018.05.007

Doerr, B., Ivan, T., & Krejca, M. S., (2025). Speeding up the nsga-ii with a simple tie-breaking rule. *Proceedings of the AAAI Conference on Artificial Intelligence*, *39*(25), 26964–26972.

Dries, D., Englert, P., & Toussaint, M., (2017). Constrained bayesian optimization of combined interaction force/task space controllers for manipulations. *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 902–907. https://doi.org/10.1109/ICRA.2017.7989111

Fitzgerald, T., McGreggor, K., Akgun, B., Thomaz, A., & Goel, A., (2015). Visual case retrieval for interpreting skill demonstrations. In E. Hüllermeier & M. Minor (Eds.), *Case-based reasoning research and development* (pp. 119–133). Springer International Publishing; Imprint: Springer. https://doi.org/10.1007/978-3-319-24586-7{\textunderscore}9

Fix, E., & Hodges, J. L., (1951). *Discriminatory analysis, nonparametric discrimination: Consistency properties* (tech. rep. No. 4) (Project 21-49-004). USAF School of Aviation Medicine, Randolph Field, Texas.

Frazier, P. I., (2018). *A tutorial on bayesian optimization*. arXiv: 1807.02811 [stat.ML]. https://arxiv.org/abs/1807.02811

Gardner, J. R., Kusner, M. J., Xu, Z. E., Weinberger, K. Q., & Cunningham, J. P., (2014). Bayesian optimization with inequality constraints. *Proceedings of the 31st International Conference on Machine Learning*, *32*, 937–945. https://proceedings.mlr.press/v32/gardner14.html

Garnett, R., (2023). *Bayesian optimization*. Cambridge University Press.

Gelbart, M. A., Snoek, J., & Adams, R. P., (2014). Bayesian optimization with unknown constraints. *Proceedings of the 30th Conference on Uncertainty in Artificial Intelligence (UAI)*, 250–259.

Ghzouli, R., Berger, T., Johnsen, E. B., Wasowski, A., & Dragule, S., (2023). Behavior trees and state machines in robotics applications. *IEEE Transactions on Software Engineering*, *49*(9), 4243–4267. https://doi.org/10.1109/TSE.2023.3269081

Görner, Robert Haschke, Helge Ritter, & Jianwei Zhang, (2019). Moveit! task constructor for task-level motion planning.

Greenhill, S., Rana, S., Gupta, S., Vellanki, P., & Venkatesh, S., (2020). Bayesian optimization for adaptive experimental design: A review. *IEEE Access*, *8*, 13937–13948. https://doi.org/10.1109/ACCESS.2020.2966228

Heuss, L., Blank, A., Dengler, S., Zikeli, G. L., Reinhart, G., & Franke, J., (2019). Modular robot software framework for the intelligent and flexible composition of its skills. In F. Ameri, K. E. Stecke, G. von Cieminski, & D. Kiritsis (Eds.), *Advances in production management systems. production management for the factory of the future* (pp. 248–256, Vol. 566). Springer International Publishing. https://doi.org/10.1007/978-3-030-30000-5{\textunderscore}32

Heuss, L., Gonnermann, C., & Reinhart, G., (2022). An extendable framework for intelligent and easily configurable skills-based industrial robot applications. *The International Journal of Advanced Manufacturing Technology*, *120*(9-10), 6269–6285. https://doi.org/10.1007/s00170-022-09071-w

Homem, T. P. D., Santos, P. E., Reali Costa, A. H., Da Costa Bianchi, R. A., & LOPEZ DE MANTARAS, R., (2020). Qualitative case-based reasoning and learning. *Artificial Intelligence*, *283*, 103258. https://doi.org/10.1016/j.artint.2020.103258

Hou, T., Nuyens, D., Roels, S., & Janssen, H., (2019). Quasi-monte carlo based uncertainty analysis: Sampling efficiency and error estimation in engineering applications. *Reliability Engineering & System Safety*, *191*, 106549. https://doi.org/10.1016/j.ress.2019.106549

Ichnowski, J., Avigal, Y., Liu, Y., & Goldberg, K., (2022). Gomp-fit: Grasp-optimized motion planning for fast inertial transport. *2022 international conference on robotics and automation (ICRA)*, 5255–5261.

International Organization for Standardization, (1998). *ISO 9283:1998 manipulating industrial robots — performance criteria and related test methods*. International Organization for Standardization. Geneva, Switzerland.

Isaac Lab, (2026). *Isaac lab teleoperation with franka + 2f-85 gripper does not work well* [GitHub issue #4038]. Retrieved June 8, 2026, from https://github.com/isaac-sim/IsaacLab/issues/4038

Kim, C. M., Danielczuk, M., Huang, I., & Goldberg, K., (2022). Ipc-graspsim: Reducing the sim2real gap for parallel-jaw grasping with the incremental potential contact model. *2022 International Conference on Robotics and Automation (ICRA)*, 6180–6187. https://doi.org/10.1109/icra46639.2022.9811777

Kirschner, Mojmír Mutný, Nicole Hiller, Rasmus Ischebeck, & Andreas Krause, (2019). Adaptive and safe bayesian optimization in high dimensions via one-dimensional subspaces.

Krause, A., & Ong, C., (2011). Contextual gaussian process bandit optimization. *Advances in neural information processing systems*, *24*.

Kucherenko, S., Albrecht, D., & Saltelli, A., (2015). Exploring multi-dimensional spaces: A comparison of latin hypercube and quasi monte carlo sampling techniques. https://arxiv.org/pdf/1505.02350

Kupcsik, A., Deisenroth, M. P., Peters, J., Ai Poh, L., Vadakkepat, P., & Neumann, G., (2017). Model-based contextual policy search for data-efficient generalization of robot skills. *Artificial Intelligence*, *247*, 415–439. https://doi.org/10.1016/j.artint.2014.11.005

Kurscheid, S., & Daub, R., (2024). Automated robot skill parameterization based on task specifications. *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 01–04. https://doi.org/10.1109/ETFA61755.2024.10711102

Letham, B., Karrer, B., Ottoni, G., & Bakshy, E., (2019). Constrained bayesian optimization with noisy experiments. *Bayesian Analysis*, *14*(2), 495–519. https://doi.org/10.1214/18-BA1110

Liu, A., Liu, H., Yao, B., Xu, W., & Yang, M., (2018). Energy consumption modeling of industrial robot based on simulated power data and parameter identification. *Advances in Mechanical Engineering*, *10*(5), 1687814018773852.

López de Mántaras, R., McSherry, D., Bridge, D., Leake, D., Smyth, B., Craw, S., Faltings, B., Maher, M. L., Cox, M. T., Forbus, K., Keane, M., Aamodt, A., & Watson, I., (2005). Retrieval, reuse, revision and retention in case-based reasoning. *The Knowledge Engineering Review*, *20*(3), 215–240. https://doi.org/10.1017/S0269888906000646

Martucci, G., Bimbo, J., Prattichizzo, D., & Malvezzi, M., (2020). Maintaining stable grasps during highly dynamic robot trajectories. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 9198–9204. https://doi.org/10.1109/iros45743.2020.9341575

Mayr, M., Rovida, F., & Krueger, V., (2023). Skiros2: A skill-based robot control platform for ros. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 6273–6280. https://doi.org/10.1109/IROS55552.2023.10342216

Metzen, J. H., Fabisch, A., & Hansen, J., (2015). Bayesian optimization for contextual policy search. *Proceedings of the 2nd Machine Learning in Planning and Control of Robot Motion Workshop (MLPC), IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Mitrevski, A., Ploger, P. G., & Lakemeyer, G., (2020). Representation and experience-based learning of explainable models for robot action execution. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5641–5647. https://doi.org/10.1109/iros45743.2020.9341470

Mittal, M., Roth, P., Tigue, J., Richard, A., Zhang, O., Du, P., Serrano-Muñoz, A., Yao, X., Zurbrügg, R., Rudin, N., et al., (2025). Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*.

Mnyusiwalla, H., Triantafyllou, P., Sotiropoulos, P., Roa, M. A., Friedl, W., Sundaram, A. M., Russell, D., & Deacon, G., (2020). A bin-picking benchmark for systematic evaluation of robotic pick-and-place systems. *IEEE Robotics and Automation Letters*, *5*(2), 1389–1396.

Nakazawa, N., Kim, I.-h., Inooka, H., & Ikeura, R., (2001). Force control of a robot gripper based on human grasping schemes. *Control Engineering Practice*, *9*(7), 735–742. https://doi.org/10.1016/s0967-0661(01)00038-7

Nguyen, D. H., Schneider, T., Duret, G., Kshirsagar, A., Belousov, B., & Peters, J., (2024). *Tacex: Gelsight tactile simulation in isaac sim – combining soft-body and visuotactile simulators*.

NVIDIA, (2023). *Isaac Sim* (Version 6.0.0). https://github.com/isaac-sim/IsaacSim

NVIDIA, (2026). *Tutorial 10: Rig closed-loop structures* [Isaac Sim documentation, v6.0.0]. Retrieved June 8, 2026, from https://docs.isaacsim.omniverse.nvidia.com/6.0.0/robot_setup_tutorials/rig_closed_loop_structures.html

NVIDIA Developer Forums, (2026a). *Controlling a robotiq hand-e gripper* [Isaac Sim community forum thread]. Retrieved June 8, 2026, from https://forums.developer.nvidia.com/t/controlling-a-robotiq-hand-e-gripper/337309

NVIDIA Developer Forums, (2026b, March). *Cube slips away when trying to pick up cube with a claw gripper* [Isaac Sim community forum thread]. Retrieved June 8, 2026, from https://forums.developer.nvidia.com/t/cube-slips-away-when-trying-to-pick-up-cube-with-a-claw-gripper/363033

NVIDIA Developer Forums, (2026c, February 12). *Robotiq 2f-85 gripper issues: No actuation and collision problems (custom URDF)* [Isaac Sim community forum thread]. Retrieved June 8, 2026, from https://forums.developer.nvidia.com/t/robotiq-2f-85-gripper-issues-no-actuation-and-collision-problems-custom-urdf/360452

Pantano, M., Eiband, T., & Lee, D., (2022). Capability-based frameworks for industrial robot skills: A survey. https://doi.org/10.48550/ARXIV.2203.00538

Pedersen, M. R., Nalpantidis, L., Andersen, R. S., Schou, C., Bøgh, S., Krüger, V., & Madsen, O., (2016). Robot skills for manufacturing: From concept to industrial deployment. *Robotics and Computer-Integrated Manufacturing*, *37*, 282–291. https://doi.org/10.1016/j.rcim.2015.04.002

Profanter, Breitkreuz, Rickert, & Knoll, (2019). A hardware-agnostic opc ua skill model for robot manipulators and tools. *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1061–1068. https://doi.org/10.1109/ETFA.2019.8869205

Ram, A., Arkin, R. C., Moorman, K., & Clark, R. J., (1997). Case-based reactive navigation: A method for on-line selection and adaptation of reactive robotic control parameters. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, *27*(3), 376–394. https://doi.org/10.1109/3477.584946

Rasmussen, C. E., & Williams, C. K. I., (2005). *Gaussian processes for machine learning*. The MIT Press. https://doi.org/10.7551/mitpress/3206.001.0001

Robotiq Inc., (2026). *2F-85 and 2F-140 adaptive grippers — product sheet*. Robotiq Inc. Retrieved June 13, 2026, from https://robotiq.com/hubfs/Product-sheets/Adaptive%5C%20Grippers/Product-sheet-Adaptive-Grippers-EN.pdf

Rodrigues, É., (2018). Combining minkowski and chebyshev: New distance proposal and survey of distance metrics using k-nearest neighbours classifier. *Pattern Recognition Letters*, *110*, 66–71. https://doi.org/10.1016/j.patrec.2018.03.021

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N., (2016). Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, *104*(1), 148–175. https://doi.org/10.1109/jproc.2015.2494218

Simon Bøgh, Oluf Skov Nielsen, Mikkel Rath Pedersen, Volker Krüger, & Ole Madsen, (2012). Does your robot have skills? *Proceedings of the 43rd International Symposium on Robotics*. https://vbn.aau.dk/en/publications/does-your-robot-have-skills

Sloan, I. H., & Woźniakowski, H., (1998). When are quasi-monte carlo algorithms efficient for high dimensional integrals? *Journal of Complexity*, *14*(1), 1–33. https://doi.org/10.1006/jcom.1997.0463

Snoek, J., Larochelle, H., & Adams, R. P., (2012). Practical bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, *25*, 2951–2959.

Steinmetz, F., & Weitschat, R., (2016). Skill parametrization approaches and skill architecture for human-robot interaction. *2016 IEEE International Conference on Automation Science and Engineering (CASE)*, 280–285. https://doi.org/10.1109/coase.2016.7743419

Stoop, P., Ratnayake, T., & Toffetti, G., (2024). A method for multi-robot asynchronous trajectory execution in moveit2. *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 17694–17700. https://doi.org/10.1109/ICRA57147.2024.10611498

Sun, B., & Chen, H., (2021). A survey of k nearest neighbor algorithms for solving the class imbalanced problem. *Wireless Communications and Mobile Computing*, *2021*(1), 5520990. https://doi.org/10.1155/2021/5520990

Sun, G., Sawaragi, T., Horiguchi, Y., & Nakanishi, H., (2014). Knowledge-intensive teaching assistance system for industrial robots using case-based reasoning and explanation-based learning. *IFAC Proceedings Volumes*, *47*(3), 4535–4540. https://doi.org/10.3182/20140824-6-za-1003.01492

Syriopoulos, P. K., Kalampalikis, N. G., Kotsiantis, S. B., & Vrahatis, M. N., (2025). Knn classification: A review. *Annals of Mathematics and Artificial Intelligence*, *93*(1), 43–75. https://doi.org/10.1007/s10472-023-09882-x

Tesch, M., Schneider, J., & Choset, H., (2013). Expensive multiobjective optimization for robotics. *2013 ieee international conference on robotics and automation*, 973–980.

Universal Robots A/S, (2026). *UR30 — technical specifications and datasheet*. Universal Robots A/S. Retrieved June 13, 2026, from https://www.universal-robots.com/products/ur30/

AI-Based Tools

AI-Based Tool	Last Date of Access	Use Case	Scope	Remarks
ChatGPT	17.06.2026	Text generation, text revision, restructuring, formulation support, and language improvement	Entire thesis	Generated text was reviewed, adapted, and finalized by the author
Claude	17.06.2026	Code generation, debugging, implementation support, and code explanations	Entire thesis	Generated code was reviewed, adapted, and tested by the author
DeepL Translator Free	17.06.2026	Translation of the abstract	Abstract	Translated text was reviewed and revised by the author

Glossary

Ax	An open-source framework for adaptive experimentation built on top of BoTorch, providing a higher-level interface for managing the optimization loop, trial scheduling, and parameter configuration
BoTorch	An open-source library for Bayesian optimization built on top of PyTorch, providing modular components for probabilistic surrogate models, Monte Carlo-based acquisition functions, and optimizers, with first-class support for Gaussian process models via GPyTorch
Isaac Sim	A GPU-accelerated, physics-based robotics simulation platform developed by NVIDIA and built on the Omniverse framework, used for simulating robot dynamics, sensors, and contact physics in physically accurate virtual environments
JSON	JavaScript Object Notation, a lightweight text-based data-interchange format commonly used for structured data exchange
MoveIt2	An open-source motion planning framework for ROS2, providing capabilities for kinematics, trajectory planning, collision checking, and execution control for robotic manipulators
Skill Trial Runner	ROS2 node that orchestrates each optimization trial: it resets the monitoring accumulators, triggers skill execution, reads back the resulting key performance indicators, and forwards one observation to the optimizer
Sobol sequence	A low-discrepancy quasi-random sequence used in Quasi-Monte Carlo sampling to cover a multidimensional input space more uniformly than pseudo-random sampling

Appendix A Appendix

A.1 Online Optimization Trials

Each table below lists the full sequence of online trials for one evaluation context, comprising the seed (Sobol) phase and the constrained Bayesian optimization phase. Columns report the decision parameters (s_v, s_a), the measured energy E, cycle time T, and the post-trial required grip force F_{req}.

A.1.1 Test case 1

Trial	Phase	s_v	s_a	E [J]	T [s]	F_{req} [N]	Succ.
0	seed	0.300	0.300	315.34	21.51	36.01	✓
1	seed	0.550	0.234	293.87	21.71	35.71	✓
2	seed	0.384	0.637	310.48	19.61	37.93	✓
3	seed	0.070	0.324	392.34	31.33	34.97	✓
4	seed	0.858	0.905	466.63	18.40	39.80	✓
5*	seed	0.175	0.110	302.40	25.31	34.42	✓
6	opt	0.222	0.050	523.72	29.33	34.62	✓
7	opt	0.170	0.137	321.71	24.91	34.65	✓
8	opt	0.173	0.050	419.54	29.93	34.71	✓
9	opt	0.134	0.102	303.68	26.72	34.50	✓
10	opt	0.273	0.156	337.50	23.51	35.24	✓
11	opt	0.095	0.071	331.38	30.63	34.30	✓
12	opt	0.050	0.125	424.05	38.44	34.47	✓
13	opt	0.155	0.105	869.42	26.72	34.88	✓
14	opt	0.194	0.115	344.48	25.32	34.59	✓
15	opt	0.115	0.101	416.74	27.82	34.40	✓
16	opt	0.128	0.081	336.06	28.11	34.30	✓
17	opt	0.128	0.107	349.28	27.02	34.56	✓
18	opt	0.182	0.117	379.75	25.42	34.83	✓
19	opt	0.190	0.091	307.31	26.01	34.61	✓
20	opt	0.162	0.158	305.20	25.01	34.62	✓

Table A.1: Online optimization trials for test case 1 ($m = 2\,\text{kg}$, $\mu = 0.3$, $T_{\max} = 30\,\text{s}$, $F_{\max} = 70\,\text{N}$, $S = 2$). Pick at $(-0.2, 0.9)$, place at $(0.25, 0.65)$. The row marked $\star$ is the recommended configuration.

A.1.2 Test case 2

Trial	Phase	s_v	s_a	E [J]	T [s]	F_{req} [N]	Succ.
0	seed	0.300	0.300	415.92	21.42	21.19	✓
1	seed	0.959	0.574	423.95	19.31	23.33	✓
2	seed	0.433	0.152	266.63	23.21	21.74	✓
3	seed	0.112	0.766	324.74	25.62	21.00	✓
4	seed	0.536	0.371	297.47	20.11	22.73	✓
5	seed	1.000	1.000	–	–	–	×
6	opt	1.000	0.463	333.79	19.90	22.37	✓
7	opt	1.000	0.855	273.75	18.71	24.26	✓
8	opt	0.727	0.914	320.07	18.71	23.11	✓
9	opt	0.862	0.901	350.07	18.70	24.56	✓
10	opt	0.673	0.762	364.61	19.31	22.37	✓
11	opt	1.000	0.796	321.52	19.10	24.81	✓
12	opt	0.496	0.542	338.16	20.11	22.46	✓
13	opt	0.193	0.985	371.70	22.61	21.85	✓
14	opt	1.000	0.050	355.53	29.43	20.66	✓
15	opt	1.000	0.952	376.55	18.71	23.28	✓
16	opt	0.050	0.050	429.57	39.94	20.60	✓
17	opt	1.000	0.905	318.39	18.61	24.06	✓
18	opt	1.000	0.903	334.56	18.80	24.52	✓
19	opt	0.982	0.919	421.88	18.51	24.78	✓
20	opt	0.987	0.840	410.85	19.30	24.12	✓
21	opt	0.706	0.897	538.20	18.60	23.57	✓
22	opt	1.000	0.968	425.68	18.61	24.46	✓
23	opt	0.699	0.856	385.82	19.10	23.40	✓
24	opt	1.000	0.919	373.31	18.91	24.83	✓
25	opt	0.727	0.886	372.40	19.01	22.76	✓
26	opt	0.979	0.909	369.55	18.90	23.26	✓
27	opt	0.982	0.957	456.65	19.10	24.09	✓
28	opt	0.838	0.905	359.90	18.90	24.07	✓
29	opt	0.858	0.927	309.19	19.21	24.45	✓

Table A.2: Online optimization trials for test case 2 ($m = 2\,\text{kg}$, $\mu = 0.5$, $T_{\max} = 18\,\text{s}$, $F_{\max} = 200\,\text{N}$, $S = 2$). Pick at $(-0.2, 0.9)$, place at $(0.25, 0.65)$. The row marked ⋆ is the recommended configuration. Trials marked × were aborted by the skill (infeasible parameters) and contribute no objective observation.

A.1.3 Test case 7

Trial	Phase	s_v	s_a	E [J]	T [s]	F_{req} [N]	Succ.
0*	seed	0.300	0.300	299.63	23.22	4.35	✓
1	seed	0.701	0.225	438.47	23.62	4.37	✓
2	seed	0.478	0.723	422.29	20.72	4.53	✓
3	seed	0.205	0.327	302.63	24.22	4.40	✓
4	seed	0.962	0.825	–	–	–	×
5	seed	0.231	0.050	410.41	31.43	4.35	✓
6	opt	0.337	0.421	362.77	22.42	4.36	✓
7	opt	0.142	0.245	399.54	27.03	4.37	✓
8	opt	0.251	0.337	326.95	23.41	4.37	✓
9	opt	0.050	0.310	487.35	42.46	4.36	✓
10	opt	0.344	0.235	433.29	23.81	4.38	✓
11	opt	0.236	0.300	404.87	23.91	4.42	✓
12	opt	0.326	0.312	392.04	23.11	4.38	✓
13	opt	0.285	0.316	386.38	23.71	4.40	✓
14	opt	0.224	0.340	306.19	24.22	4.39	✓
15	opt	0.304	0.285	410.19	23.82	4.35	✓
16	opt	0.224	0.327	366.14	24.72	4.40	✓
17	opt	0.199	0.343	418.84	24.71	4.39	✓
18	opt	0.195	0.311	396.59	25.02	4.39	✓
19	opt	0.243	0.355	348.47	23.22	4.39	✓
20	opt	0.304	0.305	311.99	23.81	4.38	✓

Table A.3: Online optimization trials for test case 7 ($m = 0.5\,\text{kg}$, $\mu = 0.6$, $T_{\max} = 60\,\text{s}$, $F_{\max} = 200\,\text{N}$, $S = 2$). Pick at $(-0.3, 1)$, place at $(0.15, 0.55)$. The row marked $\star$ is the recommended configuration. Trials marked × were aborted by the skill (infeasible parameters) and contribute no objective observation.

A.1.4 Test case 9

Trial	Phase	s_v	s_a	E [J]	T [s]	F_{req} [N]	Succ.
0	seed	0.300	0.300	363.36	21.51	61.38	✓
1	seed	0.520	0.704	452.76	19.30	66.11	✓
2	seed	0.735	0.288	452.23	21.60	61.89	✓
3	seed	0.922	0.801	445.19	19.51	68.99	✓
4	seed	0.217	0.267	356.64	22.91	60.70	✓
5	seed	0.263	0.050	486.59	29.83	59.97	✓
6	opt	0.072	0.312	548.33	32.22	60.61	✓
7	opt	0.254	0.275	389.77	22.61	60.74	✓
8	opt	0.194	0.311	426.83	22.91	60.68	✓
9	opt	0.199	0.242	390.08	23.81	60.33	✓
10	opt	0.223	0.217	349.56	23.42	60.55	✓
11	opt	0.306	0.438	410.16	21.41	62.53	✓
12	opt	0.323	0.240	414.71	22.81	61.72	✓
13	opt	0.211	0.090	439.23	26.72	60.55	✓
14*	opt	0.284	0.362	337.08	21.71	62.11	✓
15	opt	0.270	0.423	352.99	21.71	61.56	✓
16	opt	0.279	0.360	342.96	22.12	61.39	✓
17	opt	0.285	0.385	352.85	21.70	62.61	✓
18	opt	0.295	0.343	384.41	21.91	61.87	✓
19	opt	0.251	0.203	403.59	23.51	60.70	✓
20	opt	0.241	0.236	417.93	23.02	60.87	✓

Table A.4: Online optimization trials for test case 9 ($m = 2.9\,\text{kg}$, $\mu = 0.25$, $T_{\max} = 35\,\text{s}$, $F_{\max} = 200\,\text{N}$, $S = 2$). Pick at $(0.04, 1.09)$, place at $(0.39, 0.79)$. The row marked $\star$ is the recommended configuration.

A.1.5 Test case 10

Trial	Phase	s_v	s_a	E [J]	T [s]	F_{req} [N]	Succ.
0	seed	0.300	0.300	386.00	21.91	33.03	✓
1	seed	0.626	0.722	402.07	19.22	35.10	✓
2	seed	0.327	0.481	311.89	20.71	33.50	✓
3	seed	0.217	0.801	294.06	21.51	33.40	✓
4	seed	0.990	0.092	398.27	26.52	32.43	✓
5	seed	0.190	0.722	336.25	22.41	33.24	✓
6	opt	0.158	0.849	370.38	23.42	32.99	✓
7	opt	0.240	0.790	362.29	21.41	33.87	✓
8	opt	0.208	0.840	323.07	22.01	33.67	✓
9	opt	0.208	0.783	342.31	22.21	33.60	✓
10	opt	0.213	0.813	310.37	22.02	34.24	✓
11*	opt	0.210	0.866	270.79	22.12	33.86	✓
12	opt	0.199	0.871	316.15	22.52	33.58	✓
13	opt	0.222	0.865	316.87	22.32	33.77	✓
14	opt	0.327	0.507	272.66	21.41	33.47	✓
15	opt	0.315	0.505	320.08	21.51	33.32	✓
16	opt	0.339	0.507	419.98	21.51	32.94	✓
17	opt	0.323	0.522	273.17	21.33	33.88	✓
18	opt	0.327	0.452	313.63	22.31	33.31	✓
19	opt	0.350	0.440	309.22	22.21	33.96	✓
20	opt	0.209	0.861	394.46	22.63	33.67	✓

Table A.5: Online optimization trials for test case 10 ($m = 2.5\,\text{kg}$, $\mu = 0.4$, $T_{\max} = 25\,\text{s}$, $F_{\max} = 70\,\text{N}$, $S = 2$). Pick at $(-0.2, 0.9)$, place at $(0.25, 0.65)$. The row marked $\star$ is the recommended configuration.

Declaration of Independent Work

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Garching, January 1, 2024

(Signature)